

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Bùi Khắc Trung - Trần Anh Tú

NGHIÊN CỨU PHƯƠNG PHÁP
XÂY DỰNG NEOBERT
CHO TIẾNG VIỆT

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 07/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Bùi Khắc Trung - Trần Anh Tú
22120396 - 22120400

NGHIÊN CỨU PHƯƠNG PHÁP
XÂY DỰNG NEOBERT
CHO TIẾNG VIỆT

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN
TS. Nguyễn Hồng Bửu Long - TS. Lương An Vinh

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Chúng tôi xin cam đoan khóa luận tốt nghiệp với đề tài “*Nghiên cứu phương pháp xây dựng NeoBERT cho tiếng Việt*” là công trình nghiên cứu của chính nhóm chúng tôi, được thực hiện dưới sự hướng dẫn của TS. Nguyễn Hồng Bửu Long và TS. Lương An Vinh.

Các số liệu, kết quả thực nghiệm được trình bày trong khóa luận là trung thực, do chính nhóm thực hiện và chưa từng được công bố trong bất kỳ công trình nào khác trước đây. Tất cả các tài liệu, công trình nghiên cứu liên quan được kế thừa trong khóa luận đều đã được trích dẫn nguồn gốc rõ ràng, đầy đủ trong danh mục tài liệu tham khảo.

Chúng tôi xin chịu hoàn toàn trách nhiệm trước Bộ môn, Khoa và Nhà trường về lời cam đoan này.

Thành phố Hồ Chí Minh, tháng 07 năm 2026

Nhóm sinh viên thực hiện

Bùi Khắc Trung

Trần Anh Tú

Lời cảm ơn

Lời đầu tiên, chúng tôi xin gửi lời cảm ơn sâu sắc đến thầy TS. Nguyễn Hồng Bửu Long và thầy TS. Lương An Vinh. Hai thầy đã cho phép nhóm thực hiện đề tài khóa luận này và dành nhiều thời gian, tận tâm hướng dẫn nhóm trong suốt quá trình nghiên cứu. Những kiến thức sâu rộng, những định hướng đúng đắn cùng các góp ý quý báu của các thầy đã giúp chúng tôi không chỉ hoàn thành tốt khóa luận mà còn trưởng thành hơn về tư duy nghiên cứu trong lĩnh vực Học sâu nói chung và Xử lý ngôn ngữ tự nhiên nói riêng.

Chúng tôi xin chân thành cảm ơn quý thầy cô Bộ môn Công nghệ tri thức đã tạo điều kiện thuận lợi về môi trường nghiên cứu trong quá trình nhóm thực hiện đề tài.

Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn đến Khoa Công nghệ thông tin, Trường Đại học Khoa học tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh vì đã mang lại một môi trường học tập và phát triển tuyệt vời. Chúng tôi cũng xin gửi lời cảm ơn đến tất cả quý thầy cô đã tận tâm truyền đạt những kiến thức nền tảng và những bài học quý giá trong suốt quá trình chúng tôi học tập tại trường.

Chúng tôi cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các anh chị khóa trên đã luôn đồng hành, động viên và giúp đỡ nhóm trong suốt quá trình học tập và nghiên cứu. Chính sự quan tâm và giúp đỡ này là nguồn động lực to lớn để chúng tôi luôn cố gắng phấn đấu hoàn thành thật tốt những mục tiêu đã đặt ra.

Mặc dù đã rất nỗ lực, khóa luận khó tránh khỏi những thiếu sót. Chúng

tôi rất mong nhận được sự thông cảm cùng những ý kiến đóng góp của quý thầy cô và các bạn để khóa luận được hoàn thiện hơn.

Một lần nữa, chúng tôi xin chân thành cảm ơn tất cả những người đã giúp đỡ nhóm trong quá trình thực hiện khóa luận tốt nghiệp này.

Thành phố Hồ Chí Minh, tháng 07 năm 2026

Nhóm sinh viên thực hiện

Bùi Khắc Trung – Trần Anh Tú



fit@hcmus

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ CƯƠNG KHOÁ LUẬN TỐT NGHIỆP NGHIÊN CỨU PHƯƠNG PHÁP XÂY DỰNG NEOBERT CHO TIẾNG VIỆT

1 THÔNG TIN CHUNG

Người hướng dẫn:

- TS. Nguyễn Hồng Bửu Long (Khoa Công nghệ Thông tin)
- TS. Lương An Vinh (Khoa Công nghệ Thông tin)

Nhóm Sinh viên thực hiện:

1. Bùi Khắc Trung (MSSV: 22120396)
2. Trần Anh Tú (MSSV: 22120400)

Loại đề tài: Nghiên cứu

Thời gian thực hiện: Từ 1/2026 đến 7/2026

2 NỘI DUNG THỰC HIỆN

2.1 Giới thiệu về đề tài

Trong lịch sử phát triển của lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), các mô hình dạng Encoder-only như BERT và RoBERTa từng giữ vai trò nền tảng trong các bài toán hiểu ngôn ngữ. Tuy nhiên, trong khi các dòng mô hình Decoder-only (LLMs) liên tục tiến hóa với quy mô dữ liệu và kiến trúc đột phá, các bộ Encoder gần như không có sự cải thiện đáng kể trong suốt vài năm qua. Phải đến năm 2025, sự ra đời của NeoBERT [1] mới thực sự thu hẹp khoảng cách này. NeoBERT tái định nghĩa khả năng của mô hình bằng cách tích hợp các cải tiến kiến trúc tiên tiến nhất như hàm kích hoạt SwiGLU, vị trí tương đối Rotary Position Embeddings (RoPE), và chuẩn hóa Pre-RMSNorm. Đặc biệt, NeoBERT tối ưu hóa tỷ lệ chiều sâu trên chiều rộng (optimal depth-to-width ratio) với 28 lớp và kích thước ẩn 768, cho phép mô hình đạt hiệu suất vượt trội trên chuẩn đánh giá MTEB so với các dòng BERT-large dù chỉ sở hữu 250 triệu tham số.

Mặc dù có khả năng suy luận mạnh và hỗ trợ độ dài ngữ cảnh lên tới 4.096 token, việc thích ứng NeoBERT cho tiếng Việt vẫn đối mặt với những thách thức lớn. Việc huấn luyện lại từ đầu (from scratch) trên quy mô hàng nghìn tỷ token như NeoBERT gốc đòi hỏi lượng tài nguyên tính toán rất lớn. Bên cạnh đó, các phương pháp tiếp tục tiền huấn luyện (Continued Pre-training) thông thường dễ dẫn đến hiện tượng mất thông tin (catastrophic forgetting), làm phá vỡ các đặc trưng biểu diễn quý giá mà mô hình đã học từ dữ liệu ban đầu.

Xuất phát từ bối cảnh trên, đề tài tập trung nghiên cứu phương pháp thích ứng ngôn ngữ cho NeoBERT bằng kỹ thuật SALT (Semantic Aware Linear Transfer) [2]. Ý tưởng cốt lõi của nghiên cứu là tái sử dụng (recycling) không gian véc-tơ từ các mô hình tiền huấn luyện tiếng Việt chất lượng cao như ViDeBERTa [3]. Thay vì khởi tạo ngẫu nhiên, phương pháp thực hiện chuyển đổi tuyến tính dựa trên độ tương đồng ngữ nghĩa: đối với mỗi token không trùng lặp, thuật toán sẽ

thiết lập các đường hồi quy tuyến tính riêng biệt dựa trên tập từ vựng chung. Quá trình này giúp ánh xạ chính xác các thành phần ngữ nghĩa từ không gian của ViDeBERTa sang không gian của NeoBERT.

Thông qua đó, nghiên cứu hướng tới việc xây dựng mô hình Vietnamese NeoBERT có khả năng xử lý ngữ cảnh dài, tốc độ suy luận nhanh và độ chính xác cao. Kết quả của đề tài không chỉ mang ý nghĩa về mặt hiệu suất mà còn khẳng định tính khả thi của việc tối ưu hóa tài nguyên huấn luyện. Cụ thể, việc tái sử dụng không gian biểu diễn từ các mô hình ngôn ngữ mạnh sẵn có mở ra một hướng tiếp cận tiết kiệm và hiệu quả để chuyển một kiến trúc Encoder thể hệ mới sang tiếng Việt, thay vì phải chịu chi phí rất lớn của việc huấn luyện lại từ đầu.

2.2 Mục tiêu đề tài

Trong bối cảnh các mô hình ngôn ngữ lớn đang đạt được những bước tiến vượt bậc về khả năng suy luận, các kiến trúc Encoder-only vẫn đóng vai trò nền tảng không thể thay thế trong các tác vụ trích xuất đặc trưng và biểu diễn ngữ nghĩa văn bản. Tuy nhiên, các mô hình Encoder dành cho tiếng Việt hiện nay chủ yếu dựa trên những kiến trúc cũ, thiếu đi sự tối ưu từ các kỹ thuật hiện đại và bị giới hạn ở độ dài ngữ cảnh ngắn. Việc huấn luyện lại các kiến trúc thể hệ mới như NeoBERT từ đầu cho tiếng Việt đòi hỏi nguồn lực tính toán rất lớn. Do đó, việc nghiên cứu một phương pháp thích ứng ngôn ngữ hiệu quả, ít tốn kém nhưng vẫn bảo toàn được năng lực của mô hình gốc là một nhu cầu cấp thiết.

Đề tài tập trung nghiên cứu việc khai thác và tái sử dụng không gian biểu diễn từ các mô hình ngôn ngữ tiền huấn luyện tiếng Việt sẵn có vào kiến trúc NeoBERT thông qua kỹ thuật SALT. Mục tiêu cụ thể là sử dụng các tập điểm neo (anchor) để tính toán các ánh xạ tuyến tính cục bộ riêng cho từng token, từ đó chuyển không gian biểu diễn của mô hình nguồn véc-tơ (tiêu biểu như ViDeBERTa) sang không gian véc-tơ của NeoBERT. Thông qua cách tiếp cận này, nghiên cứu hướng

đến việc xây dựng một mô hình Vietnamese NeoBERT có khả năng xử lý ngữ cảnh dài (lên tới 4.096 token) với tốc độ hội tụ nhanh và chi phí huấn luyện thấp hơn đáng kể so với các phương pháp truyền thống.

Kết quả của đề tài sẽ góp phần chứng minh hiệu quả của việc tái sử dụng không gian biểu diễn từ các mô hình ngôn ngữ mạnh sẵn có, giúp rút ngắn thời gian hội tụ và giảm hiện tượng mất thông tin (catastrophic forgetting). Nghiên cứu không chỉ cung cấp một bộ Encoder tiên tiến cho các tác vụ xử lý ngôn ngữ thực tế tại Việt Nam, mà còn đề xuất một quy trình thích ứng ngôn ngữ tối ưu, tạo tiền đề cho việc cập nhật và ứng dụng các kiến trúc mô hình ngôn ngữ thế hệ mới một cách linh hoạt, tiết kiệm và bền vững.

2.3 Phạm vi của đề tài

Đề tài tập trung vào việc nghiên cứu thực nghiệm phương pháp thích ứng mô hình ngôn ngữ thế hệ mới NeoBERT cho ngôn ngữ tiếng Việt bằng cách tái sử dụng không gian biểu diễn từ các mô hình tiền huấn luyện sẵn có. Phạm vi cụ thể bao gồm các thành phần sau:

Đối tượng nghiên cứu chính:

- *Kiến trúc mô hình NeoBERT*: Tập trung vào các cải tiến hiện đại như SwiGLU activation, RoPE, và cấu trúc Pre-RMSNorm với khả năng hỗ trợ ngữ cảnh lên đến 4.096 token.
- *Thuật toán SALT*: Nghiên cứu cơ chế trích xuất tập điểm neo (anchor token) dựa trên độ tương đồng ngữ nghĩa và phương pháp hồi quy tuyến tính (Linear Least Squares Transform) để ánh xạ không gian véc-tơ.
- *Mô hình nguồn véc-tơ (donor)*: Sử dụng không gian véc-tơ tiếng Việt chất lượng cao từ mô hình ViDeBERTa để làm cơ sở cho phép chiếu.

Tập dữ liệu thực nghiệm:

- *Ngữ liệu tiền huấn luyện*: Sử dụng các tập dữ liệu văn bản tiếng Việt quy mô lớn, được trích xuất và làm sạch từ Wikipedia và các trang tin tức tổng hợp nhằm phục vụ giai đoạn huấn luyện thích ứng (Language Adaptive Continual Pre-training).
- *Dữ liệu đánh giá*: Sử dụng các bộ benchmark tiêu chuẩn cho tiếng Việt như VLSP (phân loại văn bản, nhận dạng thực thể) và các tập dữ liệu con trong MTEB (Massive Text Embedding Benchmark) để đo lường khả năng biểu diễn ngữ nghĩa.

Các giới hạn và ràng buộc:

- *Về phương pháp*: Nghiên cứu không thực hiện huấn luyện mô hình NeoBERT từ đầu do hạn chế về tài nguyên tính toán; thay vào đó, tập trung tối ưu hóa quá trình chuyển đổi không gian véc-tơ.
- *Về tác vụ*: Đề tài tập trung vào các bài toán thuộc nhóm Hiểu ngôn ngữ tự nhiên (NLU) và Trích xuất đặc trưng (Feature Extraction), không bao gồm các bài toán sinh văn bản tự do (NLG) của các mô hình Decoder-only.

2.4 Cách tiếp cận dự kiến

Các nghiên cứu liên quan:

Trong những năm qua, hướng nghiên cứu về các mô hình Encoder-only đã đạt được nhiều thành tựu quan trọng. Điển hình là kiến trúc BERT [4] và các biến thể cải tiến như RoBERTa [5] đã thiết lập nền tảng vững chắc cho việc biểu diễn ngữ nghĩa. Tại Việt Nam, các mô hình như PhoBERT hay ViDeBERTa [3] đã tận dụng phương pháp huấn luyện tiếp tục (Continued Pre-training) trên quy mô dữ liệu lớn để tối ưu hóa khả năng xử lý tiếng Việt. Tuy nhiên, các mô hình này chủ yếu dựa trên kiến trúc Transformer cũ, gặp hạn chế về tốc độ huấn luyện và độ dài ngữ cảnh.

Gần đây, nghiên cứu về NeoBERT [1] đã tái định nghĩa sức mạnh của Encoder

bằng cách tích hợp các thành phần hiện đại. NeoBERT chứng minh rằng một mô hình quy mô 250 triệu tham số, nếu được tối ưu về tỷ lệ chiều sâu-chiều rộng và huấn luyện trên 2,1 nghìn tỷ token, có thể vượt qua hiệu suất của các mô hình lớn hơn gấp nhiều lần trên chuẩn đánh giá MTEB.

Bên cạnh đó, bài toán chuyển đổi ngôn ngữ đã có bước tiến với kỹ thuật SALT [2]. Thay vì thay thế tập từ vựng một cách máy móc, SALT đề xuất cơ chế tái sử dụng không gian véc-tơ từ các mô hình tiền huấn luyện của ngôn ngữ mục tiêu. Kết quả thực nghiệm cho thấy SALT giúp tăng tốc độ hội tụ nhanh hơn và đạt mức tổn thất (loss) thấp hơn so với các phương pháp khởi tạo truyền thống, đồng thời vẫn duy trì được hiệu quả khi chuyển sang ngôn ngữ mới.

Phương pháp thực hiện dự kiến:

Dựa trên việc kế thừa sức mạnh kiến trúc của NeoBERT và phương pháp chuyển đổi hiệu quả của SALT, đề tài đề xuất quy trình xây dựng mô hình Vietnamese NeoBERT thông qua các bước sau:

- 1. Căn chỉnh không gian (Alignment Stage):** Sử dụng không gian véc-tơ tiếng Việt từ ViDeBERTa làm nguồn. Thông qua các véc-tơ tĩnh (như fastText), thuật toán sẽ trích xuất một tập điểm neo (Anchor Extraction) gồm các token có độ tương đồng ngữ nghĩa cao giữa từ vựng của NeoBERT và ViDeBERTa.
- 2. Chuyển đổi tuyến tính dựa trên độ tương đồng ngữ nghĩa:** Đối với mỗi token không trùng lặp trong từ điển tiếng Việt, hệ thống thiết lập các đường hồi quy tuyến tính riêng biệt (unique regression lines) thông qua phương pháp Bình phương tối thiểu (Linear Least Squares). Bước này giúp chuyển đổi trực tiếp các biểu diễn ngữ nghĩa từ ViDeBERTa vào đúng hệ tọa độ không gian mà NeoBERT đã được học.
- 3. Huấn luyện thích ứng có chọn lọc:** Tiến hành huấn luyện tiếp tục trên ngữ liệu tiếng Việt. Để ngăn chặn hiện tượng mất thông tin, mô hình áp dụng

chiến lược đóng băng (freezing) một phần các lớp Transformer và sử dụng bộ lập lịch tốc độ học (learning rate scheduler) tối ưu nhằm tinh chỉnh lớp véc-tơ mới sao cho tương thích với các lớp phía trên.

Sự khác biệt so với các nghiên cứu trước đây:

So với các phương pháp xây dựng mô hình tiếng Việt truyền thống, cách tiếp cận này không yêu cầu huấn luyện lại từ đầu mà thực hiện ánh xạ toán học để tái sử dụng không gian biểu diễn sẵn có. Điểm mới của đề tài là việc áp dụng SALT lên kiến trúc NeoBERT – một kiến trúc hiện đại hỗ trợ ngữ cảnh dài 4.096 token và sử dụng FlashAttention-2 – thay vì áp dụng trên các kiến trúc BERT cũ. Điều này kỳ vọng tạo ra một bộ Encoder mạnh, tối ưu tài nguyên và có tốc độ suy luận cao cho tiếng Việt.

2.5 Kết quả dự kiến của đề tài

Đề tài dự kiến đạt được các kết quả cụ thể về mặt định lượng, sản phẩm và đóng góp khoa học như sau:

Số liệu định lượng:

- *Hiệu suất mô hình:* Vietnamese NeoBERT dự kiến đạt kết quả tương đương hoặc vượt trội so với các mô hình baseline hiện tại (như PhoBERT, ViDeBERTa) trên các bài toán NLU thuộc bộ benchmark VLSP và tập dữ liệu MTEB tiếng Việt.
- *Tốc độ hội tụ và chi phí:* Nhờ việc khởi tạo lớp véc-tơ bằng SALT, quá trình huấn luyện dự kiến có tốc độ hội tụ nhanh hơn, giảm từ 40% đến 50% số lượng token cần thiết so với việc huấn luyện lại từ đầu hoặc tiếp tục tiền huấn luyện không có định hướng.
- *Tốc độ thực thi (Throughput):* Nhờ kiến trúc tối ưu (optimal depth-to-width), mô hình dự kiến đạt tốc độ suy luận nhanh hơn khoảng 30% - 45% so với các mô hình Encoder-large truyền thống trên cùng một cấu hình phần cứng.

- *Khả năng xử lý ngữ cảnh*: Mô hình xử lý ổn định các văn bản có độ dài lên tới 4.096 token, khắc phục được rào cản 512 token của các kiến trúc tiền nhiệm.

Sản phẩm đầu ra:

- Trọng số mô hình (Model Weights) Vietnamese NeoBERT hoàn chỉnh (250M tham số), tương thích với thư viện Hugging Face Transformers.
- Bộ mã nguồn (Source Code) bao gồm thuật toán trích xuất tập điểm neo, hàm chuyển đổi tuyến tính SALT và kịch bản huấn luyện.
- Báo cáo thực nghiệm chi tiết so sánh độ chính xác, tốc độ và biểu đồ tổn thất (loss curve) giữa các phương pháp.

Đóng góp khoa học:

- Khẳng định khả năng ứng dụng của kiến trúc NeoBERT kết hợp với kỹ thuật SALT trong việc giải quyết bài toán thiếu hụt tài nguyên tính toán cho các ngôn ngữ ít tài nguyên.
- Kết quả nghiên cứu có triển vọng phát triển thành bài báo khoa học công bố tại các hội nghị chuyên ngành uy tín trong nước và quốc tế.

2.6 Kế hoạch thực hiện

Thời gian	Nội dung công việc	Người thực hiện
03/01/2026 – 20/01/2026	Khảo sát các tài liệu nghiên cứu về kiến trúc NeoBERT, thuật toán SALT và các phương pháp Cross-lingual Transfer.	Trung, Tú
21/01/2026 – 16/02/2026	Thu thập và tiền xử lý ngữ liệu tiếng Việt; xây dựng pipeline; chuẩn bị môi trường huấn luyện.	Trung, Tú
16/02/2026 – 10/03/2026	Cài đặt thuật toán trích xuất tập điểm neo; lập trình mô-đun hồi quy tuyến tính (Least Squares) cho các non-shared token.	Trung
	Huấn luyện mô hình NeoBert baseline trên dữ liệu tiếng Việt mà không dùng SALT.	Tú
11/03/2026 – 31/04/2026	Thực hiện giai đoạn ánh xạ không gian véc-tơ và tiến hành huấn luyện thích ứng (Language Adaptive Continual Pre-training). Bên cạnh đó thực nghiệm đánh giá mô hình NeoBert baseline về cả hiệu suất và chi phí huấn luyện.	Trung, Tú
01/04/2026 – 15/04/2026	Thực nghiệm đánh giá mô hình NeoBert có sử dụng SALT và so sánh kết quả với baseline. Đánh giá cả hai mô hình trên cả quá trình huấn luyện và hiệu suất của mô hình.	Trung, Tú
16/04/2026 – 15/05/2026	Thực hiện các thí nghiệm để so sánh mô hình NeoBert tiếng Việt với các mô hình khác. Đánh giá và phân tích kết quả của thí nghiệm.	Trung, Tú
16/05/2026 – 28/07/2026	Viết báo cáo khóa luận, trực quan hóa kết quả bằng biểu đồ và chuẩn bị slide bảo vệ.	Trung, Tú

Tài liệu

- [1] L. L. Breton, Q. Fournier, J. X. Morris, M. E. Mezouar, and S. Chandar, "NeoBERT: A Next-Generation BERT," *arXiv preprint arXiv:2502.19587v2*, 2025.
- [2] S. Lee, S. Hong, H. Moon, and H. Lim, "Semantic Aware Linear Transfer by Recycling Pre-trained Language Models for Cross-lingual Transfer," *arXiv preprint arXiv:2505.10945v2*, 2025.
- [3] V. D. Lai, C. V. Nguyen, N. T. Ngo, T. Nguyen, F. Dernoncourt, R. A. Rossi, and T. H. Nguyen, "ViDeBERTa: A Powerful Pre-trained Language Model

for Vietnamese," in *Findings of the Association for Computational Linguistics: EACL 2023*, pp. 1032–1043, 2023.

- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 4171–4186, 2019.
- [5] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019.

XÁC NHẬN
CỦA NGƯỜI HƯỚNG DẪN
(Ký và ghi rõ họ tên)

TP. Hồ Chí Minh, ngày... tháng... năm...
NHÓM SINH VIÊN THỰC HIỆN
(Ký và ghi rõ họ tên)

Danh sách các từ viết tắt

KÝ HIỆU	TÊN TIẾNG ANH
BLEU	Bilingual Evaluation Understudy
CPT	Continued Pre-training
EM	Exact Match
FFN	Feed-Forward Network
GELU	Gaussian Error Linear Unit
GLU	Gated Linear Unit
LLM	Large Language Model
MCC	Matthews Correlation Coefficient
MLM	Masked Language Modeling
MRC	Machine Reading Comprehension
MRR	Mean Reciprocal Rank
MTEB	Massive Text Embedding Benchmark
nDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
NLU	Natural Language Understanding
NSP	Next Sentence Prediction
PLM	Pre-trained Language Model
PPL	Perplexity
PPPL	Pseudo-Perplexity
RMSNorm	Root Mean Square Normalization
RoPE	Rotary Position Embeddings
SALT	Semantic Aware Linear Transfer
SiLU	Sigmoid Linear Unit
WSD	Warmup–Stable–Decay

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Đề cương	iv
Danh sách các từ viết tắt	xiv
Mục lục	xiv
Danh sách hình	xix
Danh sách bảng	xx
Tóm tắt	xx
1 Tổng quan	1
1.1 Bối cảnh và sự cần thiết của đề tài	1
1.2 Mục tiêu nghiên cứu	3
1.3 Phạm vi nghiên cứu	4
1.4 Đóng góp của đề tài	5
1.5 Cấu trúc đề tài	6
2 Cơ sở lý thuyết và các công trình liên quan	8
2.1 Kiến trúc Transformer và mô hình Encoder-only	9
2.1.1 Cơ chế tự chú ý trong kiến trúc Transformer	9

2.1.2	BERT và mô hình hóa ngôn ngữ có mặt nạ	10
2.1.3	Giới hạn của các mô hình Encoder thế hệ cũ	11
2.2	Kiến trúc NeoBERT	12
2.2.1	Tổng quan	12
2.2.2	Tỷ lệ chiều sâu trên chiều rộng tối ưu	14
2.2.3	Mã hóa vị trí quay (Rotary Position Embeddings)	14
2.2.4	Hàm kích hoạt SwiGLU	18
2.2.5	Chuẩn hóa Pre-RMSNorm	20
2.2.6	Dữ liệu và chiến lược tiền huấn luyện	22
2.3	Chuyển giao không gian véc-tơ và chuyển đổi ngôn ngữ	24
2.3.1	Hình học của không gian biểu diễn	24
2.3.2	Chuyển đổi ngôn ngữ và rủi ro mất thông tin	24
2.3.3	Khởi tạo véc-tơ dựa trên ngữ nghĩa và kỹ thuật SALT	25
2.4	Nhận xét định hướng	27
3	Phương pháp xây dựng NeoBERT cho tiếng Việt	28
3.1	Tổng quan phương pháp	28
3.1.1	Đặt vấn đề	28
3.1.2	Mô hình đích và mô hình nguồn véc-tơ	29
3.1.3	Quy trình xây dựng ViNeoBERT	30
3.2	Lựa chọn mô hình nguồn véc-tơ và tokenizer	31
3.2.1	Vai trò của tokenizer trong mô hình ngôn ngữ	31
3.2.2	So sánh PhoBERT và ViDeBERTa làm mô hình nguồn véc-tơ	32
3.3	Giảm kích thước và đồng bộ từ vựng	33
3.4	Trích xuất tập điểm neo (anchor set)	34
3.4.1	Vai trò của tập điểm neo	34
3.4.2	Điểm neo trùng mặt chữ và điểm neo chữ số	35
3.4.3	Mở rộng tập điểm neo bằng cặp dịch Anh–Việt	35
3.5	Khởi tạo lớp véc-tơ đầu vào và lớp đầu ra bằng kỹ thuật SALT	39

3.5.1	Chọn các điểm neo gần nghĩa nhất bằng Sparsemax	39
3.5.2	Xây dựng véc-tơ cho từng token bằng ánh xạ tuyến tính cục bộ	40
3.5.3	Áp dụng phép chiếu cho cả lớp véc-tơ đầu vào và lớp đầu ra	41
3.5.4	Tinh chỉnh kết quả khởi tạo: độ lớn véc-tơ, hệ số điều chỉnh và trọng số lớp đầu ra	42
3.6	Giai đoạn căn chỉnh với phần thân đóng băng	46
3.7	Tiếp tục tiền huấn luyện (Continued Pre-Training)	47
3.8	Tổng kết phương pháp	48
4	Thực nghiệm và Đánh giá	49
4.1	Môi trường và dữ liệu thực nghiệm	49
4.1.1	Dữ liệu tiền huấn luyện	50
4.1.2	Dữ liệu đánh giá	50
4.2	Cấu hình huấn luyện	51
4.3	Độ đo đánh giá	53
4.4	So sánh các phương án khởi tạo	54
4.4.1	Kiểm chứng hình học của kỹ thuật SALT	54
4.4.2	Lựa chọn hệ số tỉ lệ trọng số đầu ra	56
4.4.3	Lựa chọn chiến lược khởi tạo ma trận từ vựng	57
4.5	Kết quả của mô hình cuối	61
4.5.1	Chất lượng theo lượng dữ liệu huấn luyện	61
4.5.2	Đánh giá đa tác vụ	64
4.5.3	Mở rộng ngữ cảnh lên 4.096 token	66
4.6	Thảo luận	67
5	Kết luận và Hướng phát triển	69
5.1	Kết luận	69
5.2	Hạn chế của đề tài	71
5.3	Hướng phát triển	71

Tài liệu tham khảo	73
Phụ lục	80
A Bộ lọc chính tả tiếng Việt cho trích xuất cặp dịch Anh– Việt	81
B Định nghĩa hình thức các độ đo đánh giá	83

Danh sách hình

2.1	Luồng xử lý của mô hình Encoder-only.	10
2.2	Kiến trúc NeoBERT [5].	13
2.3	Cơ chế mã hóa vị trí quay (RoPE)	17
2.4	Cấu trúc cổng SwiGLU [16].	20
2.5	Sơ đồ tổng quan SALT	26
3.1	Quy trình xây dựng ViNeoBERT.	30
3.2	Lịch điều chỉnh tốc độ học Warmup–Stable–Decay (WSD).	48
4.1	Lớp véc-tơ tiếng Việt trước và sau khi áp dụng SALT.	55
4.2	Mất mát MLM theo hệ số tỉ lệ γ	57
4.3	So sánh năm phương án khởi tạo hai ma trận từ vựng (xám: hai đường cơ sở ngẫu nhiên; nâu: dùng chung trọng số; xanh: SALT). (a) mất mát MLM trên tập đánh giá; (b) F1 trên UIT-ViQuAD, thanh lỗi là độ lệch chuẩn qua ba seed ngẫu nhiên.	59
4.4	Đường cong mất mát giai đoạn CPT (WSD).	62
4.5	Điểm F1 trên UIT-ViQuAD theo số token CPT.	63
4.6	Pseudo-perplexity theo độ dài chuỗi, trước và sau pha mở rộng 4.096 token.	67

Danh sách bảng

2.1	Cấu hình NeoBERT [5].	22
3.1	So sánh tokenizer của PhoBERT và ViDeBERTa.	33
4.1	Thống kê dữ liệu thực nghiệm.	50
4.2	Các tác vụ đánh giá.	52
4.3	Các mô hình Encoder được đánh giá.	52
4.4	Siêu tham số huấn luyện.	53
4.5	Lượng dữ liệu tiếng Việt đã xử lý.	64
4.6	Kết quả đánh giá đa tác vụ tại điểm kiểm tra 5 tỷ token (giá trị tốt nhất in đậm).	64

Tóm tắt

Các mô hình Encoder-only như BERT giữ vai trò nền tảng cho các bài toán hiểu ngôn ngữ tự nhiên, nhưng trong nhiều năm gần như không được cập nhật về kiến trúc. NeoBERT (2025) thu hẹp khoảng cách này bằng cách tích hợp các cải tiến từ thế hệ mô hình ngôn ngữ lớn – RoPE, SwiGLU, Pre-RMSNorm, tỷ lệ sâu–hẹp tối ưu và ngữ cảnh 4.096 token – song chỉ được tiền huấn luyện trên tiếng Anh, trong khi các Encoder tiếng Việt hiện có vẫn dựa trên kiến trúc thế hệ cũ.

Khóa luận xây dựng **ViNeoBERT** – mô hình NeoBERT cho tiếng Việt – bằng một quy trình chuyển đổi ngôn ngữ thay vì huấn luyện lại từ đầu: (1) khởi tạo lớp véc-tơ đầu vào và lớp đầu ra bằng kỹ thuật Chuyển đổi tuyến tính trong không gian có ngữ nghĩa (Semantic Aware Linear Transfer – SALT), tái sử dụng không gian véc-tơ của mô hình tiếng Việt ViDeBERTa thông qua các ánh xạ tuyến tính cục bộ theo từng token; (2) căn chỉnh với phần thân đóng băng: đóng băng toàn bộ phần thân Transformer, chỉ huấn luyện hai ma trận vừa khởi tạo cho khớp với phần thân đó; và (3) tiếp tục tiền huấn luyện trên ngữ liệu CulturaX tiếng Việt theo lịch điều chỉnh tốc độ học Warmup–Stable–Decay. So với SALT gốc, đề tài đưa ra ba cải tiến: mở rộng tập điểm neo bằng các cặp dịch Anh–Việt; khởi tạo riêng lớp đầu ra – khởi tạo hệ số điều chỉnh theo tần suất xuất hiện của token trong tiếng Việt, đồng thời thu nhỏ trọng số lớp đầu ra để tín hiệu tần suất này chi phối giai đoạn huấn luyện đầu; và bổ sung giai đoạn căn chỉnh với phần thân đóng băng.

Thực nghiệm so sánh có kiểm soát trên cùng một lượng token huấn

luyện xác nhận từng lựa chọn thiết kế, từ việc giảm biên độ trọng số lớp đầu ra tới khởi tạo riêng hai ma trận và giai đoạn căn chỉnh với phần thân đóng băng. Trên bộ đánh giá gồm mười hai tác vụ hiểu tiếng Việt (phân loại, suy luận, gán nhãn chuỗi, tương đồng ngữ nghĩa và biểu diễn câu), ViNeoBERT sau 5 tỷ token ($\sim 20\text{GB}$) đạt kết quả tương đương PhoBERT-base-v2 ở đa số tác vụ và dẫn đầu ở gán nhãn từ loại. Trên tác vụ đọc hiểu với bộ dữ liệu UIT-ViQuAD, mô hình đạt 76,6 điểm F1 – hơn XLM-R-base 3,3 điểm và cạnh tranh với PhoBERT-large (lớn gấp 1,5 lần) dù chỉ dùng khoảng 3,5% lượng token mà mô hình này đã huấn luyện (~ 140 tỷ). Ngoài ra, một pha mở rộng ngắn ~ 200 triệu token ở độ dài 4.096 token giữ pseudo-perplexity ổn định trên toàn dải ngữ cảnh, biến trần 4.096 token của kiến trúc thành năng lực sử dụng được. Kết quả cho thấy tái sử dụng không gian biểu diễn sẵn có là con đường tiết kiệm và hiệu quả để đưa các kiến trúc Encoder thế hệ mới đến với tiếng Việt.

Từ khóa: NeoBERT, tiếng Việt, chuyển đổi ngôn ngữ, khởi tạo véc-tơ cho ngôn ngữ mới, SALT, tiếp tục tiền huấn luyện.

Chương 1

Tổng quan

1.1 Bối cảnh và sự cần thiết của đề tài

Trong lịch sử phát triển của lĩnh vực Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing – NLP), các mô hình dạng Encoder-only như BERT [1] và RoBERTa [2] từng đóng vai trò nền tảng không thể thiếu cho các bài toán hiểu ngôn ngữ (Natural Language Understanding – NLU) như phân loại văn bản, nhận dạng thực thể có tên, suy luận ngôn ngữ tự nhiên và trả lời câu hỏi. Nhờ cơ chế mã hóa hai chiều (bidirectional encoding), các mô hình này có khả năng nắm bắt ngữ cảnh từ cả hai phía của một chuỗi đầu vào, tạo ra biểu diễn ngữ nghĩa phong phú và được ứng dụng rộng rãi trong thực tế.

Tuy nhiên, trong khi các dòng mô hình Decoder-only – hay còn gọi là các Mô hình Ngôn ngữ Lớn (Large Language Models – LLMs) như LLaMA [3] và DeepSeek [4] – liên tục tiến hóa với quy mô dữ liệu và những đột phá về kiến trúc, các bộ Encoder-only lại gần như không có sự cải thiện đáng kể trong suốt nhiều năm qua. BERT và RoBERTa vẫn tiếp tục được sử dụng rộng rãi dù kiến thức của chúng ngày càng trở nên lỗi thời, đặc biệt trong các ứng dụng đòi hỏi hiểu ngữ cảnh dài hoặc tốc độ suy luận cao.

Sự ra đời của NeoBERT [5] vào năm 2025 đã thu hẹp đáng kể khoảng cách này. NeoBERT tái định nghĩa khả năng của các mô hình Encoder

bằng cách tích hợp các cải tiến kiến trúc tiên tiến nhất từ thế giới LLM: hàm kích hoạt (activation function) SwiGLU, cơ chế mã hóa vị trí tương đối Rotary Position Embeddings (RoPE), chuẩn hóa Pre-RMSNorm, và tỷ lệ chiều sâu trên chiều rộng tối ưu – sâu 28 lớp nhưng chỉ giữ kích thước ẩn (hidden size) ở mức 768. Với khoảng 250 triệu tham số và được huấn luyện trên 2,1 nghìn tỷ token, NeoBERT vượt trội BERT_{large} và RoBERTa_{large} trên chuẩn đánh giá MTEB (Massive Text Embedding Benchmark), đồng thời hỗ trợ ngữ cảnh lên tới 4.096 token – gấp 8 lần so với BERT và RoBERTa (512 token).

Tuy nhiên, NeoBERT chỉ được huấn luyện trên ngữ liệu tiếng Anh. Đối với tiếng Việt, nhu cầu về một bộ Encoder mạnh mẽ và hiện đại ngày càng cấp thiết khi các ứng dụng NLP trong nước – từ hệ thống tìm kiếm ngữ nghĩa, phân tích cảm xúc, đến các hệ thống hỏi-đáp tự động – đòi hỏi chất lượng biểu diễn ngữ nghĩa ngày càng cao. Các mô hình Encoder tiếng Việt hiện có như PhoBERT [6] và ViDeBERTa [7] tuy hiệu quả nhưng vẫn dựa trên kiến trúc Transformer cũ, bị giới hạn ở độ dài ngữ cảnh 512 token và thiếu đi các tối ưu hóa kiến trúc hiện đại.

Việc huấn luyện lại NeoBERT từ đầu (from scratch) cho tiếng Việt trên quy mô hàng nghìn tỷ token là điều bất khả thi về mặt tài nguyên tính toán đối với hầu hết các nhóm nghiên cứu. Bên cạnh đó, phương pháp tiếp tục tiền huấn luyện (Continued Pre-training) thông thường – tức là khởi tạo ngẫu nhiên lớp véc-tơ mới rồi huấn luyện trực tiếp – thường dẫn đến hiện tượng *mất thông tin* (catastrophic forgetting), làm phá vỡ các đặc trưng biểu diễn quý giá mà mô hình đã học từ dữ liệu tiếng Anh.

Xuất phát từ những thách thức trên, đề tài nghiên cứu phương pháp chuyển đổi ngôn ngữ hiệu quả cho NeoBERT thông qua kỹ thuật Chuyển đổi tuyến tính trong không gian có ngữ nghĩa (Semantic Aware Linear Transfer – SALT) [8]. Thay vì huấn luyện từ đầu, SALT tái sử dụng (recycling) bộ véc-tơ từ mô hình tiền huấn luyện tiếng Việt chất lượng cao ViDeBERTa, chuyển đổi không gian biểu diễn sang không gian của NeoBERT thông qua các phép ánh xạ tuyến tính cục bộ dựa trên độ tương

đồng ngữ nghĩa. Đây là một hướng tiếp cận tiết kiệm tài nguyên, vừa kế thừa không gian biểu diễn tiếng Việt sẵn có, vừa bảo toàn được sức mạnh kiến trúc của NeoBERT.

1.2 Mục tiêu nghiên cứu

Đề tài đặt ra các mục tiêu nghiên cứu cụ thể như sau:

1. **Nghiên cứu và nắm vững kiến trúc NeoBERT:** Phân tích chi tiết các cải tiến kiến trúc của NeoBERT so với BERT/RoBERTa truyền thống, bao gồm SwiGLU, RoPE, Pre-RMSNorm và tỷ lệ chiều sâu trên chiều rộng tối ưu; từ đó xác định những điểm cần điều chỉnh khi thích ứng sang tiếng Việt.
2. **Áp dụng và cải tiến kỹ thuật SALT để chuyển giao không gian biểu diễn:** Nghiên cứu và triển khai thuật toán SALT – bao gồm trích xuất tập điểm neo (anchor extraction) dựa trên độ tương đồng ngữ nghĩa và ánh xạ tuyến tính cục bộ theo phương pháp bình phương tối thiểu – nhằm ánh xạ không gian véc-tơ từ ViDeBERTa sang không gian véc-tơ của NeoBERT. Trên nền tảng đó, đề tài đề xuất ba cải tiến so với SALT gốc: mở rộng tập điểm neo bằng các cặp dịch Anh–Việt, khởi tạo riêng lớp đầu ra kèm hệ số điều chỉnh theo tần suất token, và bổ sung giai đoạn căn chỉnh với phần thân đóng băng.
3. **Xây dựng mô hình NeoBERT cho tiếng Việt (ViNeoBERT):** Kết hợp lớp véc-tơ đầu vào và lớp đầu ra đã được khởi tạo bằng SALT với phần thân NeoBERT gốc, sau đó huấn luyện qua giai đoạn căn chỉnh với phần thân đóng băng và tiếp tục tiền huấn luyện (Continued Pre-training) trên ngữ liệu tiếng Việt quy mô lớn theo lịch tốc độ học Warmup–Stable–Decay.

4. **Đánh giá hiệu quả của phương pháp:** Đánh giá ViNeoBERT trên một bộ tác vụ hiểu ngôn ngữ tiếng Việt đa dạng – gồm hỏi đáp đọc hiểu (UIT-ViQuAD), suy luận, phân loại văn bản, gán nhãn chuỗi, tương đồng ngữ nghĩa và biểu diễn câu (Retrieval, Reranking) – và đối chiếu với các mô hình tiếng Việt và đa ngôn ngữ hiện có (PhoBERT, XLM-R). Bên cạnh đó, đề tài so sánh các phương án khởi tạo khác nhau khi cùng được huấn luyện trên một lượng token như nhau, nhằm đo riêng phần đóng góp của từng thành phần trong phương pháp.
5. **Kiểm chứng tính hiệu quả của việc tái sử dụng không gian biểu diễn:** Thông qua kết quả thực nghiệm, xác minh rằng việc khởi tạo véc-tơ bằng kỹ thuật SALT – vốn dựa trên không gian biểu diễn sẵn có của ViDeBERTa – kết hợp với giai đoạn căn chỉnh khi phân thân bị đóng băng giúp giảm đáng kể lượng token cần huấn luyện và giữ cho quá trình thích ứng ổn định, thay vì phải học lại gần như từ đầu như khi khởi tạo ngẫu nhiên.

1.3 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu thực nghiệm trong các giới hạn sau:

Đối tượng nghiên cứu chính bao gồm: (1) kiến trúc NeoBERT với trọng số gốc được công bố công khai; (2) thuật toán SALT với cơ chế trích xuất tập điểm neo và ánh xạ tuyến tính cục bộ; (3) mô hình nguồn véc-tơ (donor) ViDeBERTa cung cấp không gian véc-tơ tiếng Việt cho bước chuyển giao.

Dữ liệu thực nghiệm gồm phần tiếng Việt của kho ngữ liệu web đa ngôn ngữ CulturaX cho giai đoạn tiếp tục tiền huấn luyện, cùng một bộ tác vụ đánh giá tiếng Việt đa dạng cho bước tinh chỉnh cho tác vụ ứng dụng – từ hỏi đáp đọc hiểu (UIT-ViQuAD), suy luận ngôn ngữ tự nhiên (XNLI, ViANLI, QNLI), phân loại văn bản (UIT-VSFC, UIT-VSMEC,

UIT-ViCTSD), đánh giá tính hợp ngữ pháp (ViCoLA-syn), gán nhãn từ loại (UD-VTB), tương đồng ngữ nghĩa (STS, Paraphrase), tối biểu diễn câu (Retrieval, Reranking).

Các ràng buộc và giới hạn: Nghiên cứu *không* thực hiện huấn luyện NeoBERT từ đầu do hạn chế tài nguyên tính toán. Phạm vi bài toán chỉ bao gồm các tác vụ NLU và trích xuất đặc trưng (Feature Extraction), không bao gồm các bài toán sinh văn bản tự do của các mô hình Decoder-only. Mô hình kế thừa kiến trúc hỗ trợ ngữ cảnh tới 4.096 token của NeoBERT và được huấn luyện bổ sung một pha ngắn ở độ dài này; việc kiểm chứng ngữ cảnh dài chỉ dừng ở một độ đo nội tại – tức đo trực tiếp trên bản thân mô hình ngôn ngữ mà không thông qua tác vụ ứng dụng, cụ thể là pseudo-perplexity theo độ dài chuỗi – còn đánh giá chuyên biệt cho ngữ cảnh dài được để lại như hướng phát triển.

1.4 Đóng góp của đề tài

Đề tài đưa ra các đóng góp cụ thể sau:

- **Mô hình NeoBERT cho tiếng Việt (ViNeoBERT):** Cung cấp một bộ Encoder hiện đại khoảng 250 triệu tham số cho tiếng Việt, kế thừa các cải tiến kiến trúc của NeoBERT (RoPE, SwiGLU, Pre-RMSNorm, FlashAttention) cùng trần ngữ cảnh 4.096 token, tương thích với thư viện Hugging Face Transformers và sẵn sàng sử dụng cho các bài toán NLU thực tế.
- **Quy trình chuyển đổi ngôn ngữ:** Đề xuất và kiểm chứng một quy trình đưa NeoBERT sang tiếng Việt – gồm khởi tạo véc-tơ, căn chỉnh với phần thân đóng băng và tiếp tục tiền huấn luyện – lần đầu tiên áp dụng cho một kiến trúc Encoder thế hệ mới. Ở giai đoạn khởi tạo, đề tài kết hợp kỹ thuật SALT với ba cải tiến so với nghiên cứu gốc: (1) mở rộng tập điểm neo bằng các cặp dịch Anh–Việt khai thác tự động; (2) khởi tạo riêng lớp đầu ra, kèm hệ số điều chỉnh

theo tần suất token và hệ số tỉ lệ trọng số giúp phân phối tần suất chi phối giai đoạn huấn luyện đầu; (3) bổ sung giai đoạn căn chỉnh với phần thân đóng băng trước khi tiếp tục tiền huấn luyện.

- **Bộ mã nguồn công khai:** Bao gồm thuật toán trích xuất tập điểm neo, phần cài đặt kỹ thuật SALT cho lớp véc-tơ đầu vào và lớp đầu ra, cùng kịch bản huấn luyện thích ứng, phục vụ cho cộng đồng nghiên cứu NLP tiếng Việt.
- **Đóng góp khoa học:** Kết quả thực nghiệm khẳng định tính khả thi của việc tái sử dụng không gian biểu diễn từ các mô hình sẵn có để chuyển một kiến trúc Encoder thể hệ mới sang một ngôn ngữ khác với lượng token huấn luyện nhỏ hơn nhiều so với huấn luyện lại từ đầu. Hướng tiếp cận này tiết kiệm chi phí cho việc cập nhật, áp dụng các kiến trúc thể hệ mới, và đặc biệt có ý nghĩa với những ngôn ngữ ít tài nguyên.

1.5 Cấu trúc đề tài

Phần còn lại của khóa luận được tổ chức như sau:

- **Chương 1 – Tổng quan:** Giới thiệu bối cảnh và sự cần thiết của đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu và các đóng góp chính.
- **Chương 2 – Cơ sở lý thuyết và các công trình liên quan:** Trình bày kiến trúc Transformer và dòng mô hình Encoder-only, các đặc điểm chính của NeoBERT, cơ sở hình học của không gian biểu diễn, cùng các phương pháp khởi tạo véc-tơ khi chuyển một mô hình sang ngôn ngữ mới, dẫn tới kỹ thuật SALT.
- **Chương 3 – Phương pháp xây dựng NeoBERT cho tiếng Việt:** Trình bày chi tiết quy trình xây dựng của đề tài: giảm kích

thước từ vựng và trích xuất tập điểm neo, kỹ thuật SALT cho cả lớp véc-tơ đầu vào lẫn lớp đầu ra, giai đoạn căn chỉnh với phần thân đóng băng và tiếp tục tiền huấn luyện theo lịch WSD.

- **Chương 4 – Thực nghiệm và Đánh giá:** Mô tả dữ liệu, cấu hình huấn luyện, các độ đo và phân tích kết quả thực nghiệm, bao gồm so sánh các phương án khởi tạo và đánh giá trên tác vụ ứng dụng.
- **Chương 5 – Kết luận và Hướng phát triển:** Tổng kết các kết quả đạt được so với mục tiêu đề ra và đề xuất các hướng phát triển tiếp theo.

Chương 2

Cơ sở lý thuyết và các công trình liên quan

Chương này trình bày nền tảng lý thuyết và các công trình liên quan làm cơ sở cho phương pháp được đề xuất. Trước hết, chúng tôi khảo sát kiến trúc Transformer cùng dòng mô hình Encoder-only đã trở thành chuẩn mực cho các bài toán hiểu ngôn ngữ (Mục 2.1). Tiếp theo, chúng tôi trình bày các đặc điểm chính của NeoBERT – mô hình nền mà đề tài kế thừa – cùng những cải tiến có ảnh hưởng trực tiếp tới hướng chuyển giao tiếng Việt (Mục 2.2). Sau đó, chúng tôi trình bày cơ sở hình học của không gian biểu diễn, bài toán chuyển đổi ngôn ngữ và các phương pháp khởi tạo véc-tơ khi chuyển một mô hình sang ngôn ngữ mới, dẫn tới kỹ thuật Chuyển đổi tuyến tính trong không gian có ngữ nghĩa (Semantic Aware Linear Transfer – SALT) mà đề tài áp dụng (Mục 2.3). Cuối cùng, chương đưa ra một số nhận xét định hướng cho phương pháp được trình bày ở chương tiếp theo.

2.1 Kiến trúc Transformer và mô hình Encoder-only

2.1.1 Cơ chế tự chú ý trong kiến trúc Transformer

Transformer được Vaswani và cộng sự [9] giới thiệu như một kiến trúc xử lý chuỗi không dùng cơ chế hồi quy. Thay vì đọc token tuần tự như RNN/LSTM, Transformer cho phép mỗi vị trí quan sát trực tiếp các vị trí còn lại thông qua cơ chế tự chú ý (self-attention). Đây là lý do kiến trúc này vừa song song hóa tốt trên GPU, vừa học được các phụ thuộc xa trong câu.

Với ma trận trạng thái ẩn X , mô hình tạo ba phép chiếu tuyến tính gồm truy vấn Q , khóa K và giá trị V . Trọng số chú ý được tính bằng tích vô hướng có tỷ lệ:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.1)$$

Trong đó, ma trận $QK^\top$ biểu diễn mức liên hệ giữa mọi cặp token. Vì ma trận này có kích thước $n \times n$, chi phí tăng theo $O(n^2)$ khi độ dài chuỗi tăng. Đây là một lý do các Encoder hiện đại cần kết hợp thêm kỹ thuật tối ưu bộ nhớ như FlashAttention khi mở rộng ngữ cảnh lên hàng nghìn token.

Transformer thường dùng tự chú ý đa đầu (Multi-Head Attention) để học nhiều kiểu quan hệ song song. Công thức tổng quát được viết gọn như sau:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \end{aligned} \quad (2.2)$$

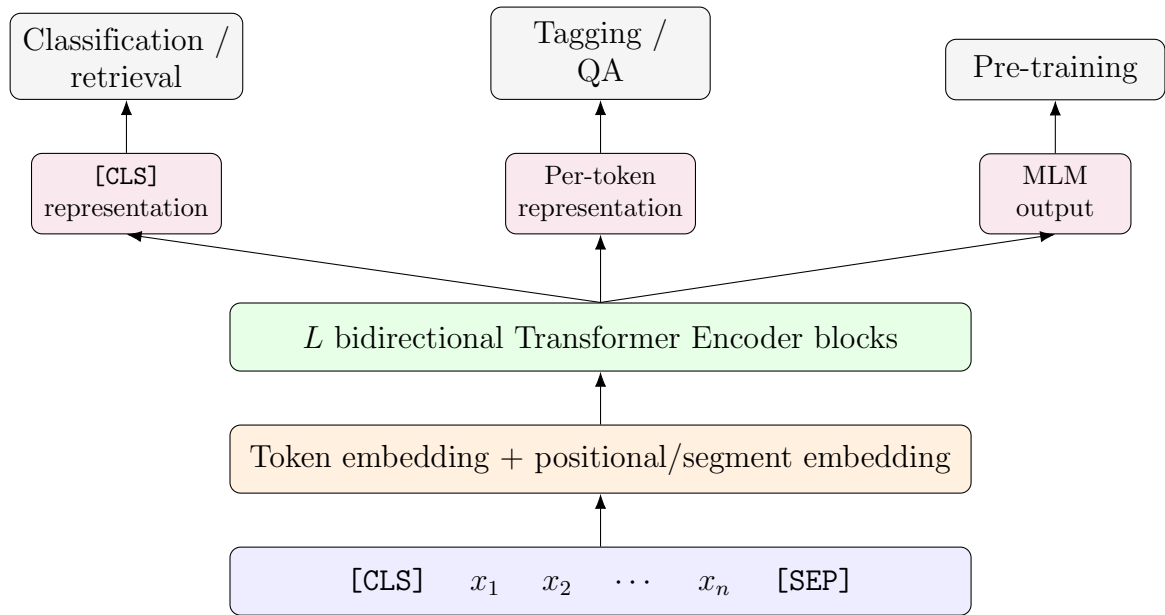
Sau attention, mỗi khối Transformer còn có mạng truyền thẳng theo vị trí (position-wise feed-forward network), kết nối tắt (residual connection) và

lớp chuẩn hóa (normalization layer). Các biến thể hiện đại chủ yếu khác nhau ở cách mã hóa vị trí, loại chuẩn hóa và hàm kích hoạt. Những điểm này sẽ được dùng để giải thích NeoBERT ở Mục 2.2.

2.1.2 BERT và mô hình hóa ngôn ngữ có mặt nạ

Từ phần encoder của Transformer, Devlin và cộng sự [1] đề xuất BERT (Bidirectional Encoder Representations from Transformers). BERT là mô hình Encoder-only: nó không có bộ giải mã tự hồi quy và không che các token phía sau (causal mask), nên mỗi token có thể khai thác đồng thời ngữ cảnh bên trái lẫn bên phải. Đầu ra của BERT là một chuỗi biểu diễn theo ngữ cảnh, phù hợp với các tác vụ hiểu ngôn ngữ như phân loại văn bản, gán nhãn chuỗi, hỏi đáp trích xuất và tìm kiếm thông tin.

Hình 2.1 minh họa luồng xử lý điển hình của một mô hình Encoder-only kiểu BERT.



Hình 2.1: Luồng xử lý của mô hình Encoder-only.

Khác với các mô hình ngôn ngữ tự hồi quy chỉ mô hình hóa xác suất từ trái sang phải, BERT được tiền huấn luyện đồng thời trên hai tác vụ tự giám sát (self-supervised). Tác vụ thứ nhất và chủ yếu là mô hình hóa

ngôn ngữ có mặt nạ (Masked Language Modeling – MLM): 15% token trong chuỗi được chọn ngẫu nhiên, trong đó 80% thay bằng token đặc biệt [MASK], 10% thay bằng token ngẫu nhiên, và 10% giữ nguyên – cơ chế này giúp giảm sự khác biệt giữa huấn luyện (có [MASK]) và tinh chỉnh (không có [MASK]). Mô hình dự đoán lại token gốc dựa trên ngữ cảnh còn lại:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i \mid x_{\setminus \mathcal{M}}) \quad (2.3)$$

Trong đó $\mathcal{M}$ là tập vị trí bị che. Cách huấn luyện này buộc mô hình học biểu diễn hai chiều thay vì chỉ dự đoán token kế tiếp. Tác vụ thứ hai là dự đoán câu tiếp theo (Next Sentence Prediction – NSP): phân loại nhị phân xem câu B có thực sự theo sau câu A trong văn bản gốc hay không. Tuy nhiên, NSP về sau được Liu và cộng sự [2] chứng minh là không cần thiết và có thể gây hại, nên RoBERTa và các mô hình hiện đại kế tiếp đã loại bỏ hoàn toàn.

Tiếp nối BERT, Liu và cộng sự [2] đề xuất RoBERTa với các cải tiến về quy trình tiền huấn luyện, nổi bật là loại bỏ NSP, dùng mặt nạ động (dynamic masking – vị trí che được sinh lại qua mỗi epoch thay vì cố định một lần như BERT) và tăng quy mô dữ liệu. Các mô hình tiếng Việt như PhoBERT [6] và ViDeBERTa [7] cũng kế thừa hướng Encoder-only này. Do đó, khi xây dựng NeoBERT cho tiếng Việt, bài toán trọng tâm không phải thay đổi bản chất Encoder-only, mà là tái sử dụng phần thân Encoder hiện đại và trang bị cho nó một bộ từ vựng tiếng Việt.

2.1.3 Giới hạn của các mô hình Encoder thế hệ cũ

BERT và RoBERTa vẫn rất quan trọng, nhưng chúng mang nhiều giả định của thời kỳ đầu: ngữ cảnh thường giới hạn ở 512 token, mã hóa vị trí tuyệt đối khó mở rộng, và kiến trúc chưa có các thành phần hiện đại như RoPE, SwiGLU, RMSNorm hay FlashAttention. Khi độ dài chuỗi tăng từ 512 lên 4.096 token, riêng số phần tử của ma trận chú ý – ma trận trọng

số giữa mọi cặp token trong chuỗi – đã tăng 64 lần (bình phương của mức tăng độ dài), nên việc mở rộng ngữ cảnh không thể chỉ dựa vào tăng độ dài đầu vào.

Ngoài kiến trúc, quy mô dữ liệu cũng thay đổi mạnh. BERT được huấn luyện trên BooksCorpus và Wikipedia với khoảng 13GB dữ liệu, RoBERTa tăng lên khoảng 160GB, trong khi các Encoder gần đây như NomicBERT [10], ModernBERT [11] và NeoBERT [5] chuyển sang dữ liệu web lớn hơn, ngữ cảnh dài hơn và quy trình huấn luyện tối ưu hơn. Vì vậy, NeoBERT được chọn trong đề tài như một phần thân Encoder hiện đại để thích ứng sang tiếng Việt thay vì tiếp tục dựa trên thế hệ BERT cũ.

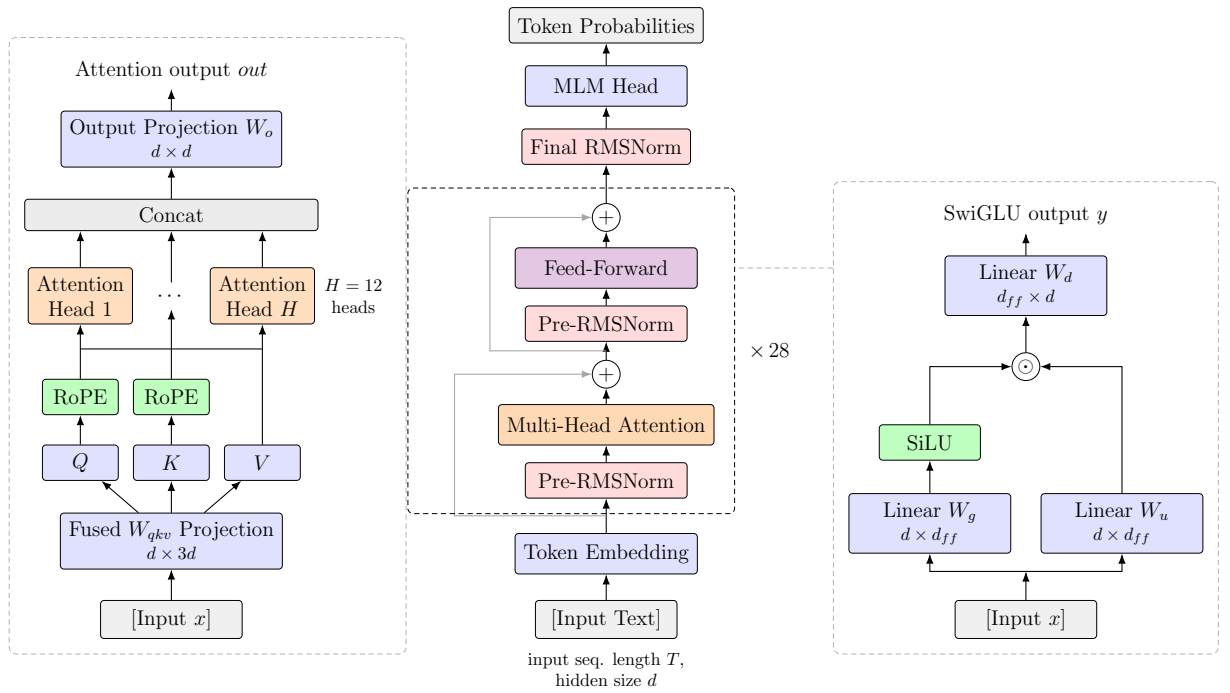
2.2 Kiến trúc NeoBERT

2.2.1 Tổng quan

Về bản chất, NeoBERT [5] vẫn là một mô hình thuộc dòng BERT: mã hóa hai chiều toàn chuỗi và được tiền huấn luyện với mục tiêu MLM. Điều làm nó khác biệt là toàn bộ các thành phần kiến trúc bên trong đã được thay mới theo những gì thế hệ mô hình ngôn ngữ lớn gần đây chứng minh là hiệu quả. Mô hình có khoảng 250 triệu tham số (nhóm tác giả đặt tên là bản NeoBERT_{250M} theo số làm tròn), giữ kích thước ẩn 768 như BERT-base nhưng sâu 28 lớp, dùng 12 đầu chú ý (attention head, mỗi đầu 64 chiều), hỗ trợ ngữ cảnh 4.096 token, và được tiền huấn luyện trên khoảng 2,1 nghìn tỷ token văn bản web tiếng Anh (RefinedWeb); mục tiêu tiền huấn luyện là MLM thuần túy (bỏ NSP theo RoBERTa), tỷ lệ che 20%, và luôn dùng token [MASK] để che (bỏ quy tắc 80/10/10 của BERT).

Kiến trúc của mô hình được minh họa trong Hình 2.2. So với BERT, có bốn thay đổi cốt lõi, cũng chính là bốn nội dung sẽ được phân tích trong các mục con tiếp theo: (1) tỷ lệ chiều sâu trên chiều rộng được chọn lại theo phân tích lý thuyết – sâu hơn nhưng không rộng hơn; (2) véc-tơ vị trí tuyệt đối bị loại bỏ hoàn toàn, thay bằng phép quay RoPE áp dụng trên

truy vấn và khóa bên trong từng khối; (3) mạng truyền thẳng dùng hàm kích hoạt SwiGLU thay cho GELU; và (4) chuẩn hóa RMSNorm đặt *trước* mỗi lớp con (chú ý hoặc truyền thẳng), thay vì đặt *sau* như LayerNorm ở BERT. Ngoài ra, khác với BERT và RoBERTa vốn cho đầu MLM *dùng chung trọng số* với ma trận véc-tơ đầu vào, NeoBERT tách riêng hai thành phần này: đầu MLM là một lớp tuyến tính có trọng số độc lập. Đây là căn cứ để phương pháp ở Chương 3 khởi tạo riêng hai ma trận.



Hình 2.2: Kiến trúc NeoBERT [5].

Điểm cần chú ý khi đọc Hình 2.2: khác với sơ đồ BERT quen thuộc, *không có* khối véc-tơ vị trí cộng vào véc-tơ token – thông tin vị trí được đưa vào bên trong từng khối qua RoPE; chuẩn hóa nằm *trước* lớp tự chú ý và lớp truyền thẳng, còn hai kết nối tắt cộng thẳng vào luồng chính; và sau khối cuối cùng có một lớp RMSNorm khép lại trước khi tín hiệu đi vào các đầu tác vụ (task head – lớp đầu ra riêng cho từng tác vụ). Trong phạm vi đề tài, NeoBERT được xem là *kiến trúc đích*: phần thân Transformer đã được tiền huấn luyện mạnh trên tiếng Anh và sẽ được giữ nguyên, còn

lớp véc-tơ và đầu MLM gắn với từ vựng tiếng Anh là phần phải thay thế. Mỗi cải tiến dưới đây đều để lại một ràng buộc hoặc một thuận lợi cụ thể cho việc thay thế đó.

2.2.2 Tỷ lệ chiều sâu trên chiều rộng tối ưu

Với một ngân sách tham số cố định, câu hỏi đầu tiên của thiết kế kiến trúc là nên phân bổ tham số vào *chiều sâu* (số lớp L) hay *chiều rộng* (kích thước ẩn d). Levine và cộng sự [12] phân tích lý thuyết khả năng biểu diễn của self-attention và chỉ ra rằng hai đại lượng này phải tăng *đồng bộ*: một mạng quá nông-rộng lãng phí tham số vì các phụ thuộc bậc cao giữa token không thể được hợp thành qua ít bước chú ý, trong khi một mạng quá sâu-hẹp lại bị chiều rộng chặn khả năng lưu trữ thông tin ở mỗi lớp. Từ đó tồn tại một *vùng sâu-rộng tối ưu* cho từng cỡ mô hình.

Theo phân tích này, các cấu hình BERT truyền thống ($12 \text{ lớp} \times 768$ hoặc $24 \text{ lớp} \times 1.024$) nằm lệch về phía *quá rộng so với độ sâu*. NeoBERT do đó giữ kích thước ẩn 768 như BERT-base nhưng tăng số lớp lên 28 – trở thành một mô hình “sâu mà hẹp” so với BERT-large dù có ít tham số hơn. Đối với đề tài, thiết kế này mang hai ý nghĩa: (1) phần thân đủ sâu để học các biến đổi phi tuyến mạnh – đây chính là phần có giá trị cần được giữ nguyên khi chuyển giao; và (2) kích thước ẩn 768 trùng với chiều véc-tơ của ViDeBERTa-base (mô hình nguồn véc-tơ), nên phép chiếu ở Chương 3 diễn ra giữa hai không gian *cùng số chiều*.

2.2.3 Mã hóa vị trí quay (Rotary Position Embeddings)

BERT mã hóa vị trí bằng một bảng véc-tơ vị trí *tuyệt đối* học được, cộng trực tiếp vào véc-tơ token trước khối Transformer đầu tiên: $h_i = e_i + p_i$. Cách tiếp cận này mang hai hạn chế căn bản: (1) bảng p có kích thước cố định theo độ dài huấn luyện (512 với BERT), không thể mở rộng sang

chuỗi dài hơn mà không huấn luyện lại; (2) điểm chú ý (attention score) giữa hai token phụ thuộc vào vị trí tuyệt đối thay vì khoảng cách tương đối – vốn mới là đại lượng mang ý nghĩa cú pháp, vì “token A đứng trước token B hai bước” quan trọng hơn “token A ở vị trí 37”. Ngoài ra, do chỉ được cộng ở đầu vào, thông tin vị trí phải lan truyền qua toàn bộ các lớp chuẩn hóa và phi tuyến phía sau, dễ bị pha loãng dần trong mạng sâu.

Mục tiêu hình thức của mã hóa vị trí tương đối là tìm hàm biến đổi f_q, f_k sao cho tích vô hướng giữa truy vấn (Q) tại vị trí m và khóa (K) tại vị trí n chỉ phụ thuộc vào nội dung token và khoảng cách tương đối:

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n). \quad (2.4)$$

Xuất phát từ trường hợp 2D. Su và cộng sự [13] bắt đầu với trường hợp $d = 2$, khai thác hình học mặt phẳng phức. Véc-tơ $q = W_q x_m \in \mathbb{R}^2$ được xem như số phức $q = \|q\|e^{i\theta_q}$, và tương tự cho k . Điều kiện ban đầu yêu cầu $f_q(x_m, 0) = W_q x_m$ và $f_k(x_n, 0) = W_k x_n$ (véc-tơ không mang thông tin vị trí khi ở vị trí 0). Từ đó, Su và cộng sự chỉ ra rằng một nghiệm thỏa mãn Công thức (2.4) là:

$$f_q(x_m, m) = (W_q x_m) e^{im\theta}, \quad f_k(x_n, n) = (W_k x_n) e^{in\theta}, \quad (2.5)$$

với $\theta \in \mathbb{R}$ là hằng số ấn định tần số quay (góc quay ứng với mỗi bước dịch vị trí). Ý nghĩa hình học rất trực quan: mỗi token được *quay* thêm một góc tỷ lệ với vị trí của nó trong mặt phẳng phức. Tích vô hướng trở thành:

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = \text{Re} \left[(W_q x_m) (W_k x_n)^* e^{i(m-n)\theta} \right],$$

chỉ phụ thuộc vào $m - n$ – thỏa mãn ràng buộc trong Công thức (2.4). Đây là ý tưởng cốt lõi của RoPE: *mã hóa vị trí bằng phép quay, không phải phép cộng*.

Dạng tổng quát cho véc-tơ d_h chiều. Để mở rộng sang $x \in \mathbb{R}^{d_h}$ (d_h chẵn), không gian được chia thành $d_h/2$ mặt phẳng con hai chiều độc lập,

mỗi mặt phẳng quay với tần số riêng θ_i . Ma trận quay khối-chéo $R_{\Theta,m}$ – đường chéo chính gồm $d_h/2$ khối quay 2×2 – kết hợp tất cả mặt phẳng con:

$$R_{\Theta,m} = \text{diag}(R_{m,1}, \dots, R_{m,d_h/2}),$$

$$R_{m,i} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d_h}. \quad (2.6)$$

Trong đó:

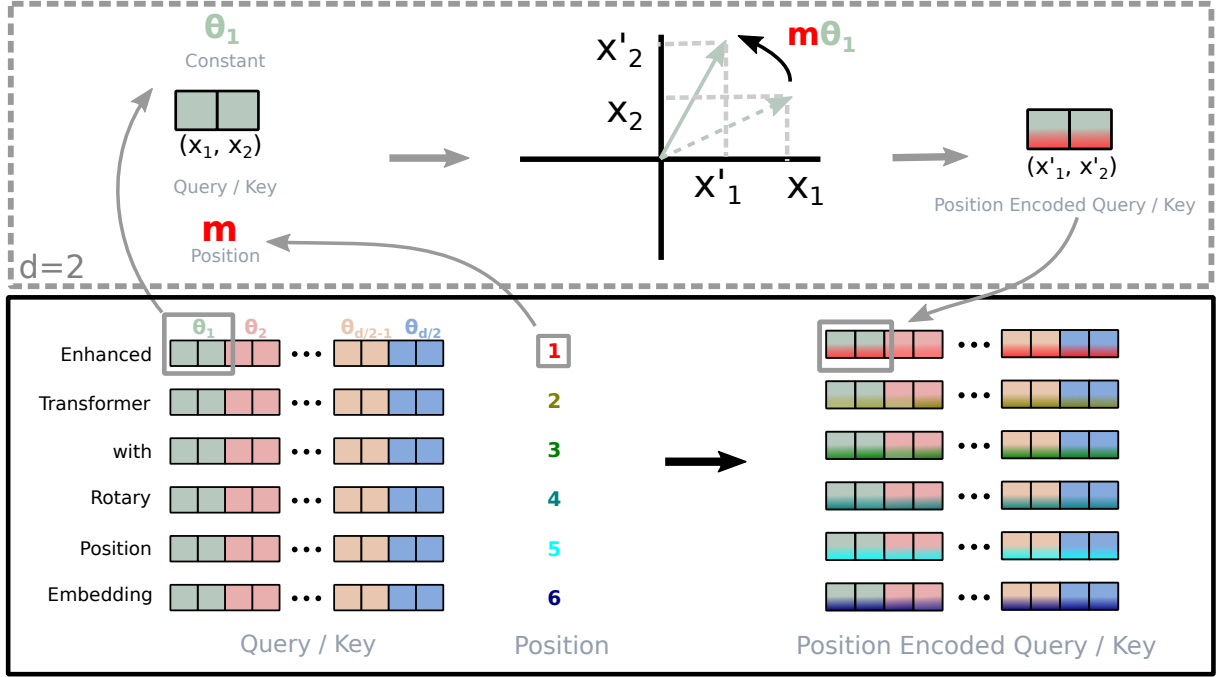
- $\theta_i = 10000^{-2(i-1)/d_h}$ tạo dải tần số hình học: các cặp chiều thấp (i nhỏ) có θ_i lớn, *quay nhanh*, nắm bắt quan hệ cục bộ giữa các token gần nhau; các cặp chiều cao (i lớn) có θ_i nhỏ, *quay chậm*, mã hóa quan hệ toàn cục. Thiết kế này phản chiếu mã hóa sinusoidal của Transformer gốc nhưng đưa thông tin vị trí vào qua phép quay thay vì phép cộng;
- $R_{\Theta,m}$ là ma trận trực giao ($R_{\Theta,m}^\top R_{\Theta,m} = I$), bảo toàn hoàn toàn độ lớn của véc-tơ – phép quay chỉ đổi hướng, không thay đổi độ dài;
- $d_h = 64$ là số chiều mỗi đầu chú ý trong NeoBERT.

Hình 2.3 minh họa cơ chế quay. Phép biến đổi và tính chất tương đối được viết gọn như Công thức (2.7):

$$q_m = R_{\Theta,m} W_q x_m, \quad k_n = R_{\Theta,n} W_k x_n, \quad q_m^\top k_n = x_m^\top W_q^\top R_{\Theta,n-m} W_k x_n, \quad (2.7)$$

trong đó $R_{\Theta,n-m} = R_{\Theta,m}^\top R_{\Theta,n}$. Tích của hai ma trận trực giao vẫn là ma trận trực giao, và tích vô hướng $q_m^\top k_n$ chỉ phụ thuộc vào $n - m$ – xác nhận RoPE thỏa mãn ràng buộc trong Công thức (2.4).

Triển khai hiệu quả. Do $R_{\Theta,m}$ thưa (chỉ $2d_h$ phần tử khác 0 trên ma trận $d_h \times d_h$), phép nhân ma trận đầy đủ tốn $O(d_h^2)$ là lãng phí. Su và cộng sự [13] cung cấp cách tính tương đương bằng phép nhân theo từng phần



Hình 2.3: Minh họa cơ chế RoPE (Nguồn: Su và cộng sự [13]).

tử (Hadamard), hoàn toàn tránh phép nhân ma trận:

$$R_{\Theta, m} x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d_h-1} \\ x_{d_h} \end{pmatrix} \odot \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d_h/2} \\ \cos m\theta_{d_h/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d_h} \\ x_{d_h-1} \end{pmatrix} \odot \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d_h/2} \\ \sin m\theta_{d_h/2} \end{pmatrix}. \quad (2.8)$$

Mỗi chiều chỉ tham gia đúng hai phép nhân và một phép cộng – tổng chi phí $O(d_h)$, dễ dàng song song hóa hoàn toàn trên GPU mà không cần dựng ma trận quay đầy đủ.

Suy giảm theo khoảng cách (long-term decay). Một tính chất quan trọng của RoPE là điểm chú ý *suy giảm tự nhiên* khi khoảng cách

$|n - m|$ tăng. Viết tích vô hướng dưới dạng tổng số phức:

$$q_m^\top k_n = \text{Re} \left[\sum_{i=1}^{d_h/2} q_{[2i-1:2i]} k_{[2i-1:2i]}^* e^{i(m-n)\theta_i} \right], \quad (2.9)$$

trong đó $q_{[2i-1:2i]}$ là cặp chiều thứ i của q biểu diễn như số phức. Áp dụng biến đổi Abel (tổng từng phần), biểu thức này bị chặn trên bởi một đại lượng tỉ lệ với $\frac{1}{d_h/2} \sum_j |S_j|$, với $S_j = \sum_{i=1}^j e^{i(m-n)\theta_i}$. Khi $|n - m|$ lớn, các số hạng $e^{i(m-n)\theta_i}$ trải đều trên vòng tròn đơn vị và triệt tiêu lẫn nhau, khiến $|S_j|$ – và theo đó điểm chú ý – giảm dần theo khoảng cách. Tính chất này hoàn toàn tự nhiên từ bản chất quay và phù hợp với trực giác ngôn ngữ học rằng hai token xa nhau nên ít liên quan hơn, mà không cần thêm bất kỳ ràng buộc kỹ thuật nào.

Mở rộng ngữ cảnh. Vì không có bảng vị trí cố định, NeoBERT nâng ngữ cảnh từ 1.024 lên 4.096 token chỉ bằng một pha huấn luyện bổ sung [5]. Xa hơn nữa, kỹ thuật YaRN [14] nội suy tần số θ_i để kéo dài ngữ cảnh mà không cần huấn luyện lại toàn bộ mô hình – một lợi thế trực tiếp từ kiến trúc không dùng bảng vị trí cố định.

Tổng kết lại, RoPE mang lại đồng thời bốn ưu điểm: (1) *tính tương đối* của điểm chú ý (Công thức (2.7)); (2) *không thêm tham số* – không có bảng véc-tơ vị trí; (3) *bảo toàn chuẩn* – phép quay trực giao không thay đổi độ lớn véc-tơ, nhất quán với triết lý chuẩn hóa theo độ lớn của RMSNorm; (4) thông tin vị trí *không nằm trong ma trận véc-tơ*, nên việc thay thế toàn bộ lớp véc-tơ tiếng Việt không đụng chạm tới cơ chế vị trí của mô hình.

2.2.4 Hàm kích hoạt SwiGLU

Trong BERT, mạng truyền thẳng của mỗi khối là hai phép chiếu với hàm kích hoạt GELU [15] ở giữa, như Công thức (2.10):

$$\text{FFN}_{\text{GELU}}(x) = \text{GELU}(xW_1 + b_1) W_2 + b_2 \quad (2.10)$$

Trong đó $W_1 \in \mathbb{R}^{d \times 4d}$ và $W_2 \in \mathbb{R}^{4d \times d}$ lần lượt mở rộng rồi thu hẹp chiều biểu diễn. Mọi chiều của biểu diễn trung gian đều đi qua cùng một phi tuyến – không có cơ chế nào cho phép mô hình *chọn lọc* thông tin truyền qua.

NeoBERT thay bằng SwiGLU [16]: thay cho một phi tuyến duy nhất, biểu diễn được tách thành hai nhánh chiếu song song – một nhánh *cổng* đi qua phi tuyến, một nhánh *giá trị* tuyến tính – rồi nhân hai nhánh với nhau theo *từng phần tử*. Mỗi phần tử của nhánh cổng là một hệ số nhân vào phần tử cùng vị trí ở nhánh giá trị: hệ số gần 0 thì gần như loại bỏ phần tử đó, hệ số lớn thì cho nó đi qua. Cách làm này thuộc họ *đơn vị tuyến tính có cổng* (Gated Linear Unit – GLU), như Công thức (2.11):

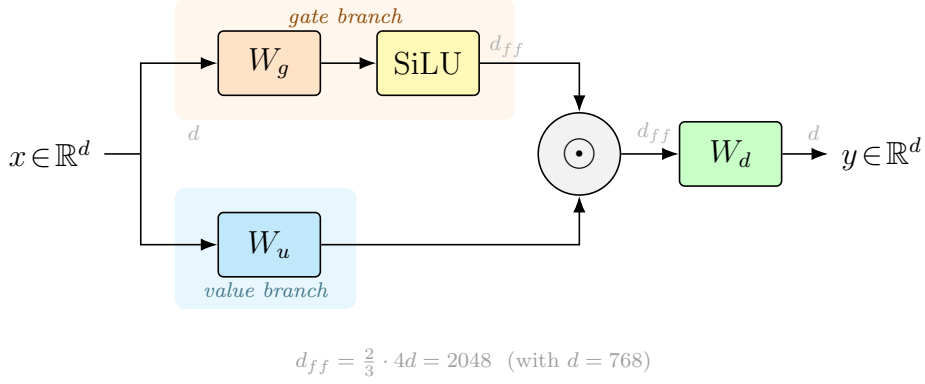
$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{SiLU}(xW_g) \odot (xW_u)) W_d, \quad \text{SiLU}(z) = z \cdot \sigma(z) \quad (2.11)$$

Trong đó:

- $W_g, W_u \in \mathbb{R}^{d \times d_{ff}}$ là hai phép chiếu song song: nhánh *cổng* (qua phi tuyến SiLU) và nhánh *giá trị* (tuyến tính);
- $\odot$ là phép nhân theo từng phần tử: mỗi phần tử của nhánh cổng quyết định phần tử cùng vị trí ở nhánh giá trị được truyền qua bao nhiêu;
- $W_d \in \mathbb{R}^{d_{ff} \times d}$ chiếu về chiều gốc; cả ba ma trận đều không dùng bias;
- σ là hàm sigmoid.

Hình 2.4 thể hiện luồng tính toán của FFN SwiGLU. Vì SwiGLU dùng *ba* ma trận thay vì hai, chiều trung gian được thu lại theo quy tắc $d_{ff} = \frac{2}{3} \cdot 4d$ để giữ nguyên ngân sách tham số [16]. Với NeoBERT: $d = 768$; nếu theo tỉ lệ $4d$ như mạng truyền thẳng thông thường thì chiều trung gian là 3.072, sau khi nhân $\frac{2}{3}$ còn $d_{ff} = 2.048$. Thực nghiệm của Shazeer cho thấy các biến thể GLU cải thiện chất lượng tiền huấn luyện và tinh chỉnh so với GELU/ReLU ở cùng chi phí; cơ chế cổng đặc biệt hữu ích với các kiến trúc

sâu (nhiều lớp, như 28 lớp của NeoBERT), nơi khả năng điều tiết luồng thông tin qua từng lớp trở nên quan trọng.



Hình 2.4: Cấu trúc cổng SwiGLU [16].

2.2.5 Chuẩn hóa Pre-RMSNorm

Cải tiến thứ tư gồm hai quyết định độc lập nhưng bổ trợ nhau: *loại chuẩn hóa* (RMSNorm thay cho LayerNorm) và *vị trí chuẩn hóa* (trước thay vì sau lớp con).

RMSNorm. LayerNorm truyền thống chuẩn hóa véc-tơ đầu vào $a \in \mathbb{R}^d$ bằng cách trừ trung bình rồi chia độ lệch chuẩn, như Công thức (2.12):

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i + \beta_i, \quad \mu = \frac{1}{d} \sum_{i=1}^d a_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (a_i - \mu)^2 + \epsilon} \quad (2.12)$$

Trong đó $g, \beta \in \mathbb{R}^d$ là hai véc-tơ tham số được điều chỉnh trong quá trình huấn luyện: g co giãn còn β dịch đầu ra sau khi chuẩn hóa. Quá trình này cần *hai lượt duyệt* qua d phần tử: lượt đầu tính μ , lượt sau tính σ (vì σ phụ thuộc μ vừa tính). Ngoài chi phí tính toán, riêng véc-tơ β đã tốn thêm d tham số, chỉ dùng để dịch đầu ra.

Zhang và Sennrich [17] chỉ ra rằng lợi ích của LayerNorm chủ yếu đến từ việc *chia theo độ lớn* (re-scaling), chứ không phải từ việc *trừ trung bình* (re-centering). Vì vậy, họ đề xuất RMSNorm: bỏ hẳn bước trừ trung bình,

chỉ chia véc-tơ cho *căn bậc hai của trung bình bình phương* các phần tử (Root Mean Square – RMS), như Công thức (2.13):

$$\bar{a}_i = \frac{a_i}{\text{RMS}(a)} g_i, \quad \text{RMS}(a) = \sqrt{\frac{1}{d} \sum_{i=1}^d a_i^2 + \epsilon} \quad (2.13)$$

Trong đó $g \in \mathbb{R}^d$ là véc-tơ tỷ lệ. RMSNorm bỏ hoàn toàn μ và β : chỉ cần *một lượt* duyệt qua d phần tử để tính tổng bình phương, không có phép trừ trung bình, và giảm đi d tham số. Thực nghiệm cho thấy RMSNorm đạt chất lượng tương đương hoặc tốt hơn LayerNorm ở cùng chi phí – do đó trở thành lựa chọn mặc định của các kiến trúc hiện đại như LLaMA, Gemma và NeoBERT.

Pre-norm. Về vị trí, BERT dùng Post-norm (chuẩn hóa *sau* khi cộng kết nối tắt), còn NeoBERT dùng Pre-norm (chuẩn hóa *trước* mỗi lớp con), như Công thức (2.14):

$$\underbrace{x_{l+1} = \text{Norm}(x_l + F_l(x_l))}_{\text{Post-norm (BERT)}} \quad \text{so với} \quad \underbrace{x_{l+1} = x_l + F_l(\text{Norm}(x_l))}_{\text{Pre-norm (NeoBERT)}} \quad (2.14)$$

Trong đó F_l là lớp con thứ l (chú ý hoặc FFN). Xiong và cộng sự [18] chứng minh rằng với Post-norm, độ lớn gradient tại thời điểm khởi tạo tăng theo độ sâu – buộc phải khởi động tốc độ học (learning rate) rất thận trọng; với Pre-norm, đường kết nối tắt là một đường dẫn thẳng, giữ nguyên tín hiệu, không bị lớp chuẩn hóa cắt ngang, nên gradient chảy ổn định qua mọi lớp. Với 28 lớp – sâu hơn nhiều so với thể hệ BERT – lựa chọn Pre-RMSNorm là điều kiện để NeoBERT huấn luyện ổn định.

Đối với đề tài, đặc điểm này để lại một ràng buộc trực tiếp: Pre-RMSNorm khiến NeoBERT *nhạy với độ lớn* của véc-tơ đầu vào. Trong sơ đồ Pre-norm, lớp véc-tơ được cộng thẳng vào luồng chính qua kết nối tắt mà không qua chuẩn hóa, nên độ lớn của nó đi trực tiếp vào phần thân và chi phối cân bằng tín hiệu ở đó. Nếu phân phối độ lớn của các hàng véc-tơ

tiếng Việt khởi tạo mới lệch xa so với thống kê của NeoBERT gốc, tín hiệu đưa vào phần thân sẽ bị sai lệch. Vì vậy, sau khi chiếu, Chương 3 có bước hiệu chỉnh độ lớn cho lớp véc-tơ về độ lớn trung bình của NeoBERT trong khi giữ nguyên hướng (Mục 3.5.4).

2.2.6 Dữ liệu và chiến lược tiền huấn luyện

Bảng 2.1 tóm tắt toàn bộ cấu hình của NeoBERT. Đây là những chi tiết quan trọng vì ViNeoBERT kế thừa trực tiếp kiến trúc này, chỉ thay thế bộ từ vựng và lớp véc-tơ tiếng Anh bằng tiếng Việt.

Bảng 2.1: Cấu hình NeoBERT [5].

Thông số	Giá trị
Số lớp Transformer	28
Kích thước ẩn d	768
Số đầu chú ý	12 (mỗi đầu $d_h = 64$)
Chiều FFN trung gian d_{ff}	2.048 (SwiGLU, sau điều chỉnh 2/3)
Tổng tham số	≈ 250 triệu (tính đủ lớp véc-tơ + thân + đầu ra)
Mã hóa vị trí	RoPE, $\theta_{\text{base}} = 10.000$
Hàm kích hoạt FFN	SwiGLU (không bias)
Chuẩn hóa	Pre-RMSNorm
Từ vựng tiếng Anh	30.522 token (WordPiece, tương thích BERT)
Ngữ cảnh tối đa	4.096 token
Dữ liệu tiền huấn luyện	RefinedWeb (600 tỷ token; đã thấy $\approx 2,1$ nghìn tỷ)
Mục tiêu huấn luyện	MLM, không có NSP; tỷ lệ che 20%; luôn dùng [MASK]
Bộ tối ưu	AdamW, $\beta = (0,9; 0,95)$, weight decay 0,1
Gradient clipping	norm tối đa 1,0
Lịch điều chỉnh tốc độ học	Cosine decay (đỉnh 6×10^{-4} , khởi động 2.000 bước, làm nguội về 10% trong 90% số bước)
Batch size hiệu dụng	~ 2 triệu token/bước
Số bước huấn luyện	≈ 1 triệu bước (pha chính)
Tối ưu bộ nhớ	FlashAttention-2
Pha mở rộng ngữ cảnh	~ 100 tỷ token ở 4.096 token (50 nghìn bước)

NeoBERT được tiền huấn luyện trên RefinedWeb [19] – kho ngữ liệu web quy mô lớn xây dựng từ CommonCrawl (khoảng 600 tỷ token); qua nhiều lượt lặp, mô hình đã thấy tổng cộng $\approx 2,1$ nghìn tỷ token trong tiền huấn luyện. Về mục tiêu huấn luyện, NeoBERT chỉ dùng MLM (bỏ NSP

theo RoBERTa [2]), tăng tỷ lệ che lên 20% [20] – cao hơn 15% của BERT, được chứng minh hiệu quả hơn cho các Encoder hiện đại – và đơn giản hóa lược đồ che bằng cách *luôn* thay token được chọn bằng [MASK] (bỏ quy tắc 80/10/10 của BERT). Tối ưu bằng AdamW [21] với $\beta = (0,9; 0,95)$, weight decay (suy giảm trọng số) 0,1, gradient clipping (cắt gradient theo chuẩn) tối đa 1,0 và lịch cosine [22] (làm nguội về 10% tốc độ học đỉnh trong 90% số bước). FlashAttention-2 [23] được dùng để giảm chi phí bộ nhớ khi huấn luyện trên chuỗi dài.

Quá trình tiền huấn luyện được chia *hai pha*: pha chính (≈ 1 triệu bước, mỗi bước ~ 2 triệu token) ở độ dài chuỗi 1.024 token, sau đó một pha ngắn (≈ 100 tỷ token, 50 nghìn bước) ở 4.096 token để kích hoạt khả năng ngữ cảnh dài. Thiết kế hai pha này là hợp lý vì huấn luyện toàn bộ ở độ dài 4.096 token sẽ tốn kém hơn nhiều: chi phí tính toán của lớp tự chú ý tăng theo bình phương độ dài chuỗi – gấp 16 lần khi chuyển từ 1.024 lên 4.096 token – trong khi phần chi phí còn lại của mô hình chỉ tăng tuyến tính theo số token (gấp 4 lần). ViNeoBERT áp dụng chiến lược hoàn toàn tương tự (nhưng ở quy mô nhỏ hơn nhiều) khi mở rộng ngữ cảnh ở Giai đoạn 3 (Chương 3 và Mục 4.5.3).

Điều quan trọng nhất cần lưu ý: qua 2,1 nghìn tỷ token tiếng Anh, phần thân Encoder của NeoBERT – 28 lớp với RoPE, SwiGLU và Pre-RMSNorm – đã học được năng lực biểu diễn ngôn ngữ rất mạnh, cũng là phần tốn kém nhất khi huấn luyện. Về mặt cấu trúc, trong toàn mô hình chỉ có *lớp véc-tơ đầu vào* và *lớp đầu ra* là gắn riêng với từ vựng tiếng Anh. Do đó, đề tài chọn *giữ nguyên* phần thân và chỉ khởi tạo lại hai lớp này cho tiếng Việt bằng SALT, thay vì huấn luyện lại toàn bộ từ đầu.

2.3 Chuyển giao không gian véc-tơ và chuyển đổi ngôn ngữ

Đối với đề tài, thách thức trung tâm không chỉ là thay bộ từ vựng tiếng Anh của NeoBERT bằng bộ từ vựng tiếng Việt. Lớp véc-tơ mới còn phải nằm đủ gần không gian biểu diễn mà phần thân Transformer của NeoBERT đã học, để quá trình tiếp tục tiền huấn luyện không bắt đầu từ một trạng thái ngẫu nhiên, kém ổn định. Vì vậy, mục này gộp hai nền tảng có liên quan chặt chẽ: hình học của không gian biểu diễn và các phương pháp khởi tạo véc-tơ cho chuyển đổi ngôn ngữ.

2.3.1 Hình học của không gian biểu diễn

Không gian véc-tơ của mô hình ngôn ngữ thường có hình học phức tạp hơn một không gian Euclid đều đặn. Ba kết quả có liên quan trực tiếp tới đề tài là: biểu diễn theo ngữ cảnh có tính bất đẳng hướng (anisotropy) [24]; các không gian đơn ngữ được huấn luyện độc lập không nhất thiết đẳng cấu (isomorphic) [25]; và ma trận véc-tơ đầu vào không luôn có cùng hình học với ma trận đầu ra khi mô hình không dùng chung trọng số [26].

Các nhận xét này dẫn tới một lựa chọn thiết kế quan trọng: không nên dùng một ánh xạ tuyến tính toàn cục để chuyển toàn bộ từ vựng tiếng Việt sang NeoBERT. Thay vào đó, đề tài ưu tiên hướng chuyển giao cục bộ theo từng token, trong đó mỗi token mới được ánh xạ dựa trên các token điểm neo gần nó về mặt ngữ nghĩa.

2.3.2 Chuyển đổi ngôn ngữ và rủi ro mất thông tin

NeoBERT được tiền huấn luyện trên tiếng Anh, còn huấn luyện lại một mô hình tương đương cho tiếng Việt từ đầu là không thực tế về chi phí. Hướng khả thi hơn là tiếp tục tiền huấn luyện (Continued Pre-training – CPT) trên ngữ liệu tiếng Việt, tương tự tinh thần thích ứng miền (domain

adaptation) của Gururangan và cộng sự [27].

Rủi ro của CPT là mô hình có thể phải học lại từ tín hiệu đầu vào nhiều nếu lớp véc-tơ mới được khởi tạo ngẫu nhiên. Các nghiên cứu về tái sử dụng mô hình cho một ngôn ngữ mới [28] và phân tích CPT gần đây [29], [30] đều gợi ý rằng điểm khởi đầu của lớp véc-tơ ảnh hưởng lớn tới eo gần nghĩa nhất độ ổn định khi huấn luyện. Vì vậy, đề tài xem khởi tạo véc-tơ là bước quyết định trước khi CPT.

Một khía cạnh thực hành quan trọng khác của CPT là *lịch điều chỉnh tốc độ học*. Lịch cosine truyền thống yêu cầu cam kết trước tổng số bước huấn luyện, kém linh hoạt khi tổng số bước chưa được biết chắc từ đầu. Lịch Warmup–Stable–Decay (WSD) [31] tách quá trình thành pha khởi động, pha ổn định ở tốc độ học đỉnh và một pha làm nguội ngắn ở cuối, cho phép kéo dài hoặc dừng huấn luyện linh hoạt; Hägele và cộng sự [32] cho thấy cách tiếp cận này đạt chất lượng tương đương cosine, trong đó dạng làm nguội theo căn bậc hai ($1 - \sqrt{f}$) cho kết quả tốt nhất ở cùng ngân sách. Ngoài ra, khi huấn luyện tiếp từ một điểm kiểm tra (checkpoint) mà tốc độ học đã được làm nguội về gần 0, cần *khởi động lại* tốc độ học – nếu để nguyên ở mức gần 0, mô hình gần như không cập nhật trọng số [30]. Các kết quả này là cơ sở cho lịch huấn luyện được thiết kế ở Chương 3.

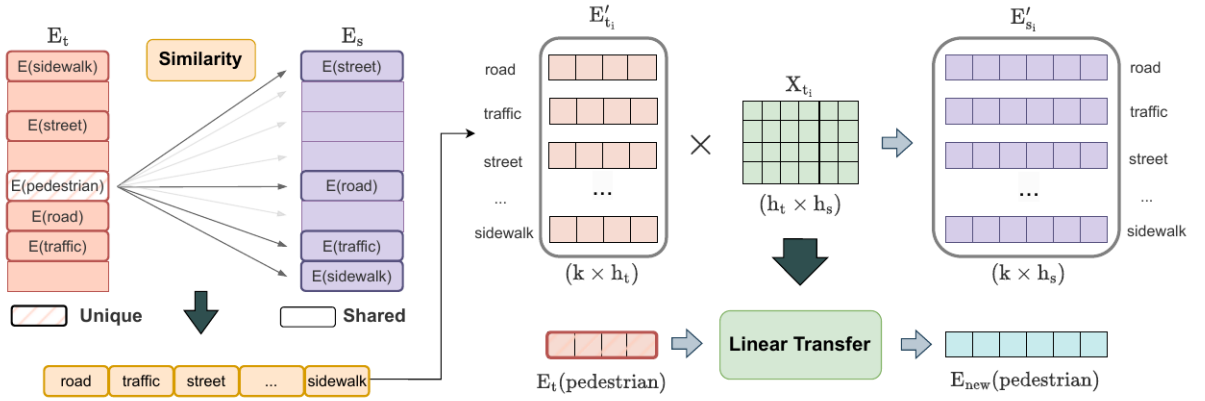
2.3.3 Khởi tạo véc-tơ dựa trên ngữ nghĩa và kỹ thuật SALT

Các phương pháp như WECHSEL [33], FOCUS [34] và OFA [35] đều theo cùng một hướng: token mới không được xem như ký hiệu rỗng, mà được khởi tạo bằng thông tin ngữ nghĩa từ token hoặc không gian véc-tơ đã biết. WECHSEL dựa trên véc-tơ tính song ngữ, FOCUS tận dụng phần từ vựng chồng lấp (overlapping vocabulary – những token có mặt ở cả hai bộ từ vựng) và Sparsemax [36], còn OFA mở rộng sang thiết lập đa ngôn ngữ lớn hơn.

SALT kế thừa tinh thần này nhưng phù hợp hơn với đề tài vì nó tận

dụng trực tiếp một mô hình đã tiền huấn luyện ở ngôn ngữ đích, thay vì chỉ dựa vào véc-tơ tĩnh, hoặc tổ hợp lại véc-tơ của các token đã có sẵn trong từ vựng gốc.

Cụ thể, SALT của Lee và cộng sự [8] khởi tạo từ vựng đích bằng cách chuyển biểu diễn từ một mô hình tiền huấn luyện ở ngôn ngữ đích sang không gian của mô hình nền cần thích ứng. Thay vì học một ánh xạ chung cho toàn bộ từ vựng, SALT dùng các token điểm neo trong phần từ vựng chồng lấp để ước lượng ánh xạ cục bộ cho từng token mới. Hình 2.5 minh họa luồng xử lý. Với mỗi token không trùng (ví dụ *pedestrian*), SALT trước hết tìm trong phần chồng lấp các điểm neo gần nghĩa nhất với nó. Vì mỗi điểm neo có sẵn một véc-tơ ở không gian ban đầu và một véc-tơ ở không gian của mô hình nền, SALT ghép các véc-tơ ban đầu thành một ma trận và các véc-tơ ở mô hình nền thành ma trận còn lại, rồi học một ánh xạ tuyến tính cục bộ X giữa chúng. Cuối cùng, SALT áp dụng X lên véc-tơ ban đầu của token để đưa nó sang không gian của mô hình nền.



Hình 2.5: Sơ đồ tổng quan SALT (Nguồn: Lee và cộng sự [8]).

Trong nghiên cứu gốc, SALT chủ yếu được kiểm chứng trên các mô hình dạng decoder như Gemma và XGLM. Đề tài kế thừa ý tưởng này nhưng áp dụng vào bối cảnh khác: mô hình nền là NeoBERT Encoder-only, ngôn ngữ đích là tiếng Việt, và lớp đầu ra của NeoBERT cần được xử lý như một ma trận riêng. Các điều chỉnh cụ thể được trình bày trong Chương 3.

2.4 Nhận xét định hướng

Từ các cơ sở trên, chúng tôi xem NeoBERT là một nền tảng phù hợp để xây dựng Encoder tiếng Việt hiện đại: phần thân Transformer đã mạnh, hỗ trợ ngữ cảnh dài, nhưng lớp từ vựng và không gian đầu vào vẫn mang thiên lệch tiếng Anh. Vì vậy, hướng tiếp cận hợp lý không phải huấn luyện lại từ đầu, mà là tái sử dụng phần thân NeoBERT và khởi tạo lại lớp véc-tơ tiếng Việt một cách phù hợp.

Nhận xét quan trọng nhất của chúng tôi là bước khởi tạo không gian véc-tơ quyết định chất lượng thích ứng ban đầu. Nếu điểm khởi đầu lệch quá xa không gian NeoBERT, quá trình CPT dễ tiêu tốn tài nguyên để sửa lỗi căn chỉnh thay vì học tiếng Việt. Do đó, Chương 3 sẽ tập trung vào một biến thể SALT phù hợp với NeoBERT, bao gồm lựa chọn tập điểm neo, chiếu cục bộ, xử lý riêng lớp đầu ra và hiệu chỉnh độ lớn của véc-tơ.

Chương 3

Phương pháp xây dựng NeoBERT cho tiếng Việt

Chương này trình bày chi tiết phương pháp xây dựng mô hình NeoBERT cho tiếng Việt (ViNeoBERT). Trước hết, chúng tôi mô tả tổng quan quy trình cùng các mô hình nguồn được sử dụng (Mục 3.1). Tiếp theo, chúng tôi giải thích lý do lựa chọn ViDeBERTa làm mô hình nguồn véc-tơ và so sánh tokenizer với PhoBERT (Mục 3.2), rồi trình bày bước giảm kích thước và đồng bộ từ vựng (Mục 3.3), quá trình trích xuất tập điểm neo – được mở rộng bằng các cặp dịch, một cải tiến của đề tài so với SALT gốc (Mục 3.4), và kỹ thuật SALT áp dụng cho cả lớp véc-tơ đầu vào lẫn lớp đầu ra của NeoBERT (Mục 3.5). Cuối cùng, chúng tôi mô tả giai đoạn căn chỉnh với phần thân đóng băng (Mục 3.6) và giai đoạn tiếp tục tiền huấn luyện (Mục 3.7). Các giá trị siêu tham số cụ thể được trình bày trong Chương 4.

3.1 Tổng quan phương pháp

3.1.1 Đặt vấn đề

Như đã phân tích trong Chương 2, việc xây dựng một Encoder hiện đại cho tiếng Việt từ NeoBERT đối mặt đồng thời ba thách thức: (1) NeoBERT

chỉ được tiền huấn luyện trên ngữ liệu tiếng Anh, nên toàn bộ lớp véc-tơ của nó phản ánh phân phối từ vựng Anh ngữ; (2) tiếp tục tiền huấn luyện với một lớp véc-tơ *khởi tạo ngẫu nhiên* để dẫn tới hiện tượng mất thông tin; và (3) huấn luyện lại từ đầu trên quy mô hàng nghìn tỷ token đòi hỏi lượng tài nguyên tính toán rất lớn. Phương pháp của đề tài giải quyết đồng thời ba thách thức này bằng cách khởi tạo lớp véc-tơ tiếng Việt *một cách phù hợp* thông qua kỹ thuật SALT, sau đó tinh chỉnh mô hình trên ngữ liệu tiếng Việt với chi phí thấp.

3.1.2 Mô hình đích và mô hình nguồn véc-tơ

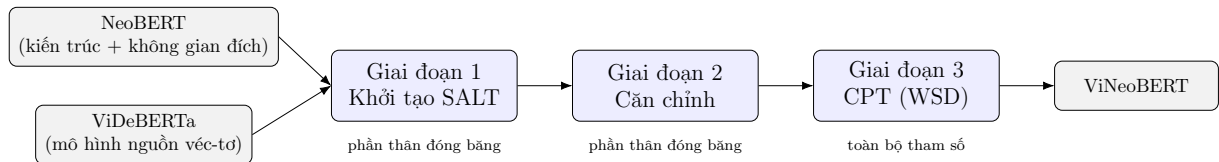
Phương pháp sử dụng hai mô hình tiền huấn luyện:

- **NeoBERT** [5] đóng vai trò *kiến trúc đích*: toàn bộ phần thân Transformer (28 lớp với RoPE, SwiGLU, Pre-RMSNorm) cùng không gian biểu diễn tiếng Anh được giữ nguyên và dùng làm hệ tọa độ đích cho phép chiếu. Một đặc điểm quan trọng là NeoBERT *không dùng chung trọng số* giữa lớp đầu ra và lớp véc-tơ đầu vào: lớp đầu ra MLM là một ánh xạ tuyến tính riêng $y = W_{\text{dec}} h + b_{\text{dec}}$ với ma trận trọng số W_{dec} và véc-tơ hệ số điều chỉnh (bias) b_{dec} *độc lập* với ma trận véc-tơ đầu vào E . Do đó, việc thích ứng sang tiếng Việt đòi hỏi khởi tạo *cả hai* ma trận: ma trận véc-tơ đầu vào E và ma trận đầu ra W_{dec} .
- **ViDeBERTa** [7] (bản **base**) đóng vai trò *mô hình nguồn véc-tơ* (donor): cung cấp không gian véc-tơ tiếng Việt chất lượng cao làm nguồn để tái sử dụng. ViDeBERTa dùng tokenizer SentencePiece theo thuật toán Unigram với kích thước từ vựng lớn (khoảng 128 nghìn token), trong đó mỗi token đi kèm một *điểm log-xác suất* (logarit của xác suất xuất hiện của token; giá trị càng cao thì token càng thường gặp trong ngữ liệu huấn luyện tiếng Việt).

3.1.3 Quy trình xây dựng ViNeoBERT

Mô hình ViNeoBERT được xây dựng qua ba giai đoạn nối tiếp, minh họa trong Hình 3.1:

1. **Khởi tạo SALT**: xây dựng tập từ vựng tiếng Việt, trích xuất tập điểm neo, rồi chiếu không gian véc-tơ của ViDeBERTa sang không gian NeoBERT để khởi tạo cả ma trận véc-tơ đầu vào lẫn ma trận đầu ra; hệ số điều chỉnh của đầu ra được khởi tạo theo tần suất token (Mục 3.3–3.5). Giai đoạn này *không* cập nhật phần thân Transformer.
2. **Căn chỉnh với phần thân đóng băng**: đóng băng toàn bộ phần thân, chỉ huấn luyện ma trận véc-tơ và ma trận đầu ra trên một lượng nhỏ ngữ liệu tiếng Việt, nhằm để hai ma trận vừa khởi tạo căn chỉnh với nhau và với phần thân (Mục 3.6).
3. **Tiếp tục tiền huấn luyện**: tinh chỉnh toàn bộ mô hình trên ngữ liệu tiếng Việt quy mô lớn với mục tiêu huấn luyện MLM theo lịch điều chỉnh tốc độ học WSD (Mục 3.7).



Hình 3.1: Quy trình xây dựng ViNeoBERT.

3.2 Lựa chọn mô hình nguồn véc-tơ và tokenizer

3.2.1 Vai trò của tokenizer trong mô hình ngôn ngữ

Tokenizer là thành phần đầu vào không thể thiếu của mô hình ngôn ngữ: nó quyết định cách *tách* văn bản thô thành các đơn vị (token) trước khi đưa vào mô hình. Chất lượng của tokenizer ảnh hưởng trực tiếp tới ba mặt: (1) *chất lượng tách token* – một tokenizer tốt cho tiếng Việt tạo ra các token mang nghĩa rõ ràng, còn tokenizer kém dễ cắt một từ sai chỗ, tạo ra những mảnh vụn vô nghĩa; (2) *độ bao phủ của bộ từ vựng* – nếu bộ từ vựng không đủ lớn, mô hình phải ghép nhiều token mới biểu diễn được một khái niệm đơn giản, khiến chuỗi dài ra và biểu diễn kém hiệu quả; và (3) *mức phù hợp cho việc chuyển giao* – bộ từ vựng của mô hình nguồn véc-tơ phải đủ phong phú để cung cấp nhiều điểm neo ngữ nghĩa chất lượng cao cho bước SALT.

Trong xử lý ngôn ngữ tự nhiên, hai thuật toán token hóa phổ biến nhất cho tiếng Việt là BPE (Byte-Pair Encoding) và Unigram SentencePiece; hai thuật toán này xây dựng bộ từ vựng theo hai hướng ngược nhau. BPE (PhoBERT dùng) đi theo hướng *gộp dần*: bắt đầu từ các ký tự đơn lẻ, rồi lặp lại việc gộp cặp ký hiệu liền kề xuất hiện nhiều nhất cho tới khi đạt kích thước từ vựng mong muốn. Unigram SentencePiece (ViDeBERTa dùng) đi theo hướng ngược lại – *loại dần*: bắt đầu từ một bộ từ vựng lớn, rồi loại bỏ dần những token ít hữu ích nhất theo tiêu chí xác suất Unigram; nhờ đó mỗi token được gán một *điểm log-xác suất* thể hiện mức độ phổ biến của nó.

3.2.2 So sánh PhoBERT và ViDeBERTa làm mô hình nguồn véc-tơ

Đáng chú ý, nghiên cứu gốc SALT [8] sử dụng chính PhoBERT làm mô hình nguồn véc-tơ tiếng Việt. Trong đề tài này, chúng tôi chủ đích chọn ViDeBERTa thay cho PhoBERT vì ba lý do chính:

1. Tokenizer xử lý trực tiếp văn bản thô, không cần bước tiền xử lý tách từ. PhoBERT dùng thuật toán BPE huấn luyện trên văn bản đã được *tách từ* sẵn bằng VnCoreNLP [37] – tức đã gom các âm tiết rời thành từ hoàn chỉnh (*word segmentation*), ví dụ “học_sinh đi học_kỳ này”; do đó mọi văn bản đưa vào PhoBERT đều phải qua bước tiền xử lý tách từ này, và một mô hình kế thừa từ vựng PhoBERT sẽ mang theo ràng buộc đó cùng nguy cơ lệch phân phối khi công cụ tách từ hoạt động sai. Ngược lại, ViDeBERTa dùng tokenizer SentencePiece Unigram thao tác ở cấp *từ con* (sub-word) trực tiếp trên chuỗi Unicode; ViNeoBERT kế thừa tokenizer này nên xử lý được văn bản thô của CulturaX – kho dữ liệu dùng trong CPT của đề tài – mà không phụ thuộc vào một công cụ tách từ riêng.

2. Từ vựng lớn hơn – điểm neo phong phú hơn. ViDeBERTa có từ vựng ≈ 128.000 token (so với 64.000 của PhoBERT), trong đó mỗi đơn vị kèm điểm log-xác suất Unigram. Từ vựng lớn hơn đồng nghĩa với nhiều ứng viên điểm neo hơn trong bước SALT – đặc biệt quan trọng cho việc bao phủ từ vựng thuần Việt và từ ghép. Bảng 3.1 tóm tắt sự khác biệt chính.

3. Không gian véc-tơ mã hóa ngữ nghĩa tiếng Việt tốt hơn. ViDeBERTa được tiền huấn luyện theo kiến trúc DeBERTaV3 [38] trên kho ngữ liệu web CC100 tiếng Việt quy mô lớn (khoảng 138GB văn bản) [7], lớn hơn nhiều lần so với 20GB (Wikipedia + tin tức) của PhoBERT. Nhờ học trên lượng dữ liệu tiếng Việt lớn hơn nhiều, không gian véc-tơ của ViDeBERTa mã hóa ngữ nghĩa tiếng Việt chính xác hơn. Do phép chiếu SALT dựa trực tiếp vào các véc-tơ điểm neo lấy từ không gian này, véc-tơ

nguồn càng mã hóa ngữ nghĩa chính xác thì các véc-tơ khởi tạo mà SALT sinh ra càng tốt.

Bảng 3.1: So sánh tokenizer của PhoBERT và ViDeBERTa.

Đặc điểm	PhoBERT	ViDeBERTa
Thuật toán token hóa	BPE	SentencePiece Unigram
Kích thước từ vựng	64.000	≈ 128.000
Đầu vào yêu cầu	Đã tách từ (VnCoreNLP)	Văn bản thô
Điểm log-xác suất Unigram	Không có	Có (dùng để giảm kích thước)
Dữ liệu tiền huấn luyện	20GB (Wikipedia + tin tức)	$\approx 138\text{GB}$ (CC100)
Tương thích CulturaX (raw)	Không trực tiếp	Có
Tiềm năng điểm neo SALT	Hạn chế	Phong phú

3.3 Giảm kích thước và đồng bộ từ vựng

Bộ từ vựng của ViDeBERTa (khoảng 128.000 token) lớn hơn nhiều so với 30.522 token của NeoBERT. Chúng tôi giảm bộ từ vựng tiếng Việt về đúng 30.522 token nhằm giữ cho ViNeoBERT có số tham số tương đương NeoBERT gốc (khoảng 250 triệu): nếu giữ nguyên 128.000 token, riêng hai ma trận véc-tơ đầu vào và đầu ra sẽ lớn hơn khoảng bốn lần, làm tăng đáng kể tổng số tham số của mô hình và chi phí huấn luyện. Quá trình này giữ lại toàn bộ các token đặc biệt (như [CLS], [SEP], [MASK], [PAD]), sau đó chọn bổ sung các token thông thường có *điểm log-xác suất Unigram cao nhất* – tức các token xuất hiện nhiều nhất trong tiếng Việt – cho tới khi đạt đúng 30.522 token.

Gọi $\mathcal{V}_s$ là tập token đặc biệt, $\mathcal{V}_n$ là tập token thông thường của ViDeBERTa, và $\text{score}(t)$ là điểm log-xác suất Unigram của token t . Tập từ vựng tiếng Việt sau giảm kích thước $\mathcal{V}_t$ được xác định như Công thức (3.1):

$$\mathcal{V}_t = \mathcal{V}_s \cup \text{TopK}(\mathcal{V}_n, \text{score}, K), \quad K = 30,522 - |\mathcal{V}_s| \quad (3.1)$$

Trong đó $\text{TopK}(\mathcal{V}_n, \text{score}, K)$ trả về K token có điểm Unigram cao nhất. Tokenizer được ghi lại với ánh xạ chỉ số mới, đồng thời lưu một ánh xạ

π : id mới $\rightarrow$ id gốc trong ViDeBERTa để truy xuất đúng hàng véc-tơ nguồn ở các bước sau. Việc so khớp chuỗi token là chính xác vì mỗi đơn vị Unigram là duy nhất trong từ vựng.

3.4 Trích xuất tập điểm neo (anchor set)

3.4.1 Vai trò của tập điểm neo

Tập điểm neo (anchor set) $\mathcal{A}$ là tập các cặp token (t_{vi}, t_{neo}) có tương ứng ngữ nghĩa rõ ràng giữa từ vựng tiếng Việt (ViDeBERTa) và từ vựng tiếng Anh (NeoBERT). Mỗi cặp điểm neo cung cấp một điểm *đã biết* của ánh xạ giữa hai không gian véc-tơ, và giữ hai vai trò trong kỹ thuật SALT:

- **Khởi tạo trực tiếp:** bản thân token điểm neo t_{vi} được khởi tạo bằng cách *sao chép nguyên* véc-tơ NeoBERT của t_{neo} , nên nằm chính xác trong không gian đích.
- **Tập giám sát:** các cặp (e_{vi}, e_{neo}) của tập điểm neo đóng vai trò tập giám sát (support set) để *học các ánh xạ tuyến tính cục bộ*, từ đó suy ra véc-tơ cho những token *không* thuộc tập điểm neo (Mục 3.5).

Do đó, chất lượng và độ phủ của tập điểm neo quyết định trực tiếp chất lượng của kỹ thuật SALT.

Trong nghiên cứu gốc, SALT [8] cũng như FOCUS [34] chỉ khai thác các token *trùng mặt chữ* (viết giống hệt nhau) giữa hai từ vựng làm điểm neo. Đối với cặp ngôn ngữ Anh–Việt vốn khác biệt lớn về hình thái, số token trùng mặt chữ là khá hạn chế và độ phủ ngữ nghĩa hẹp. Vì vậy, chúng tôi xây dựng tập điểm neo từ *ba nguồn bổ sung cho nhau*: (1) điểm neo trùng mặt chữ *đã xác minh nghĩa*, (2) điểm neo chữ số, và (3) các *cặp dịch* Anh–Việt. Trong đó, việc mở rộng tập điểm neo bằng cặp dịch là một trong những đóng góp chính của đề tài.

3.4.2 Điểm neo trùng mặt chữ và điểm neo chữ số

Điểm neo trùng mặt chữ. Theo ý tưởng gốc của SALT, các token *viết giống hệt nhau* ở cả hai từ vựng – tức trùng nhau từng ký tự, chẳng hạn “internet” có mặt trong cả từ vựng tiếng Việt lẫn tiếng Anh – là ứng viên điểm neo tự nhiên, bởi trùng mặt chữ thường hàm ý cùng khái niệm – điển hình là từ vay mượn, tên riêng và thuật ngữ khoa học (*internet, vitamin, radio*). Cụ thể, chúng tôi lấy phần giao của hai từ vựng, rồi chỉ giữ những token *ứng với một từ hoàn chỉnh* và *đủ dài* làm tập ứng viên.

Tuy nhiên, vì tiếng Việt cũng dùng hệ chữ Latin như tiếng Anh, *cùng dạng chuỗi chưa chắc cùng nghĩa*: nhiều token trùng chuỗi thực chất là hiện tượng *đồng tự khác nghĩa* (false friends). Chẳng hạn “song” trong tiếng Việt nghĩa là *song song* hoặc liên từ *nhưng*, còn “song” trong tiếng Anh nghĩa là *bài hát*; “sang” tiếng Việt nghĩa là *sang trọng*, còn trong tiếng Anh là dạng quá khứ của *sing*. Một cặp điểm neo đồng tự khác nghĩa sẽ tạo ra một điểm ánh xạ sai, làm lệch phép chiếu. Để loại các điểm neo giả này, chúng tôi *xác minh nghĩa* từng ứng viên bằng dịch máy MarianMT Việt–Anh [39]: chỉ giữ lại làm điểm neo khi bản dịch tiếng Anh *trùng khớp chính chuỗi gốc*. Một khái niệm chung thực sự (vay mượn, tên riêng) sẽ dịch về chính nó và được giữ; ngược lại, một token đồng tự khác nghĩa sẽ dịch sang một từ tiếng Anh khác và bị loại bỏ.

Điểm neo chữ số. Song song, chúng tôi giữ trực tiếp các token *số* xuất hiện ở cả hai từ vựng làm điểm neo mà *không* cần xác minh nghĩa, vì ký hiệu số mang tính phổ quát (universal) giữa các ngôn ngữ – “2024” biểu thị cùng một giá trị bất kể ngôn ngữ. Đây là nguồn điểm neo đáng tin cậy và không phụ thuộc chất lượng dịch máy.

3.4.3 Mở rộng tập điểm neo bằng cặp dịch Anh–Việt

Hai nguồn trên cho số điểm neo hạn chế và tập trung ở từ vay mượn, tên riêng và số – không bao phủ được phần lớn kho từ vựng thuần Việt.

Để mở rộng số lượng và độ phủ của tập điểm neo, chúng tôi bổ sung nguồn điểm neo thứ ba: các cặp dịch Anh–Việt, trong đó t_{vi} và t_{neo} khác dạng chuỗi nhưng tương ứng về nghĩa (ví dụ “nước” $\leftrightarrow$ “water”). Trong giai đoạn này, phải đảm bảo được số lượng và chất lượng của các điểm neo. Đây cũng là đóng góp chính của đề tài ở bước khởi tạo.

Trước khi khai thác, chúng tôi lọc để chỉ giữ các từ tiếng Việt đầy đủ và hợp lệ: loại token chứa các chữ cái không thuộc bảng chữ cái tiếng Việt (f, j, w, z), giới hạn độ dài, yêu cầu đúng một cụm nguyên âm liền mạch, và tuân thủ các quy tắc chính tả tiếng Việt (chẳng hạn k, gh, ngh chỉ đứng trước $i, e, ê, y$; q luôn đi kèm u). Bộ lọc chính tả này bảo đảm ứng viên là từ tiếng Việt thực thụ, tránh nhiều từ các chuỗi không thuộc tiếng Việt; toàn bộ quy tắc của bộ lọc được liệt kê chi tiết ở Phụ lục A. Trên tập đã lọc, cặp dịch được khai thác theo ba tầng bổ sung cho nhau:

- **Tầng 1 – dịch ngược (back-translation):** mỗi từ tiếng Việt được dịch sang tiếng Anh rồi dịch ngược trở lại tiếng Việt; cặp điểm neo chỉ được chấp nhận khi (a) kết quả dịch ngược trùng với từ tiếng Việt gốc, và (b) thuộc từ vựng NeoBERT. Ví dụ, $sách \rightarrow$ “book” $\rightarrow$ sách: dịch ngược trùng từ gốc và “book” thuộc từ vựng NeoBERT, nên nhận cặp điểm neo ($sách, “book”$).
- **Tầng 2 – từ ghép:** các từ ghép tiếng Việt (đa âm tiết) chưa được tầng trước lấy làm điểm neo được dịch trực tiếp sang tiếng Anh, giữ lại khi bản dịch là một từ thuộc từ vựng NeoBERT. Ví dụ, từ ghép *bệnh viện* dịch thành “hospital” – một từ thuộc từ vựng NeoBERT – nên nhận cặp (*bệnh viện, “hospital”*).
- **Tầng 3 – từ đơn còn lại:** những từ đơn còn lại sau 2 tầng trên được kiểm tra lại bằng cách chỉ dịch một chiều sang tiếng Anh không kiểm chứng dịch ngược sau đó được lọc thủ công qua blacklist. Mục đích của tầng này là bao phủ thêm các từ đơn có chất lượng tốt nhưng không qua được tầng 1. Do số lượng từ vựng không quá nhiều nên

tầng này được lọc thủ công qua blacklist để loại bỏ đi những từ gây nhiễu cao.

Cơ sở khoa học của phép kiểm tra *dịch ngược* ở Tầng 1 là: một từ đa nghĩa thường không dịch ngược về chính nó một cách ổn định, do bản dịch trung gian có thể ứng với một nghĩa khác. Vì vậy, ràng buộc dịch ngược trùng khớp tự nhiên loại bỏ phần lớn các từ đa nghĩa, chỉ giữ lại các cặp tương ứng một–một đáng tin cậy. Ràng buộc “bản dịch là một từ đơn thuộc từ vựng NeoBERT” bảo đảm mỗi cặp điểm neo ứng đúng *một* token NeoBERT.

Đối với tầng 2 và tầng 3 mục đích chính vẫn là mở rộng số lượng điểm neo (nếu chỉ dựa vào tầng 1 thì sẽ không đảm bảo được số lượng). Tầng 2 được dựa trên cơ sở từ ghép trong tiếng Việt thường mang nghĩa cụ thể và ít đa nghĩa hơn từ đơn, bởi sự kết hợp của nhiều âm tiết giúp thu hẹp ngữ cảnh ngữ nghĩa và giảm hiện tượng một từ biểu đạt nhiều khái niệm khác nhau. Tầng 3 hướng tới việc tăng độ bao phủ tập neo đối với các từ đơn còn lại dựa trên thực tế một số từ đơn có nghĩa rõ ràng nhưng không vượt qua được phép kiểm tra dịch ngược do hệ thống dịch hoặc do từ đơn đó tồn tại nhiều cách diễn đạt tương đương.

Nhìn chung, ba tầng khai thác được thiết kế theo nguyên tắc đánh đổi giữa độ chính xác và độ phủ. Tầng 1 ưu tiên độ tin cậy thông qua ràng buộc dịch ngược, Tầng 2 mở rộng số lượng điểm neo bằng cách khai thác đặc tính ngữ nghĩa ổn định của từ ghép tiếng Việt, trong khi Tầng 3 bổ sung các trường hợp còn thiếu thông qua kiểm duyệt thủ công. Sự kết hợp này xây dựng nên một tập điểm neo tối ưu cả về số lượng lẫn chất lượng.

Tập điểm neo cuối cùng là *hợp* của ba nguồn – điểm neo trùng mặt chữ đã xác minh, điểm neo chữ số và cặp dịch Anh–Việt – sau khi loại các cặp trùng lặp, thu được ánh xạ $t_{vi} \mapsto t_{neo}$ dùng thống nhất cho cả lớp véc-tơ đầu vào lẫn lớp đầu ra ở Mục 3.5. Thực nghiệm khởi tạo cho thấy việc mở rộng tập điểm neo bằng cặp dịch làm tăng đáng kể số lượng điểm neo và độ phủ ngữ nghĩa, qua đó cải thiện chất lượng khởi tạo so với phương án chỉ dùng điểm neo trùng mặt chữ như nghiên cứu gốc.

Để minh họa cụ thể ba nguồn điểm neo, xét bốn token tiếng Việt đại diện. Token *internet* trùng dạng chuỗi ở cả hai từ vựng và dịch máy trả về đúng “internet”, nên được giữ làm **điểm neo trùng mặt chữ** và khởi tạo bằng chính véc-tơ NeoBERT của “internet”. Token số *2024* xuất hiện ở cả hai từ vựng và được nhận trực tiếp làm **điểm neo chữ số** mà không cần qua dịch. Token thuần Việt *nước* khác hoàn toàn về dạng chuỗi so với “water”: ở Tầng 1, dịch máy cho “water”, dịch ngược cho lại “nước”, và “water” là một từ đơn thuộc từ vựng NeoBERT – cả ba điều kiện thỏa mãn nên cặp (*nước*, *water*) được thêm làm **cặp dịch**. Ngược lại, token đồng tự khác nghĩa “song” (tiếng Việt: *song song*) dịch sang “parallel” chứ không phải “song”, nên bị loại khỏi tập điểm neo trùng mặt chữ. Thuật toán 3.1 tóm tắt toàn bộ quy trình trích xuất tập điểm neo từ ba nguồn.

Thuật toán 3.1 Trích xuất tập điểm neo từ ba nguồn

Require: Từ vựng tiếng Việt $\mathcal{V}_t$, từ vựng NeoBERT $\mathcal{V}_{\text{neo}}$; dịch máy MT; bộ lọc chính tả tiếng Việt $\text{vi}?(.)$

Ensure: Tập điểm neo $\mathcal{A}$ dạng ánh xạ $t_{\text{vi}} \mapsto t_{\text{neo}}$

```

1:  $\mathcal{A} \leftarrow \emptyset$ 
2: for all từ đầy đủ  $t \in \mathcal{V}_t \cap \mathcal{V}_{\text{neo}}$  do                                ▷ (A) điểm neo trùng mặt chữ
3:   if  $\text{MT}(t) = t$  then                                                  ▷ loại đồng tự khác nghĩa
4:      $\mathcal{A} \leftarrow \mathcal{A} \cup \{(t, t)\}$ 
5: for all token số  $t \in \mathcal{V}_t \cap \mathcal{V}_{\text{neo}}$  do                                ▷ (B) điểm neo chữ số
6:    $\mathcal{A} \leftarrow \mathcal{A} \cup \{(t, t)\}$ 
7: for all từ Việt hợp lệ  $t \in \mathcal{V}_t$  thỏa  $\text{vi}(t)$  do                        ▷ (C) cặp dịch
8:    $e \leftarrow \text{MT}(t)$ 
9:   if  $e$  là từ đơn và  $e \in \mathcal{V}_{\text{neo}}$  và  $\text{MT}^{-1}(e) = t$  then
10:     $\mathcal{A} \leftarrow \mathcal{A} \cup \{(t, e)\}$                                      ▷ dịch ngược trùng khớp
11: return  $\mathcal{A}$  sau khi khử các cặp trùng lặp

```

3.5 Khởi tạo lớp véc-tơ đầu vào và lớp đầu ra bằng kỹ thuật SALT

Mục này trình bày kỹ thuật SALT – bước khởi tạo cốt lõi. Với mỗi token tiếng Việt *không thuộc* tập điểm neo, chúng tôi xây dựng véc-tơ của nó bằng một ánh xạ tuyến tính cục bộ học từ các điểm neo lân cận về mặt ngữ nghĩa. Các token điểm neo và token đặc biệt được sao chép trực tiếp véc-tơ tương ứng từ NeoBERT.

3.5.1 Chọn các điểm neo gần nghĩa nhất bằng Sparse-max

Để xác định các điểm neo liên quan tới một token đích t , chúng tôi đo độ tương đồng ngữ nghĩa trong không gian véc-tơ từ tính fastText [40], [41]. Gọi $\mathbf{f}_t$ là véc-tơ fastText của t và $\{\mathbf{f}_a\}_{a \in \mathcal{A}}$ là véc-tơ fastText của các điểm neo. Độ tương đồng cosine và trọng số điểm neo được tính như Công thức (3.2):

$$s_{t,a} = \frac{\mathbf{f}_t^\top \mathbf{f}_a}{\|\mathbf{f}_t\| \|\mathbf{f}_a\|}, \quad \mathbf{w}_t = \text{sparsemax}(\mathbf{s}_t), \quad \mathcal{S}_t = \{a : w_{t,a} > 0\} \quad (3.2)$$

Trong đó phép *sparsemax* [36] biến véc-tơ độ tương đồng $\mathbf{s}_t$ thành một bộ trọng số không âm có tổng bằng 1 – giống softmax, nhưng khác ở chỗ nó *đặt hẳn phần lớn trọng số về 0*: chỉ một tập con nhỏ $\mathcal{S}_t$ gồm các điểm neo liên quan nhất mới nhận trọng số khác 0. Việc dùng Sparsemax thay cho softmax có lý do thực tế cụ thể: softmax luôn gán trọng số khác 0 cho *tất cả* các điểm neo, kể cả những điểm neo ở rất xa token đích về mặt ngữ nghĩa – khiến ánh xạ tuyến tính cục bộ bị nhiễu bởi các điểm không liên quan. Sparsemax tự động đặt trọng số bằng 0 cho các điểm neo có độ tương đồng thấp; chỉ tập con $\mathcal{S}_t$ thực sự gần nghĩa mới tham gia phép chiếu, giúp ma trận X_t phản ánh đúng quan hệ tuyến tính cục bộ quanh

token t .

3.5.2 Xây dựng véc-tơ cho từng token bằng ánh xạ tuyến tính cục bộ

Như đã lập luận ở Mục 2.3.1, do hai không gian véc-tơ là bất đẳng hướng và không đẳng cấu, một ánh xạ tuyến tính toàn cục duy nhất là không đủ. Thay vào đó, SALT học một *ánh xạ tuyến tính riêng cho từng token* từ tập điểm neo được chọn. Gọi $A_t^{\text{src}} \in \mathbb{R}^{k \times h}$ là ma trận xếp chồng các véc-tơ *nguồn* (ViDeBERTa) của $k = |\mathcal{S}_t|$ điểm neo trong $\mathcal{S}_t$, và $A_t^{\text{tgt}} \in \mathbb{R}^{k \times h}$ là ma trận xếp chồng các véc-tơ *đích* (NeoBERT) tương ứng. Khi số điểm neo đủ lớn ($k \geq k_0$, chúng tôi chọn $k_0 = 8$ qua thực nghiệm – giá trị nhỏ hơn khiến phép khớp bình phương tối thiểu kém ổn định và chất lượng chiếu giảm đáng kể), ánh xạ cục bộ X_t và véc-tơ kết quả $\mathbf{e}_t$ được xác định như Công thức (3.3):

$$X_t = (A_t^{\text{src}})^+ A_t^{\text{tgt}}, \quad \mathbf{e}_t = \mathbf{d}_t X_t \quad (3.3)$$

Ở đây X_t là ánh xạ tuyến tính *khớp bình phương tối thiểu*: nó biến các véc-tơ nguồn của điểm neo về sát nhất với véc-tơ đích tương ứng. Ký hiệu $(\cdot)^+$ là giả nghịch đảo Moore–Penrose; khi các điểm neo quá ít hoặc trùng lặp thông tin nên có nhiều ánh xạ cùng khớp, phép này tự chọn ánh xạ ổn định nhất (các hệ số có độ lớn nhỏ nhất), tránh hệ số phóng đại gây nhiễu. Véc-tơ $\mathbf{d}_t \in \mathbb{R}^h$ là véc-tơ nguồn của token t và h là kích thước ẩn. Ánh xạ X_t học quan hệ tuyến tính giữa hai không gian *cục bộ quanh token t* , dựa trên các điểm neo gần nghĩa nhất với nó.

Khi số điểm neo không đủ ($k < k_0$), phép khớp bình phương tối thiểu trở nên kém ổn định; khi đó chúng tôi chuyển sang dùng *trung bình có trọng số Sparsemax* của các véc-tơ đích, như Công thức (3.4):

$$\mathbf{e}_t = \frac{\sum_{a \in \mathcal{S}_t} w_{t,a} \mathbf{a}^{\text{tgt}}}{\sum_{a \in \mathcal{S}_t} w_{t,a}} \quad (3.4)$$

Trường hợp token không có véc-tơ fastText hợp lệ, véc-tơ được khởi tạo ngẫu nhiên theo phân phối chuẩn khớp với thống kê của NeoBERT.

Toàn bộ kỹ thuật SALT cho *một* ma trận đích – áp dụng lần lượt cho ma trận véc-tơ đầu vào và ma trận đầu ra ở Mục 3.5.3 – được tóm tắt trong Thuật toán 3.2.

Thuật toán 3.2 Phương pháp SALT theo từng token cho một ma trận đích

Require: Từ vựng $\mathcal{V}_t$; tập điểm neo $\mathcal{A}$; véc-tơ nguồn $\{\mathbf{d}_t\}$; véc-tơ đích NeoBERT $\{\mathbf{a}^{\text{tgt}}\}$; véc-tơ fastText $\{\mathbf{f}_t\}$; ngưỡng k_0 ; chuẩn trung bình μ_{neo}

Ensure: Ma trận khởi tạo $E \in \mathbb{R}^{|\mathcal{V}_t| \times h}$

```

1: for all token  $t \in \mathcal{V}_t$  do
2:   if  $t$  là điểm neo hoặc token đặc biệt then
3:      $E[t] \leftarrow \mathbf{a}_t^{\text{tgt}}$  ▷ sao chép nguyên véc-tơ NeoBERT
4:   else if  $t$  không có véc-tơ fastText then
5:      $E[t] \leftarrow$  mẫu ngẫu nhiên khớp thống kê NeoBERT
6:   else
7:      $s_{t,a} \leftarrow \cos(\mathbf{f}_t, \mathbf{f}_a)$  cho mọi  $a \in \mathcal{A}$ 
8:      $\mathbf{w}_t \leftarrow \text{sparsemax}(\mathbf{s}_t)$ ;  $\mathcal{S}_t \leftarrow \{a : w_{t,a} > 0\}$ 
9:     if  $|\mathcal{S}_t| \geq k_0$  then
10:       $X_t \leftarrow (A_t^{\text{src}})^+ A_t^{\text{tgt}}$ ;  $E[t] \leftarrow \mathbf{d}_t X_t$  ▷ ánh xạ cục bộ
11:    else
12:       $E[t] \leftarrow (\sum_a w_{t,a} \mathbf{a}_a^{\text{tgt}}) / \sum_a w_{t,a}$  ▷ trung bình có trọng số
13:    if ma trận đích là ma trận véc-tơ  $E$  then
14:       $E[t] \leftarrow E[t] \cdot \mu_{\text{neo}} / \|E[t]\|$  ▷ hiệu chỉnh độ lớn; ma trận đầu
      ra dùng  $\gamma$  thay thế
15: return  $E$ 

```

3.5.3 Áp dụng phép chiếu cho cả lớp véc-tơ đầu vào và lớp đầu ra

Đây là điểm khác biệt then chốt so với SALT gốc. Vì đầu ra của NeoBERT *không dùng chung trọng số* với lớp véc-tơ đầu vào, ma trận đầu ra W_{dec} là một ma trận độc lập với hình học riêng – và theo kết quả về

hiện tượng thoái hóa biểu diễn (Mục 2.3.1), nó *không* thể được khởi tạo bằng cách sao chép ma trận véc-tơ. Do đó, chúng tôi áp dụng *cùng một quy trình chiếu SALT theo từng token* cho cả hai ma trận, chỉ khác ở tập véc-tơ đích:

- Với **ma trận véc-tơ đầu vào** E : véc-tơ đích A_t^{tgt} lấy từ các *véc-tơ đầu vào* NeoBERT của các điểm neo.
- Với **ma trận đầu ra** W_{dec} : véc-tơ đích A_t^{tgt} lấy từ các *véc-tơ đầu ra* NeoBERT của các điểm neo.

Nhờ vậy, mỗi ma trận được khởi tạo trong đúng không gian của nó, phản ánh trung thực quan hệ ngữ nghĩa mà các điểm neo thiết lập trong từng không gian.

3.5.4 Tinh chỉnh kết quả khởi tạo: độ lớn véc-tơ, hệ số điều chỉnh và trọng số lớp đầu ra

Sau khi chiếu SALT cho cả hai ma trận, chúng tôi thực hiện ba bước tinh chỉnh trên kết quả khởi tạo, trình bày lần lượt dưới đây.

Hiệu chỉnh độ lớn véc-tơ cho khớp với NeoBERT. Như đã lưu ý ở Mục 2.2.5, NeoBERT dùng Pre-RMSNorm vốn chuẩn hóa theo độ lớn của véc-tơ; do đó độ lớn (norm) của mỗi hàng véc-tơ ảnh hưởng trực tiếp đến hành vi mô hình. Các véc-tơ sinh ra từ kỹ thuật SALT thường có độ lớn lệch so với phân phối của NeoBERT. Để tránh gây bất ổn định, chúng tôi hiệu chỉnh độ lớn của mỗi hàng *không phải điểm neo* về độ lớn trung bình của NeoBERT trong khi giữ nguyên hướng, như Công thức (3.5):

$$\mathbf{e}_t \leftarrow \mathbf{e}_t \cdot \frac{\mu_{\text{neo}}}{\|\mathbf{e}_t\|}, \quad \mu_{\text{neo}} = \frac{1}{|\mathcal{V}_{\text{neo}}|} \sum_v \|E_v^{\text{neo}}\| \quad (3.5)$$

Trong đó μ_{neo} là độ lớn trung bình của các hàng véc-tơ NeoBERT. Các hàng điểm neo và token đặc biệt được giữ nguyên vì chúng đã khớp chính xác với không gian NeoBERT.

Khởi tạo hệ số điều chỉnh lớp đầu ra theo tần suất token tiếng Việt. Lớp đầu ra của NeoBERT có một véc-tơ hệ số điều chỉnh b_{dec} mã hóa *phân phối tần suất* của token. Các phương pháp khởi tạo trước đây thường để hệ số điều chỉnh này bằng 0, khiến mô hình ở bước đầu phải tốn thêm bước huấn luyện để học lại phân phối tần suất biên. Đề tài khởi tạo b_{dec} theo *log-tần-suất token* (unigram log-frequency) của tiếng Việt theo phương pháp của Meister và cộng sự [42], như Công thức (3.6):

$$b_{\text{dec},v} = \log p_v, \quad p_v = \frac{c_v + 1}{\sum_{u \in \mathcal{V}_t} c_u + |\mathcal{V}_t|} \quad (3.6)$$

Trong đó c_v là số lần xuất hiện của token v trong ngữ liệu tiếng Việt (đếm qua 20.000 văn bản CulturaX-vi), và phép cộng 1 là làm trơn Laplace. Các tần suất được đếm bằng chính tokenizer sẽ dùng khi huấn luyện, để chỉ số token trong b_{dec} khớp đúng với từ vựng mà ma trận véc-tơ đầu vào và ma trận đầu ra sử dụng.

Cơ sở lý thuyết của cách khởi tạo này đến từ một quan sát của Meister và cộng sự [42]: lớp tuyến tính cuối phân rã tự nhiên thành *tích của hai phân phối* (product of experts). Với mô hình ngôn ngữ có mặt nạ, xét ngữ cảnh hai chiều $\mathbf{c}$ bao quanh vị trí cần dự đoán, ta có:

$$p_\theta(y \mid \mathbf{c}) \propto \underbrace{e^{W\phi(\mathbf{c})}}_{p_{W\phi}(\cdot \mid \mathbf{c}) \text{ (ngữ cảnh)}} \cdot \underbrace{e^b}_{p_b(\cdot) \text{ (tần suất)}} \quad (3.7)$$

Thành phần p_b là phân phối vô điều kiện hoàn toàn độc lập với ngữ cảnh. Khi $b_{\text{dec}} = \log p_v$, thành phần này mã hóa phân phối unigram tiếng Việt, nhờ đó phần còn lại $p_{W\phi}$ được *giải phóng* để tập trung học thông tin ngữ nghĩa, thay vì phải học lại tần suất xuất hiện của token. Điều này có thể diễn đạt chặt chẽ qua gradient hàm mất mát theo hệ số điều chỉnh:

$$\frac{\partial \mathcal{L}}{\partial b_v} = \mathbb{E}[\text{softmax}(y)_v] - \hat{p}_v \quad (3.8)$$

tức hiệu giữa phân phối biên mô hình đang dự đoán và phân phối biên thực nghiệm của dữ liệu. Khi $b_{\text{dec}} = \log p_v$ và γ đủ nhỏ, $\mathbb{E}[\text{softmax}(y)_v] \approx p_v \approx \hat{p}_v$, nên gradient của b_v xấp xỉ bằng không ngay từ bước đầu: toàn bộ tín hiệu gradient được dồn vào W_{dec} và phần thân Transformer để học ngữ cảnh và ngữ nghĩa, thay vì tiêu tốn vào việc hiệu chỉnh phân phối biên. Meister và cộng sự xác nhận thực nghiệm rằng: (i) ngay từ những bước đầu, mô hình ngôn ngữ vốn tự kéo đầu ra về phân phối unigram bất kể ngữ cảnh, và quá trình này tốn hàng trăm đến hàng nghìn bước gradient nếu khởi tạo $b = \mathbf{0}$; (ii) khởi tạo theo log-unigram giúp mô hình *học nhanh hơn* – điểm BLEU (Bilingual Evaluation Understudy) trên tập phát triển tăng sớm hơn – và đạt điểm BLEU cuối cao hơn trên hầu hết các cặp ngôn ngữ được thử nghiệm.

Khi kết hợp với việc giảm biên độ trọng số đầu ra (Mục 3.5.4) để thành phần W_{dec} không lấn át hệ số điều chỉnh, hàm mất mát khởi điểm xấp xỉ *entropy unigram* của tiếng Việt ($\approx 7,3$ nat), thấp hơn đáng kể so với mức ngẫu nhiên $\log |\mathcal{V}_t|$ ($\approx 10,3$ nat với $|\mathcal{V}_t| = 30.522$). Cách khởi tạo hệ số điều chỉnh theo một giá trị biết trước để ổn định giai đoạn đầu huấn luyện cũng đã được dùng ở lĩnh vực khác, chẳng hạn Lin và cộng sự [43] áp dụng cho bài toán phát hiện đối tượng.

Nhân trọng số lớp đầu ra với một hệ số nhỏ để ổn định giai đoạn huấn luyện đầu. Phân phối tần suất ở trên chỉ phát huy tác dụng nếu ở bước khởi đầu, thành phần logit do trọng số W_{dec} sinh ra *không lấn át* hệ số điều chỉnh b_{dec} . Ngay sau phép chiếu, các hàng của W_{dec} tuy đã mang cấu trúc ngữ nghĩa từ SALT nhưng chưa từng được huấn luyện cùng phần thân trên tiếng Việt, nên biên độ logit của chúng chưa đáng tin cậy: nếu giữ nguyên, các logit “nhiều” này sẽ lấn át tín hiệu tần suất. Đây là điểm khác biệt then chốt so với huấn luyện từ đầu: trong ngữ cảnh đó, W_{dec} được khởi tạo ngẫu nhiên với giá trị gần 0, nên điều kiện “phân phối tần suất chi phối” thỏa mãn tự nhiên mà không cần thêm can thiệp [42]. Khi chuyển một mô hình sang ngôn ngữ mới, W_{dec} kế thừa biên độ lớn từ không gian NeoBERT tiếng Anh, khiến điều kiện đó không còn tự nhiên

thỏa mãn và phải được khôi phục thủ công – đây là nhu cầu đặc thù của bài toán chuyển giao lớp véc-tơ mà phương pháp đề xuất cần giải quyết. Vì vậy, chúng tôi *giảm biên độ* trọng số đầu ra bằng cách nhân với một hệ số vô hướng γ như Công thức (3.9):

$$y = \gamma W_{\text{dec}} h + b_{\text{dec}}, \quad \gamma = 0,1 \quad (3.9)$$

Trong đó h là biểu diễn ẩn ở đầu ra của phần thân và γ là hệ số tỉ lệ áp dụng một lần lên W_{dec} tại thời điểm khởi tạo. Cơ sở khoa học của lựa chọn này gồm hai vế:

- **Gần 0 – để phân phối tần suất chi phối:** với γ nhỏ, $\|\gamma W_{\text{dec}} h\|$ trở nên nhỏ so với biên độ của b_{dec} , nên dự đoán ban đầu do phân phối tần suất chi phối: mất mát khởi điểm xấp xỉ entropy unigram của tiếng Việt, mô hình *bỏ qua* giai đoạn phải học lại phân phối biên và dồn toàn bộ tín hiệu gradient vào việc học ngữ nghĩa.
- **Khác 0 – để giữ ngữ nghĩa:** phép nhân vô hướng bảo toàn *hướng* của các hàng W_{dec} tại thời điểm khởi tạo, nên cấu trúc lân cận ngữ nghĩa mà kỹ thuật SALT dựng nên được giữ làm *điểm xuất phát* cho lớp đầu ra – một khởi đầu tốt hơn so với một lớp đầu ra ngẫu nhiên, giúp mô hình bắt đầu từ một hướng ngữ nghĩa hợp lý thay vì phải học lại từ đầu.

Giả thuyết này được kiểm chứng bằng thực nghiệm trong Chương 4: khi giữ nguyên biên độ ($\gamma = 1$) hoặc chỉ giảm còn một nửa ($\gamma = 0,5$), mất mát khởi điểm cao hơn rõ rệt – với $\gamma = 1$ gần chạm mức đoán ngẫu nhiên $\log |\mathcal{V}_t|$, với $\gamma = 0,5$ vẫn cao hơn nhiều mức entropy unigram – và, quan trọng hơn, quá trình huấn luyện bị *mắc kẹt* ở mức mất mát cao, không hội tụ về vùng của các phương án tốt. Hiện tượng mắc kẹt này không chỉ là hệ quả của điểm khởi đầu cao: một mô hình khởi tạo ngẫu nhiên chuẩn cũng bắt đầu gần $\log |\mathcal{V}_t|$ nhưng vẫn hội tụ bình thường. Khi γ lớn, các logit do W_{dec} (chưa từng căn chỉnh trên tiếng Việt) sinh ra lấn át tín hiệu

tần suất và ở giai đoạn đầu chủ yếu là nhiễu, khiến những cập nhật đầu tiên có biên độ rất lớn nhưng mang ít thông tin hữu ích – chuẩn gradient đo được ở những bước đầu (Chương 4) phản ánh đúng điều này. Trong khi đó, $\gamma = 0,1$ vừa cho khởi điểm thấp vừa hội tụ nhanh. Do đó $\gamma = 0,1$ được chọn cho toàn bộ quy trình.

3.6 Giai đoạn căn chỉnh với phần thân đóng băng

Sau bước khởi tạo, ma trận véc-tơ và ma trận đầu ra của tiếng Việt là *mới* đối với phần thân Transformer đã được huấn luyện trên tiếng Anh. Nếu lập tức tinh chỉnh toàn bộ mô hình, phần thân có thể bị kéo theo để thích ứng với hai ma trận mới còn chưa ổn định, làm gia tăng nguy cơ mất thông tin (Mục 2.3.2). Vì vậy, chúng tôi chèn một *giai đoạn căn chỉnh với phần thân đóng băng*: đóng băng toàn bộ phần thân Transformer (28 lớp encoder – tức toàn bộ mô hình trừ ma trận véc-tơ đầu vào và ma trận đầu ra) và chỉ huấn luyện ma trận véc-tơ đầu vào, ma trận đầu ra và hệ số điều chỉnh của nó trên một lượng nhỏ ngữ liệu tiếng Việt với mục tiêu MLM. Nói cách khác, giai đoạn này giữ nguyên trọng số bên trong và chỉ điều chỉnh hai ma trận mới (véc-tơ đầu vào và đầu ra) cho khớp với không gian biểu diễn của phần thân.

Ý tưởng của bước này là để hai ma trận vừa khởi tạo *căn chỉnh với nhau và với phần thân đang đóng băng* trong không gian biểu diễn trước khi cho phép phần thân thay đổi; nhờ đó điểm khởi đầu của giai đoạn tiếp tục tiền huấn luyện ổn định hơn. Cách làm này theo tinh thần chiến lược *đóng băng rồi thích ứng* của de Vries và Nissim [28], vốn cho thấy việc tách giai đoạn học lớp véc-tơ khỏi giai đoạn tinh chỉnh toàn bộ giúp tái sử dụng mô hình hiệu quả hơn. Thực nghiệm ở Chương 4 cho thấy giai đoạn căn chỉnh này cải thiện chất lượng lớp véc-tơ khởi tạo, nên chúng tôi giữ nó trong quy trình.

3.7 Tiếp tục tiền huấn luyện (Continued Pre-Training)

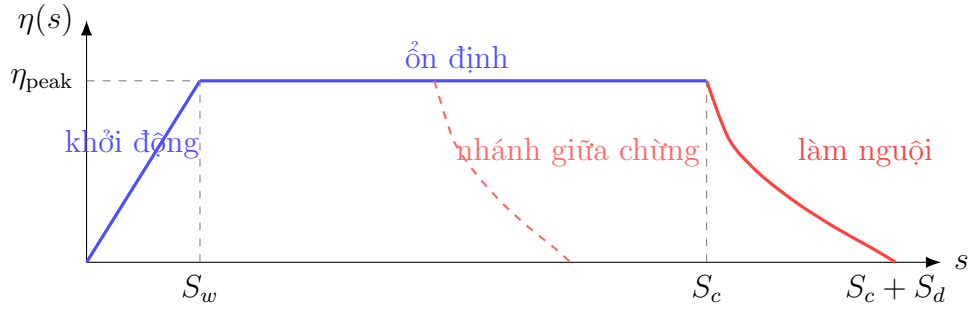
Ở giai đoạn cuối, toàn bộ mô hình được tinh chỉnh trên ngữ liệu tiếng Việt quy mô lớn CulturaX với mục tiêu MLM. Theo Gururangan và cộng sự [27], việc tiếp tục tiền huấn luyện trên dữ liệu miền đích mang lại cải thiện ổn định; ở đây miền đích là toàn bộ ngôn ngữ tiếng Việt.

Lịch điều chỉnh tốc độ học WSD. Chúng tôi sử dụng lịch *Warmup-Stable-Decay* (WSD) [31] thay cho lịch cosine cố định, vì WSD tách pha ổn định khỏi pha làm nguội nên cho phép kéo dài huấn luyện tùy ý mà không phải cam kết trước tổng số bước. Tốc độ học η theo bước s được định nghĩa như Công thức (3.10):

$$\eta(s) = \begin{cases} \eta_{\text{peak}} \cdot \frac{s}{S_w} & \text{nếu } s < S_w \quad (\text{khởi động}) \\ \eta_{\text{peak}} & \text{nếu } S_w \leq s < S_c \quad (\text{ổn định}) \\ \eta_{\text{peak}} \cdot \left(1 - \sqrt{\frac{s-S_c}{S_d}}\right) & \text{nếu } s \geq S_c \quad (\text{làm nguội}) \end{cases} \quad (3.10)$$

Trong đó S_w là mốc kết thúc khởi động, S_c là mốc bắt đầu làm nguội, và S_d là độ dài giai đoạn làm nguội. Pha *ổn định* giữ tốc độ học ở đỉnh trong phần lớn quá trình, còn pha *làm nguội* chỉ chiếm một phần nhỏ cuối ngân sách và dùng dạng căn bậc hai $1 - \sqrt{f}$ (với $f \in [0, 1]$ là phần bước đã trôi qua trong pha làm nguội) – dạng mà Hägele và cộng sự [32] cho thấy đạt chất lượng tốt nhất trên tác vụ ứng dụng ở một ngân sách cho trước. Hình 3.2 minh họa hình dạng của lịch và ưu điểm cốt lõi của nó: tại bất kỳ điểm nào trong pha ổn định đều có thể “rẽ nhánh” một pha làm nguội ngắn để thu về mô hình hoàn chỉnh, trong khi nhánh chính tiếp tục huấn luyện – điều mà lịch cosine (phải cam kết trước tổng số bước) không làm được.

Khởi động lại tốc độ học. Một điểm kiểm tra đã được làm nguội về



Hình 3.2: Lịch điều chỉnh tốc độ học Warmup–Stable–Decay (WSD).

tốc độ học gần 0 gần như không thể cập nhật trọng số nếu huấn luyện tiếp mà không xử lý. Do đó, mỗi khi tiếp tục huấn luyện trên dữ liệu mới (ví dụ một pha làm nguội trên dữ liệu mới), tốc độ học được *khởi động lại* rồi làm nguội lại theo đúng khuyến nghị của Ibrahim và cộng sự [30]. Ngoài ra, khi huấn luyện tiếp, trạng thái bộ tối ưu AdamW (các mô-men bậc nhất và bậc hai, với $\beta = (0,9; 0,95)$ theo NeoBERT) được nạp lại *nguyên vẹn*, để bộ tiền điều kiện của bộ tối ưu khớp với chế độ mà phần thân đã được huấn luyện.

3.8 Tổng kết phương pháp

Tóm lại, phương pháp xây dựng ViNeoBERT gồm ba giai đoạn: (1) khởi tạo lớp véc-tơ đầu vào và lớp đầu ra bằng kỹ thuật SALT theo từng token từ ViDeBERTa sang không gian NeoBERT, với tập điểm neo được mở rộng bằng cặp dịch và hệ số điều chỉnh đầu ra khởi tạo theo tần suất; (2) căn chỉnh với phần thân đóng băng để hai ma trận mới ổn định cùng phần thân; và (3) tiếp tục tiền huấn luyện theo lịch WSD. So với SALT gốc, đề tài có ba khác biệt chính: *mở rộng tập điểm neo bằng cặp dịch* (Mục 3.4), *áp dụng phép chiếu cho cả lớp đầu ra – kèm khởi tạo hệ số điều chỉnh theo tần suất và giảm biên độ trọng số đầu ra để phân phối tần suất chi phối giai đoạn đầu* (Mục 3.5), và *bổ sung giai đoạn căn chỉnh với phần thân đóng băng* (Mục 3.6). Hiệu quả của các thành phần này được đánh giá trong Chương 4.

Chương 4

Thực nghiệm và Đánh giá

Chương này trình bày quá trình thực nghiệm nhằm kiểm chứng phương pháp đã đề xuất ở Chương 3. Trước hết, chúng tôi mô tả môi trường, dữ liệu và cấu hình huấn luyện (Mục 4.1–4.2), cùng các độ đo được sử dụng (Mục 4.3). Phần kết quả được tổ chức theo hai câu hỏi nghiên cứu: (1) *phương án khởi tạo nào là tốt nhất?* – trả lời bằng thí nghiệm so sánh các phương án khởi tạo trên cùng một lượng token huấn luyện (Mục 4.4); và (2) *mô hình cuối cùng đạt chất lượng ra sao so với các mô hình hiện có?* – trả lời bằng đường cong chất lượng theo ngân sách huấn luyện, bảng đánh giá đa tác vụ trên mười hai tác vụ hiểu tiếng Việt, và kiểm chứng khả năng xử lý ngữ cảnh dài sau pha mở rộng lên 4.096 token (Mục 4.5). Cuối chương là phần thảo luận chung (Mục 4.6).

4.1 Môi trường và dữ liệu thực nghiệm

Toàn bộ thực nghiệm được thực hiện trên GPU NVIDIA A100 (bộ nhớ 80GB, nền tảng Google Colab), sử dụng PyTorch và hệ sinh thái Hugging Face (`transformers`, `datasets`, `safetensors`).

4.1.1 Dữ liệu tiền huấn luyện

Giai đoạn căn chỉnh với phần thân đóng băng và tiếp tục tiền huấn luyện sử dụng phần tiếng Việt của CulturaX [44] – kho ngữ liệu web đa ngôn ngữ quy mô lớn đã được làm sạch và khử trùng lặp. Tổng ngân sách xấp xỉ 5 tỷ token, tương đương khoảng 20GB văn bản. Bảng 4.1 tóm tắt thống kê dữ liệu sử dụng trong đề tài.

Bảng 4.1: Thống kê dữ liệu thực nghiệm.

Dữ liệu	Vai trò	Quy mô sử dụng
CulturaX (tiếng Việt)	Căn chỉnh với phần thân đóng băng	~20 triệu token
CulturaX (tiếng Việt)	Tiếp tục tiền huấn luyện	~5 tỷ token (~20GB văn bản)
UIT-ViQuAD 2.0 (câu hỏi trả lời được)	Tinh chỉnh cho tác vụ ứng dụng	~18 nghìn câu hỏi huấn luyện
Bộ đánh giá đa tác vụ (Mục 4.1.2)	Tinh chỉnh cho tác vụ ứng dụng	12 tác vụ: phân loại, suy luận, gán nhãn, tương đồng ngữ nghĩa, Retrieval, Reranking

4.1.2 Dữ liệu đánh giá

Mô hình được đánh giá trên một bộ tác vụ hiểu ngôn ngữ tiếng Việt đa dạng, bao quát các nhóm năng lực khác nhau của một Encoder:

- **Hỏi đáp đọc hiểu:** UIT-ViQuAD 2.0 [45] – bộ dữ liệu hỏi đáp trích xuất xây dựng theo phương pháp luận của SQuAD; mô hình phải xác định đoạn văn bản trả lời trong ngữ cảnh cho trước. Chúng tôi chỉ dùng các câu hỏi *trả lời được* (answerable); tập validation công khai được chia đôi ngẫu nhiên thành tập phát triển và tập kiểm tra.
- **Suy luận ngôn ngữ tự nhiên:** XNLI (phần tiếng Việt) [46], ViANLI (bộ NLI đối kháng, khó hơn đáng kể) [47] và QNLI bản tiếng Việt (suy luận câu hỏi–câu, phỏng theo GLUE [48]).

- **Phân loại văn bản:** UIT-VSFC (phản hồi sinh viên) [49], UIT-VSMEC (cảm xúc đa lớp) [50], UIT-ViCTSD (bình luận độc hại) [51].
- **Tính chấp nhận được về ngữ pháp:** bộ dữ liệu kiểu CoLA [52] cho tiếng Việt (ViCoLA-syn), trong đó câu sai ngữ pháp được sinh tự động từ câu đúng; mô hình phải phân biệt hai loại câu.
- **Gán nhãn chuỗi:** gán nhãn từ loại trên treebank Universal Dependencies Vietnamese-VTB [53].
- **Tương đồng ngữ nghĩa:** bộ STS bản tiếng Việt [54], đo tương quan giữa điểm dự đoán và điểm do con người gán; và tác vụ phát hiện diễn đạt lại (paraphrase) – phân loại nhị phân cặp câu tương đương ngữ nghĩa xây dựng từ dữ liệu STS.
- **Biểu diễn câu (Retrieval và Reranking):** Retrieval và Reranking xây dựng từ các cặp (câu hỏi, đoạn văn) của UIT-ViQuAD. Hai tác vụ này được đánh giá theo giao thức biểu diễn câu của NeoBERT cho MTEB [5]: lấy biểu diễn câu bằng gộp trung bình (mean pooling), rồi tinh chỉnh nhẹ bằng học tương phản (contrastive learning); quy trình này được áp dụng *như nhau* cho mọi mô hình so sánh.

Bảng 4.2 tổng hợp toàn bộ bộ đánh giá cùng nhóm năng lực và độ đo tương ứng của từng tác vụ.

Bảng 4.3 so sánh ViNeoBERT với các mô hình khác về quy mô tham số, dữ liệu tiếng Việt và độ dài ngữ cảnh.

4.2 Cấu hình huấn luyện

Bảng 4.4 liệt kê siêu tham số của từng giai đoạn trong quy trình. Các giá trị được giữ cố định cho *mọi* phương án khởi tạo trong thí nghiệm so sánh, bảo đảm khác biệt duy nhất giữa các phương án là cách khởi tạo lớp véc-tơ đầu vào và lớp đầu ra.

Bảng 4.2: Các tác vụ đánh giá.

Tác vụ (bộ dữ liệu)	Nhóm năng lực	Độ đo
UIT-ViQuAD 2.0	Hỏi đáp đọc hiểu	EM, F1 ($\uparrow$)
XNLI (vi)	Suy luận ngôn ngữ	Accuracy ($\uparrow$)
ViANLI	Suy luận đối kháng	F1-macro ($\uparrow$)
QNLI (vi)	Suy luận câu hỏi-câu	Accuracy ($\uparrow$)
UIT-VSFC	Phân loại phản hồi	F1-weighted ($\uparrow$)
UIT-VSMEC	Phân loại cảm xúc	F1-macro ($\uparrow$)
UIT-ViCTSD	Phân loại độc hại	F1-macro ($\uparrow$)
ViCoLA-syn	Chấp nhận ngữ pháp	MCC ($\uparrow$)
UD-VTB	Gán nhãn từ loại	F1-macro ($\uparrow$)
STS (vi)	Tương đồng ngữ nghĩa	Spearman ($\uparrow$)
Paraphrase (STS)	Phát hiện diễn đạt lại	F1-macro ($\uparrow$)
Retrieval (ViQuAD)	Biểu diễn câu	nDCG@10 ($\uparrow$)
Reranking (ViQuAD)	Biểu diễn câu	MRR ($\uparrow$)

Bảng 4.3: Các mô hình Encoder được đánh giá.

Mô hình	Tham số	Kiến trúc	Dữ liệu tiếng Việt	Ngữ cảnh
PhoBERT-base	135M	RoBERTa	20GB $\times$ 40 epoch $\approx$ 140B token	512
PhoBERT-base-v2	135M	RoBERTa	+120GB OSCAR-2301	512
PhoBERT-large	370M	RoBERTa	Cùng base (20GB)	512
XLM-R-base	278M	RoBERTa	$\approx$ 137GB CC-100 vi ($\approx$ 25B token vi)	512
XLM-R-large	560M	RoBERTa	Cùng base	512
ViNeoBERT	$\approx$ 250M	NeoBERT	20GB, 1 lượt ($\approx$ 5B token)	4.096

Đối với bước khởi tạo SALT, các siêu tham số chính gồm: số điểm neo kích hoạt tối đa qua sparsemax là 64, ngưỡng số điểm neo tối thiểu cho ánh xạ cục bộ $k_0 = 8$, hệ số tỉ lệ trọng số đầu ra $\gamma = 0,1$, và hệ số điều chỉnh đầu ra được ước lượng từ một mẫu CulturaX khoảng 20 triệu token (Mục 3.5). Ở bước tinh chỉnh cho tác vụ ứng dụng, tác vụ chính (hỏi đáp đọc hiểu) dùng AdamW với tốc độ học 2×10^{-5} , chuỗi tối đa 384, batch 8, tối đa 5 epoch (dừng sớm); các tác vụ còn lại trong bộ đánh giá đa tác vụ được tinh chỉnh theo quy trình tương tự với siêu tham số phù hợp từng tác vụ, giữ đồng nhất giữa các mô hình so sánh.

Bảng 4.4: Siêu tham số huấn luyện.

Siêu tham số	Căn chỉnh với phần thân đóng băng	CPT (WSD)	Mở rộng ngữ cảnh
Tốc độ học đỉnh	10^{-4}	10^{-4}	10^{-5}
Khởi động (warmup)	10% số bước	2% (chặn [20, 500] bước)	50 bước
Độ dài chuỗi tối đa	1.024	1.024	4.096
Batch size hiệu dụng	$32 \times 4 = 128$	$32 \times 16 = 512$	$4 \times 32 = 128$
Tỷ lệ che MLM	20%	20%	20%
Bộ tối ưu	AdamW, $\beta = (0.9; 0.95)$, weight decay 0.01		
Lịch điều chỉnh tốc độ học	tuyến tính	WSD, làm nguội theo CT 3.10	hằng số sau khởi động
Số epoch	1 lượt ($\sim 20M$ token)	1 lượt (~ 5 tỷ token)	1 lượt ($\sim 200M$ token)

4.3 Độ đo đánh giá

Chúng tôi sử dụng các nhóm độ đo sau; trừ khi ghi chú khác, mọi kết quả tính chỉnh được tính trung bình trên 5 lần chạy với các seed ngẫu nhiên (random seed) khác nhau và báo cáo kèm độ lệch chuẩn:

- **Mất mát MLM và perplexity (PPL)(↓)**: mất mát cross-entropy của tác vụ mô hình hóa ngôn ngữ có mặt nạ trên tập giữ lại (held-out) của CulturaX, và perplexity tương ứng $PPL = \exp(\mathcal{L}_{MLM})$ – dùng để theo dõi quá trình huấn luyện.
- **Exact Match (EM)(↑) và F1(↑)** cho hỏi đáp đọc hiểu: EM là tỷ lệ câu trả lời trùng khớp hoàn toàn với đáp án chuẩn sau chuẩn hóa; F1 đo mức chồng lấp ở cấp token giữa câu trả lời dự đoán và đáp án chuẩn.
- **Accuracy, F1-macro, F1-weighted (↑)** cho các tác vụ phân loại và suy luận: F1-macro lấy trung bình đều giữa các lớp (phù hợp dữ liệu lệch lớp), F1-weighted lấy trung bình theo tỷ trọng lớp.
- **Hệ số tương quan Matthews (MCC)(↑)** cho tác vụ chấp nhận được về ngữ pháp: độ đo cân bằng cho phân loại nhị phân lệch lớp, nhận giá trị trong $[-1; 1]$ – quy ước chuẩn của bộ CoLA gốc.
- **Hệ số tương quan Spearman (↑)** cho tác vụ tương đồng ngữ nghĩa: đo tương quan hạng giữa điểm dự đoán và điểm con người gán.

- **nDCG@10** ($\uparrow$) và **MRR** ($\uparrow$) cho Retrieval và Reranking: nDCG@10 đánh giá chất lượng 10 kết quả đầu, ưu tiên kết quả đúng nằm ở thứ hạng cao (trọng số giảm dần theo vị trí); MRR là trung bình của nghịch đảo thứ hạng kết quả đúng. Hai tác vụ này chỉ chạy một lần theo giao thức biểu diễn câu chung nên không kèm độ lệch chuẩn.

Định nghĩa hình thức (công thức tường minh) của F1 ở cấp token, MCC, nDCG@ k và MRR được trình bày trong Phụ lục B.

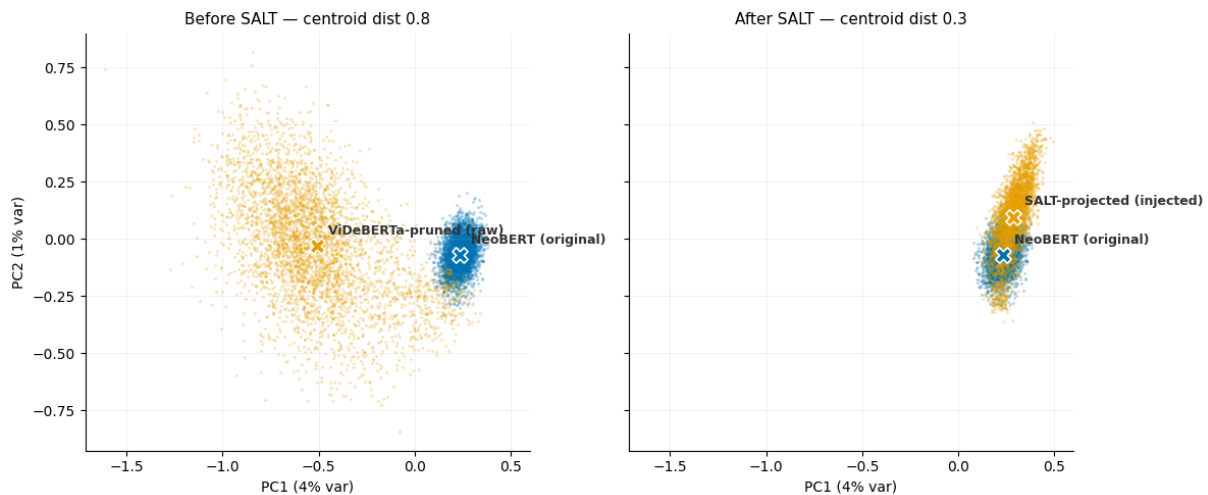
4.4 So sánh các phương án khởi tạo

Mục này kiểm chứng các lựa chọn thiết kế của phương pháp theo ba bước. Trước hết, chúng tôi kiểm tra trực quan xem kỹ thuật SALT có thực sự đưa từ vựng tiếng Việt vào đúng không gian của NeoBERT hay không (Mục 4.4.1). Sau đó, hai quyết định thiết kế được kiểm chứng bằng thí nghiệm có kiểm soát ở *cùng ngân sách dữ liệu và cùng cấu hình*, mỗi thí nghiệm chỉ thay đổi đúng thành phần đang xét: chọn hệ số tỉ lệ trọng số đầu ra γ (Mục 4.4.2), và chọn chiến lược khởi tạo hai ma trận từ vựng (Mục 4.4.3).

4.4.1 Kiểm chứng hình học của kỹ thuật SALT

Các phương án khởi tạo sẽ được so sánh đầy đủ bằng thực nghiệm huấn luyện ở những mục sau; nhưng trước hết, chúng tôi kiểm chứng trực tiếp câu hỏi căn bản nhất của phương pháp: *kỹ thuật SALT có thực sự đưa các véc-tơ tiếng Việt vào đúng vùng không gian mà phần thân NeoBERT mong đợi hay không?* Hình 4.1 trực quan hóa lớp véc-tơ đầu vào tại thời điểm khởi tạo, trước khi bất kỳ bước huấn luyện nào diễn ra.

Cách dựng hình. Mỗi véc-tơ token có 768 chiều nên không thể vẽ trực tiếp. Chúng tôi dùng *phân tích thành phần chính* (Principal Component



Hình 4.1: Lớp véc-tơ tiếng Việt trước và sau khi áp dụng SALT.

Analysis – PCA) để chiếu các véc-tơ xuống mặt phẳng hai chiều: PCA tìm những hướng mà dữ liệu biến thiên mạnh nhất và giữ lại hai hướng lớn nhất làm hai trục của hình. Điểm mấu chốt là hai trục này được tính *một lần trên hợp của cả hai tập véc-tơ* (ViDeBERTa và NeoBERT) chứ không tính riêng cho từng tập – nhờ dùng chung một hệ trục, vị trí tương đối giữa hai đám mây điểm mới so sánh được trực tiếp. Hai trục lần lượt là thành phần chính thứ nhất và thứ hai (ghi “PC1” và “PC2” trong hình), giải thích được 4% và 1% tổng phương sai của dữ liệu; tỷ lệ nhỏ này là bình thường ở không gian 768 chiều, nhưng hai hướng biến thiên mạnh nhất đó đã đủ để bộc lộ độ lệch mang tính hệ thống giữa hai không gian.

Một lưu ý khi đọc hình: *các token điểm neo và token đặc biệt đã được loại khỏi hình*. Chúng vốn được sao chép nguyên vẹn từ NeoBERT, nên nếu giữ lại sẽ tạo ra vùng chồng lấp giả tạo. Phần chồng lấp quan sát được vì thế hoàn toàn đến từ các ánh xạ tuyến tính theo từng token (Mục 3.5), chứ không phải từ những véc-tơ được sao chép.

Trước SALT (hình bên trái, “Before SALT”). Đám mây véc-tơ thô của từ vựng ViDeBERTa đã giảm kích thước (màu vàng) nằm lệch hẳn khỏi đám mây véc-tơ NeoBERT gốc (màu xanh): khoảng cách giữa hai tâm (centroid) trên mặt phẳng PCA là 0,8 đơn vị. Do hai mô hình được huấn

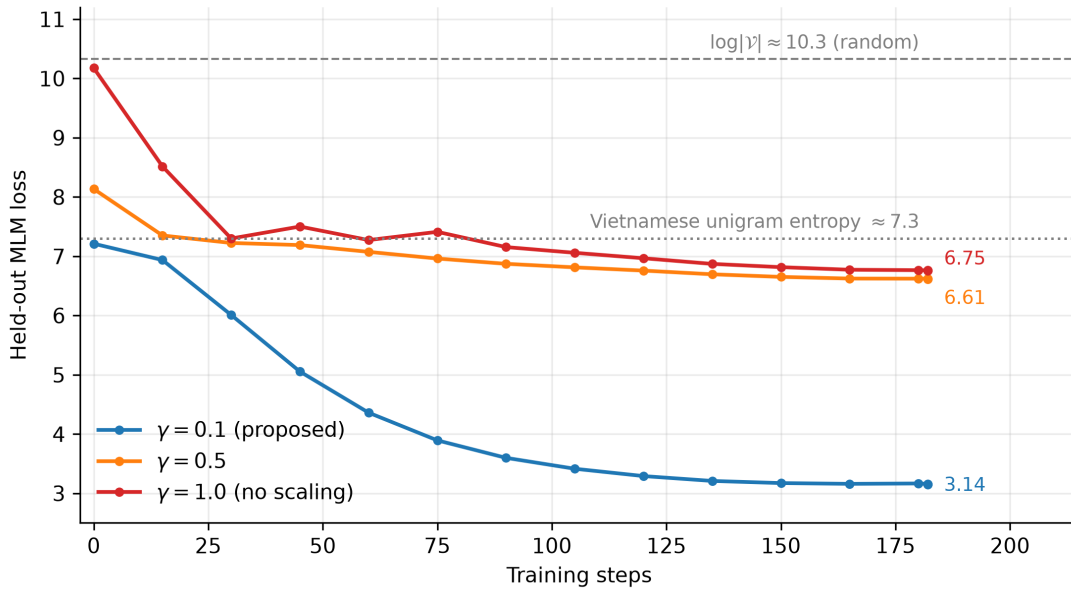
luyện hoàn toàn độc lập, không gian véc-tơ của chúng khác nhau cả về *vị trí* lẫn *độ lớn*: đám mây ViDeBERTa trải rộng hơn hẳn. Nếu đưa thẳng các véc-tơ thô này vào mô hình, phần thân sẽ nhận đầu vào lệch phân phối ở cả hai mặt, và các khối Pre-RMSNorm – vốn nhạy với độ lớn – sẽ khuếch đại sai lệch qua 28 lớp.

Sau SALT (hình bên phải, “After SALT”). Các véc-tơ sau phép chiếu – tức các véc-tơ thực sự được đưa vào mô hình – đã phủ chồng lên đám mây NeoBERT: khoảng cách hai tâm giảm còn 0,3 (khoảng một phần ba so với trước) và độ trải cũng thu về khớp với NeoBERT. Nói cách khác, phép chiếu theo từng token đưa từ vựng tiếng Việt về đúng *vị trí* trong không gian mà phần thân NeoBERT được huấn luyện để xử lý, còn bước hiệu chỉnh độ lớn (Mục 3.5) đưa chúng về đúng *độ lớn*, trong khi vẫn giữ nguyên quan hệ ngữ nghĩa tương đối giữa các token. Vẫn còn một phần đuôi màu vàng nhô nhẹ lên theo trục PC2 – chủ yếu là các token hiếm rơi vào nhánh trung bình có trọng số hoặc khởi tạo ngẫu nhiên (Mục 3.5); phần này sẽ được giai đoạn căn chỉnh với phần thân đóng băng ở bước kế tiếp đưa nốt về khớp. Đây cũng chính là lời giải thích hình học cho quan sát ở đầu mục sau: ngay sau khởi tạo SALT (trước căn chỉnh), mô hình đã đạt mất mát chỉ 7,21 thay vì mức đoán ngẫu nhiên trên 10.

4.4.2 Lựa chọn hệ số tỉ lệ trọng số đầu ra

Thí nghiệm đầu tiên trả lời câu hỏi: *cần giảm biên độ trọng số đầu ra sau kỹ thuật SALT tới mức nào?* Ba phương án giống hệt nhau về khởi tạo – SALT theo token cho cả hai ma trận, hệ số điều chỉnh đầu ra theo tần suất token – và chỉ khác đúng hệ số tỉ lệ $\gamma \in \{0,1; 0,5; 1,0\}$ được tiếp tục tiền huấn luyện ở cùng cấu hình. Hình 4.2 trình bày mất mát đánh giá, kèm hai mức tham chiếu: entropy từ đơn tiếng Việt ($\approx 7,3$) – mức mất mát khi mô hình dự đoán thuần theo tần suất – và mức đoán ngẫu nhiên $\log |\mathcal{V}_t| \approx 10,3$.

Kết quả khớp từng điểm với phân tích ở Mục 3.5.4. Phương án giữ



Hình 4.2: Mất mát MLM theo hệ số tỉ lệ γ .

nguyên biên độ ($\gamma = 1,0$) khởi đầu ở 10,18 – sát mức đoán ngẫu nhiên: các logit nhiều từ ma trận vừa chiếu đã lấn át hoàn toàn phân phối tần suất mã hóa trong hệ số điều chỉnh. Phương án giảm còn một nửa ($\gamma = 0,5$) khởi đầu ở 8,13, nằm giữa hai mức tham chiếu. Chuẩn gradient ở những bước đầu phản ánh đúng mức sai lệch: 68,4 với $\gamma = 1,0$ và 15,8 với $\gamma = 0,5$, so với chỉ 2,1 của $\gamma = 0,1$ – các cập nhật đầu tiên của hai phương án γ lớn có biên độ rất lớn nhưng mang ít thông tin hữu ích vì logit lúc này chủ yếu là nhiễu, và trên thực tế cả hai *mất mát* quanh mức 6,61–6,75 cho tới hết ngân sách. Ngược lại, phương án $\gamma = 0,1$ khởi đầu ngay tại vùng entropy unigram (7,21) – phân phối tần suất được bảo toàn làm điểm xuất phát – rồi hội tụ về 3,14, thấp hơn nhiều so với hai phương án còn lại trong suốt quá trình. Do đó, $\gamma = 0,1$ được chọn cho mọi thí nghiệm về sau.

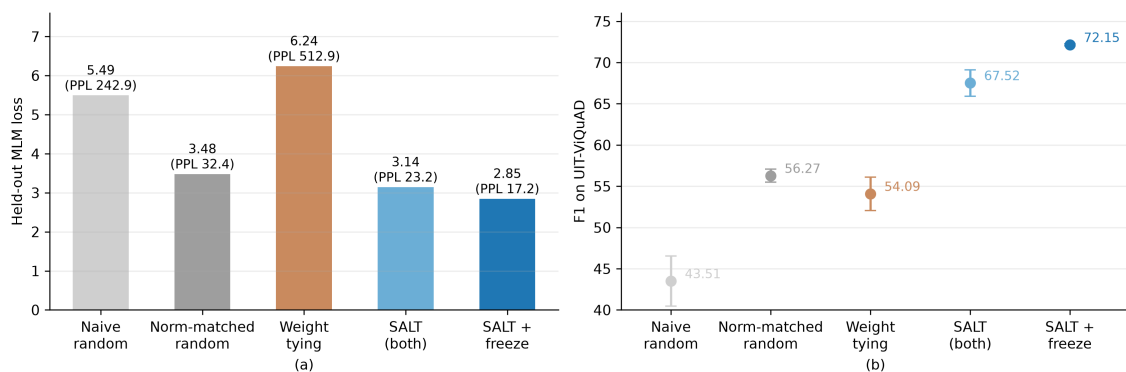
4.4.3 Lựa chọn chiến lược khởi tạo ma trận từ vựng

Thí nghiệm thứ hai so sánh năm phương án khởi tạo hai ma trận từ vựng, xếp thành hai nhóm. Hai *đường cơ sở ngẫu nhiên* đầu tiên không mang tín hiệu ngữ nghĩa nào từ mô hình nguồn véc-tơ, đóng vai trò mốc

tham chiếu; ba *chiến lược dựa trên ngữ nghĩa* còn lại tận dụng không gian của NeoBERT ở mức tăng dần:

- **Ngẫu nhiên ngây thơ (naive random)**: cả hai ma trận lấy theo công thức khởi tạo từ đầu của NeoBERT – $\mathcal{N}(0, 0,02)$ – và hệ số điều chỉnh theo tần suất đặt bằng không; cách khởi tạo này *không* mang bất kỳ thông tin nào từ không gian véc-tơ nguồn, nên là mốc sà thấp nhất;
- **Ngẫu nhiên khớp mô-men (moment-matched random)**: mỗi dòng từ vựng mới là một *hướng ngẫu nhiên* nhưng được co giãn về đúng chuẩn (norm) trung bình của lớp véc-tơ NeoBERT; lớp đầu ra suy ra từ lớp véc-tơ qua cùng ánh xạ tuyến tính toàn cục và cùng hệ số điều chỉnh theo tần suất mà SALT dùng. Đường cơ sở này khớp SALT ở mọi thành phần *trừ hướng ngữ nghĩa*, nên là mốc ngẫu nhiên công bằng và mạnh nhất;
- **Dùng chung trọng số (weight tying)**: lớp đầu ra sao chép nguyên ma trận véc-tơ tại khởi tạo – thói quen kế thừa từ các kiến trúc dùng chung trọng số, bỏ qua việc NeoBERT không dùng chung trọng số cho lớp đầu ra;
- **SALT cho hai ma trận**: chiếu theo token *riêng* cho lớp véc-tơ đầu vào và lớp đầu ra với $\gamma = 0,1$ (Mục 3.5.3);
- **SALT cho hai ma trận kết hợp căn chỉnh với phần thân đóng băng**: như trên, thêm giai đoạn huấn luyện phụ trợ ~ 20 triệu token chỉ cho hai ma trận trên phần thân đóng băng trước khi CPT (Mục 3.6).

Cả năm được tiếp tục tiền huấn luyện tới cùng mốc ~ 100 triệu token, đo mất mát trên tập đánh giá, rồi tinh chỉnh trên UIT-ViQuAD với cấu hình đồng nhất qua ba seed ngẫu nhiên. Hình 4.3 tổng hợp cả hai mặt so sánh.



Hình 4.3: So sánh năm phương án khởi tạo hai ma trận từ vừng (xám: hai đường cơ sở ngẫu nhiên; nâu: dùng chung trọng số; xanh: SALT). (a) mất mát MLM trên tập đánh giá; (b) F1 trên UIT-ViQuAD, thanh lỗi là độ lệch chuẩn qua ba seed ngẫu nhiên.

Hình 4.3b cho thấy các phương án xếp thành một bậc thang tăng gần như đều trên UIT-ViQuAD. Khởi tạo ngẫu nhiên ngây thơ – không mang bất kỳ tín hiệu nào từ mô hình nguồn – chỉ đạt $43,51 \pm 3,03$ điểm F1, đúng như kỳ vọng cho một mốc sào. Đường cơ sở ngẫu nhiên công bằng (khớp mô-men) nâng lên $56,27 \pm 0,79$ nhờ đã có sẵn đúng độ lớn, ánh xạ đầu ra và hệ số điều chỉnh theo tần suất – chỉ thiếu duy nhất *hướng* của các véc-tơ từ vừng. SALT hai ma trận đạt $67,52 \pm 1,61$, cao hơn đường cơ sở công bằng này 11,2 điểm. Vì hai phương án chỉ khác nhau đúng ở phần hướng véc-tơ đó, toàn bộ khoảng cách 11,2 điểm quy về giá trị của hướng ngữ nghĩa mà SALT sao chép từ không gian nguồn – bằng chứng trực tiếp rằng chính phần lõi của phương pháp, chứ không phải các yếu tố phụ trợ như độ lớn hay hệ số điều chỉnh theo tần suất, mới tạo ra khác biệt.

Đáng chú ý, dùng chung trọng số ($54,09 \pm 2,04$) còn *thấp hơn* cả đường cơ sở ngẫu nhiên công bằng ($56,27$), và kém SALT hai ma trận tới 13,4 điểm. Việc sao chép nguyên lớp véc-tơ sang lớp đầu ra của NeoBERT – vốn không dùng chung trọng số với lớp véc-tơ đầu vào – gây hại còn nhiều hơn một khởi tạo ngẫu nhiên đúng độ lớn. Nguyên nhân đã được phân tích ở Mục 2.3.1: lớp véc-tơ đầu vào và lớp đầu ra đảm nhận hai vai trò hình học khác nhau, nên ép chúng dùng chung một ma trận ngay từ khởi

tạo khiến cả hai nhiệm vụ cùng bị tổn hại. Phương án này khởi đầu ở mức mất mát 21,67 – cao nhất trong mọi thí nghiệm của đề tài – và tới mốc 100 triệu token vẫn dừng ở 6,24 (perplexity ≈ 513 , Hình 4.3a), cao hơn cả khởi tạo ngẫu nhiên ngẫu thơ (5,49).

Một điểm đáng chú ý là hai thước đo – mất mát MLM và điểm F1 trên tác vụ ứng dụng – không xếp hạng các phương án theo cùng một thứ tự. Rõ nhất là ở đường cơ sở ngẫu nhiên khớp mô-men: nó đạt mất mát MLM khá thấp (3,48; perplexity 32,4), gần chạm mức của SALT (3,14), nhưng trên F1 lại kém SALT tới hơn 11 điểm. Sự chênh lệch này cho thấy nếu chỉ nhìn mất mát MLM thì không đánh giá đúng chất lượng khởi tạo. Nguyên do là phần lớn mất mát MLM có thể được kéo xuống chỉ nhờ đoán trúng những token phổ biến – mà một khởi tạo đã khớp đúng độ lớn và tần suất token (như đường cơ sở công bằng) thì vốn đã làm được điều đó, dù các véc-tơ từ vựng vẫn chưa nằm đúng vùng ngữ nghĩa. Chỉ đến một tác vụ khó như hỏi đáp trích xuất – nơi mô hình buộc phải dùng đến ngữ nghĩa thực sự của từng token – khoảng cách giữa các phương án mới bộc lộ rõ. Đây cũng là lý do chúng tôi chọn UIT-ViQuAD làm thước đo quyết định thay vì chỉ dựa vào mất mát tiền huấn luyện.

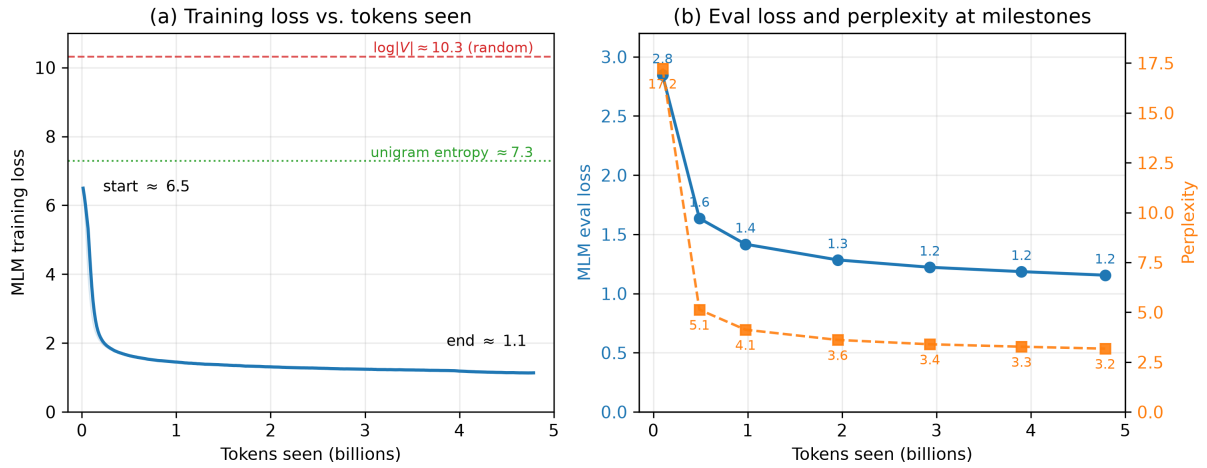
Bổ sung giai đoạn căn chỉnh với phần thân đóng băng cải thiện rõ rệt ở cả hai mặt. Về mất mát, giá trị khởi điểm giảm còn 2,85 (perplexity 17,2) – tương đương giảm khoảng 26% perplexity so với khi không căn chỉnh – nhờ hai ma trận đã kịp “khớp” với nhau và với phần thân trước khi bước vào CPT. Về tác vụ ứng dụng, phương án này đạt $72,15 \pm 0,12$ điểm F1 tại cùng mốc ngân sách, cao hơn 4,6 điểm so với khi không căn chỉnh (và gần 16 điểm so với đường cơ sở ngẫu nhiên công bằng); đồng thời độ lệch chuẩn giữa các lần chạy giảm hơn một bậc (0,12 so với 1,61), tức mô hình vừa tốt hơn vừa ổn định hơn khi tinh chỉnh. Chi phí đổi lại chỉ là ~ 20 triệu token để huấn luyện hai ma trận (Bảng 4.4). Do đó, chúng tôi chọn *SALT kết hợp căn chỉnh với phần thân đóng băng* làm cấu hình chính để huấn luyện với toàn bộ ngân sách.

4.5 Kết quả của mô hình cuối

Mô hình chính được tiếp tục tiền huấn luyện tới khoảng 5 tỷ token CulturaX theo lịch WSD. Chúng tôi phân tích kết quả theo ba góc nhìn: chất lượng thay đổi ra sao theo lượng token huấn luyện (Mục 4.5.1), kết quả trên mười hai tác vụ tại mốc cuối (Mục 4.5.2), và khả năng xử lý ngữ cảnh dài sau pha mở rộng lên 4.096 token (Mục 4.5.3). Chúng tôi so sánh với hai họ mô hình phổ biến nhất cho tiếng Việt – PhoBERT [6], đơn ngữ, huấn luyện trên dữ liệu đã tách từ, và XLM-R [55], đa ngôn ngữ – ở cả hai cỡ `base` và `large`.

4.5.1 Chất lượng theo lượng dữ liệu huấn luyện

Trước khi xét chất lượng trên các tác vụ ứng dụng, Hình 4.4 cho thấy chính quá trình tiếp tục tiền huấn luyện diễn ra thế nào: hình (a) vẽ mất mát huấn luyện (training loss) theo số token đã thấy, kèm hai mức tham chiếu – $\log |V|$ (mức mất mát của một mô hình đoán hoàn toàn ngẫu nhiên, gán xác suất như nhau cho mọi token trong từ vựng) và entropy unigram tiếng Việt; hình (b) vẽ mất mát và perplexity trên tập đánh giá (eval loss) tại các mốc. Nhờ khởi tạo SALT, mất mát MLM *khởi đầu* chỉ khoảng 6,5 – đã nằm sâu dưới mức đoán ngẫu nhiên $\log |V| \approx 10,3$ và gần với entropy unigram tiếng Việt ($\approx 7,3$) – thay vì bắt đầu từ một trạng thái kém ổn định như khi khởi tạo ngẫu nhiên. Mất mát giảm mạnh nhất ở giai đoạn đầu: chỉ trong khoảng 0,2 tỷ token đầu, nó đã rơi xuống dưới 2,0. Sau đó tốc độ học được giữ ở đỉnh trong phần lớn ngân sách (pha ổn định của lịch WSD, khoảng 4 tỷ token đầu) và mất mát tiếp tục hạ chậm; pha làm nguội trên khoảng 1 tỷ token cuối – xấp xỉ 20% ngân sách (Mục 3.7) – kéo mất mát về $\approx 1,1$. Mất mát trên tập đánh giá giảm cùng nhịp với mất mát huấn luyện: perplexity giảm từ 17,2 ở khoảng 0,1 tỷ token xuống 3,3 ở khoảng 3,9 tỷ token và 3,2 ở điểm kiểm tra cuối. Nói ngắn gọn, cả đường cong mất mát khi tiền huấn luyện lẫn đường cong điểm F1 trên các tác

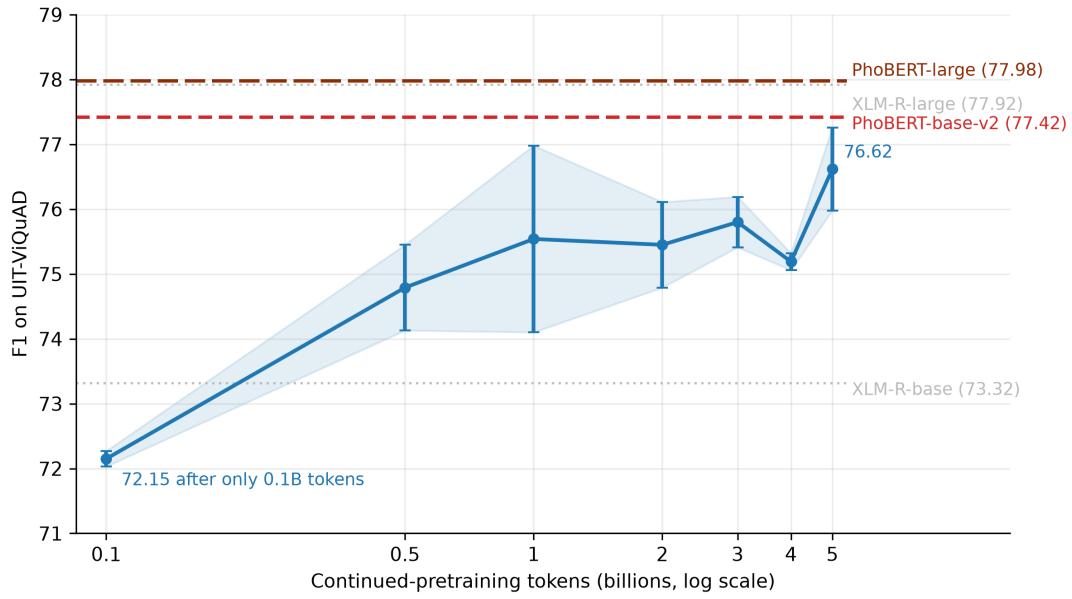


Hình 4.4: Đường cong mất mát giai đoạn CPT (WSD).

vụ ứng dụng (dưới đây) cùng cho thấy một điều: phần thân kế thừa từ NeoBERT hội tụ nhanh trên tiếng Việt thay vì phải học lại từ đầu.

Tại mỗi mốc ngân sách, chúng tôi rẽ một nhánh làm nguội ngẩn để lấy điểm kiểm tra rồi tinh chỉnh trên UIT-ViQuAD; Hình 4.5 thể hiện điểm F1 theo lượng token đã tiếp tục tiền huấn luyện (thang log). Mô hình so sánh chính là họ PhoBERT (hai đường đứt nét đậm) – cùng là mô hình đơn ngữ chuyên cho tiếng Việt nên là mốc tham chiếu trực tiếp của đề tài; hai đường chấm xám của XLM-R chỉ mang tính tham khảo, vì đây là mô hình đa ngôn ngữ với tiếng Việt chỉ là một trong một trăm ngôn ngữ.

Điểm nổi bật nhất của đường cong là *tốc độ*: ngay tại mốc đầu tiên – 0,1 tỷ token, tức chỉ 2% ngân sách – ViNeoBERT đã đạt 72,15 điểm F1, cao hơn mọi chiến lược khởi tạo đối chứng ở Mục 4.4.3 và chỉ còn cách kết quả cuối 4,5 điểm; sang mốc 0,5 tỷ token, mô hình đã vượt kết quả *cùng* của XLM-R-base (74,79 so với 73,32). Đây chính là giá trị thực tiễn của khởi tạo SALT: phần thân kế thừa không phải “học lại tiếng Việt từ đầu”, nên phần lớn chất lượng đến ngay từ những trăm triệu token đầu tiên. Sau đó đường cong gần như đi ngang quanh vùng 1–4 tỷ token trước khi tăng lên 76,62 ở mốc 5 tỷ – mốc được áp dụng pha làm nguội trên dữ liệu mới (Mục 3.7) – thu hẹp khoảng cách với PhoBERT-base-v2 còn 0,8



Hình 4.5: Điểm F1 trên UIT-ViQuAD theo số token CPT.

điểm và với PhoBERT-large còn 1,4 điểm, với xu hướng vẫn đang đi lên.

Khoảng cách còn lại đó cần được đặt cạnh lượng dữ liệu mà mỗi mô hình đã tiêu thụ, tóm tắt trong Bảng 4.5. PhoBERT huấn luyện 40 epoch trên kho 20GB ($\sim 3,5$ tỷ token mỗi lượt): bản base chạy 540 nghìn bước với batch 1.024 chuỗi 256 token, bản large 1,08 triệu bước với batch 512 – cả hai đều đã thấy xấp xỉ *140 tỷ token* [6]; bản base-v2 còn được huấn luyện thêm trên 120GB văn bản OSCAR-2301 (kho ~ 25 tỷ token mỗi lượt, số bước không được công bố). XLM-R huấn luyện 1,5 triệu bước với batch 8.192 chuỗi 512 token – tổng cỡ *6,3 nghìn tỷ token* đa ngôn ngữ; riêng phần tiếng Việt trong CC-100 đã là 24,76 tỷ token (137,3 GiB), và với tỷ lệ lấy mẫu $\alpha = 0,3$ lượng tiếng Việt mô hình này đã thấy ước tính ở mức hàng trăm tỷ token [55]. Trong khi đó, ViNeoBERT đi qua 5 tỷ token đúng *một lượt* – tức chỉ khoảng **3,5%** lượng token mà PhoBERT-large đã huấn luyện (~ 140 tỷ) – nhưng đạt 99,0% điểm F1 của PhoBERT-base-v2 và 98,3% của PhoBERT-large trên tác vụ này.

Bảng 4.5: Lượng dữ liệu tiếng Việt đã xử lý.

Mô hình	Kho ngữ liệu	Token đã thấy
PhoBERT (base/large)	20GB (~3,5 tỷ token)	~140 tỷ (40 epoch)
PhoBERT-base-v2	140GB (~25 tỷ token)	không công bố ($\geq$ bản gốc)
XLM-R (base/large)	CC-100 2,5TB; phần vi 24,76 tỷ token	~6.300 tỷ (đa ngôn ngữ); vi ước hàng trăm tỷ
ViNeoBERT	20GB (~5 tỷ token)	5 tỷ (một lượt)

4.5.2 Đánh giá đa tác vụ

Để có cái nhìn toàn diện hơn thay vì chỉ dựa vào một tác vụ đơn lẻ, mô hình cuối được đánh giá trên mười hai tác vụ thuộc năm nhóm năng lực (Mục 4.1.2; riêng UIT-ViQuAD đọc hiểu đã được phân tích tách riêng ở Mục 4.5.1 nên không lặp lại trong bảng này), so với bốn mô hình so sánh: hai bản **base** cùng hạng tham số và hai bản **large** lớn hơn đáng kể (PhoBERT-large khoảng 370 triệu, XLM-R-large khoảng 560 triệu tham số, so với ≈ 250 triệu của ViNeoBERT). Kết quả trình bày trong Bảng 4.6: mỗi ô là giá trị trung bình qua 5 lần chạy với seed ngẫu nhiên khác nhau, riêng hai tác vụ Retrieval và Reranking chạy một lần (Mục 4.3).

Bảng 4.6: Kết quả đánh giá đa tác vụ tại điểm kiểm tra 5 tỷ token (giá trị tốt nhất in đậm).

Tác vụ	Độ đo	PhoBERT-base-v2	PhoBERT-large	XLM-R-base	XLM-R-large	ViNeoBERT
UIT-VSFC	F1-weighted ($\uparrow$)	0,9345	0,9356	0,9281	0,9312	0,9284
UIT-VSMCEC	F1-macro ($\uparrow$)	0,6415	0,6132	0,6145	0,3125	0,6163
UIT-ViCTSD	F1-macro ($\uparrow$)	0,7455	0,7526	0,7110	0,5759	0,7315
ViCoLA-syn	MCC ($\uparrow$)	0,8103	0,7915	0,7827	0,4845	0,7789
XNLI (vi)	Accuracy ($\uparrow$)	0,7720	0,7769	0,7541	0,8000	0,7714
ViANLI	F1-macro ($\uparrow$)	0,4529	0,3882	0,3625	0,1666	0,4273
QNLI (vi)	Accuracy ($\uparrow$)	0,8872	0,8861	0,8592	0,8940	0,8791
UD-VTB (POS)	F1-macro ($\uparrow$)	0,7799	0,7879	0,7827	0,8062	0,8089
STS (vi)	Spearman ($\uparrow$)	0,8557	0,8509	0,8226	0,8644	0,8392
Paraphrase (STS)	F1-macro ($\uparrow$)	0,9351	0,9400	0,9016	0,9437	0,9276
Retrieval (ViQuAD)	nDCG@10 ($\uparrow$)	0,4719	0,4659	0,1386	0,2565	0,4061
Reranking (ViQuAD)	MRR ($\uparrow$)	0,8922	0,8794	0,5282	0,6741	0,8448

Từ Bảng 4.6, chúng tôi rút ra bốn nhận xét:

- **So sánh với PhoBERT:** PhoBERT là mô hình so sánh chính – mô hình chuyên biệt tiếng Việt được tiền huấn luyện quy mô lớn và là chuẩn hiện tại của lĩnh vực. So với PhoBERT-base-v2 cùng hạng tham số, ViNeoBERT cao hơn ở gần nhãn từ loại (0,8089 so với 0,7799), Retrieval (0,4061 so với 0,4719 – thấp hơn nhưng cao

hơn nhiều XLM-R-base 0,1386), và Reranking (0,8448 so với 0,8922), trong khi xấp xỉ ở VSFC (+0,0061 cho PhoBERT), XNLI (+0,0006), QNLI (+0,0081), và kém rõ hơn ở VSMEC, ViCoLA-syn, ViANLI. PhoBERT-large – lớn gấp 1,5 lần – dẫn đầu ở VSFC, ViCTSD và các tác vụ paraphrase/STS, song ViNeoBERT đã vượt qua ở gán nhãn từ loại.

- **So sánh với XLM-R:** XLM-R là mô hình so sánh thứ hai – mô hình đa ngôn ngữ không chuyên biệt tiếng Việt. So với XLM-R-base cùng hạng tham số, ViNeoBERT cao hơn ở 11/12 tác vụ bảng này, và 12/13 nếu tính đọc hiểu ở Mục 4.5.1. XLM-R-large tuy dẫn một số tác vụ suy luận và tương đồng ngữ nghĩa nhưng thể hiện *bất ổn rõ rệt khi tinh chỉnh* trên các bộ dữ liệu nhỏ (phương sai rất cao ở VSMEC, ViCoLA-syn, ViCTSD) và giảm sâu ở ViANLI – hiện tượng đã biết của các mô hình lớn trên dữ liệu ít. ViNeoBERT ổn định trên toàn bộ 12 tác vụ.
- **Hiệu quả dữ liệu – điểm mấu chốt:** lượng tiếng Việt mà ViNeoBERT tiếp xúc chỉ khoảng 5 tỷ token ($\sim 20\text{GB}$), đi qua đúng một lượt. PhoBERT được huấn luyện 40 epoch trên 20GB (Wikipedia + tin tức), tức lượng token đã thấy lớn hơn ViNeoBERT khoảng 28 lần [6]; bản base-v2 huấn luyện bổ sung trên 120GB văn bản OSCAR-2301. XLM-R sử dụng 2,5TB dữ liệu CC-100, riêng phần tiếng Việt $\approx 137\text{GB}$ [55] – gấp nhiều lần toàn bộ ngân sách của đề tài. Kết quả cạnh tranh được với các bản **large** – dù chỉ “đọc” khoảng 3,5% lượng token mà PhoBERT-large đã huấn luyện – xác nhận trực tiếp luận điểm của đề tài: khởi tạo dựa trên ngữ nghĩa cho phép kế thừa năng lực phần thân đã huấn luyện thay vì học lại từ đầu.
- **Phân bố điểm mạnh:** các tác vụ ViNeoBERT mạnh nhất (đọc hiểu trích xuất, gán nhãn từ loại, Retrieval/Reranking) gắn với chất lượng *biểu diễn theo ngữ cảnh ở cấp token* – đúng năng lực được kế

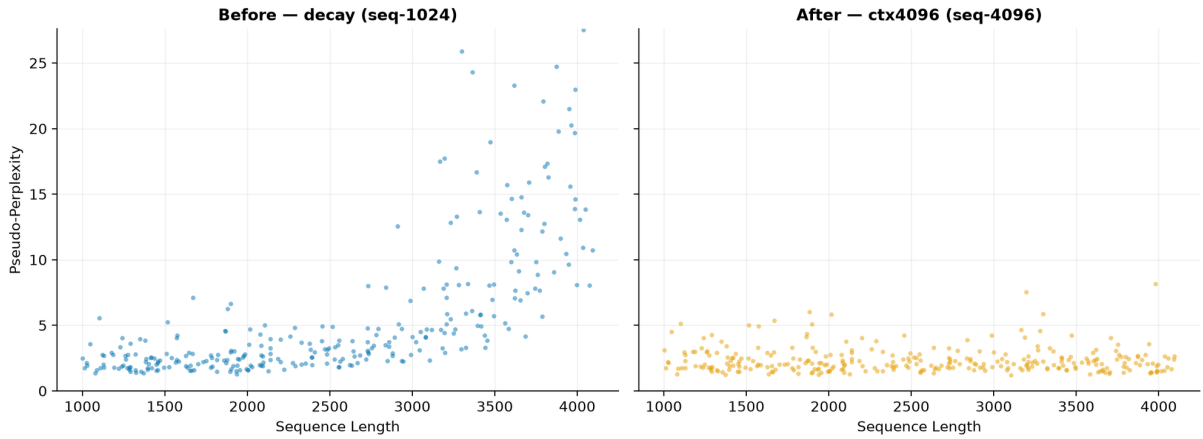
thừa từ phần thân NeoBERT sâu 28 lớp. PhoBERT giữ ưu thế ở nhóm phân loại câu ngắn và tương đồng ngữ nghĩa – vốn hưởng lợi từ tokenizer tiếng Việt và dữ liệu trong miền phong phú hơn.

4.5.3 Mở rộng ngữ cảnh lên 4.096 token

NeoBERT được tiền huấn luyện theo hai pha: pha chính ở độ dài 1.024 token, sau đó một pha ngắn ở độ dài 4.096 token để kích hoạt toàn bộ trần ngữ cảnh [5]. ViNeoBERT kế thừa kiến trúc hỗ trợ 4.096 token (tần số RoPE được tính sẵn tới độ dài này), nhưng toàn bộ quá trình CPT dùng chuỗi 1.024 token – vì vậy chúng tôi tái hiện pha thứ hai tương tự: từ điểm kiểm tra đã làm nguội ở mốc 5 tỷ token, mô hình được huấn luyện tiếp khoảng 200 triệu token ($\sim 4\%$ ngân sách CPT) ở độ dài 4.096, trên phần dữ liệu CulturaX *chưa từng thấy* (bốn đoạn 1.024 liên tiếp được ghép thành một chuỗi 4.096), với tốc độ học hằng số 10^{-5} – một phần mười đỉnh CPT (cột “Mở rộng ngữ cảnh” trong Bảng 4.4).

Khả năng ngữ cảnh dài được kiểm chứng theo đúng giao thức của NeoBERT: *pseudo-perplexity* (PPPL) [56] theo độ dài chuỗi. Với mô hình MLM, PPPL của một chuỗi được ước lượng bằng cách che lần lượt từng vị trí và lấy tích xác suất khôi phục; PPPL càng thấp và càng *ổn định theo độ dài* thì mô hình khai thác ngữ cảnh xa càng tốt. Trong Hình 4.6, mỗi điểm là một văn bản dài (1.000–4.096 token) lấy từ Wikipedia tiếng Việt; PPPL mỗi điểm được xấp xỉ bằng lấy mẫu 32 vị trí che (subsampling masking), và *cùng* tập cặp (văn bản, độ dài) được dùng cho cả hai điểm kiểm tra nên hai hình so sánh được trực tiếp.

Kết quả tái hiện đúng hiện tượng mà NeoBERT báo cáo. Trước pha mở rộng (hình trái, “Before – decay (seq-1024)”: nhờ tính chất tương đối của RoPE, mô hình ngoại suy được một phần ra ngoài độ dài huấn luyện – PPPL giữ ổn định quanh 2–3 tới khoảng 2.500–3.000 token – nhưng vượt ngưỡng đó chất lượng giảm nhanh, nhiều điểm vượt 10 và có điểm lên tới trên 25: ngữ cảnh xa không những không giúp mà còn gây nhiễu dự đoán.



Hình 4.6: Pseudo-perplexity theo độ dài chuỗi, trước và sau pha mở rộng 4.096 token.

Sau pha mở rộng (hình phải, “After – ctx4096 (seq-4096)”): đám mây điểm phẳng quanh 2–3 trên *toàn dải* tới 4.096 token, không còn xu hướng tăng theo độ dài. Nói cách khác, chỉ với ~ 200 triệu token huấn luyện bổ sung, trần ngữ cảnh 4.096 token kế thừa từ kiến trúc đã trở thành năng lực sử dụng được trong thực tế.

4.6 Thảo luận

Chúng tôi đối chiếu kết quả thực nghiệm với các mục tiêu đặt ra ở Chương 1. Thí nghiệm so sánh khởi tạo (Mục 4.4) chứng minh từng lựa chọn thiết kế của phương pháp đều mang lại cải thiện đo được: kỹ thuật SALT đưa từ vựng tiếng Việt về đúng vị trí và độ lớn của không gian NeoBERT (khoảng cách tâm giảm từ 0,8 còn 0,3); hệ số tỉ lệ $\gamma = 0,1$ là cần thiết – nếu để nguyên biên độ hoặc chỉ giảm một nửa, huấn luyện bị mắc kẹt; khởi tạo SALT riêng cho hai ma trận đạt điểm F1 cao hơn hẳn cách dùng chung trọng số mặc định (67,52 so với 54,09); và căn chỉnh với phần thân đóng băng giảm khoảng 26% perplexity, nâng F1 lên 72,15 với chi phí không đáng kể. Đánh giá theo ngân sách (Mục 4.5) cho thấy phương pháp đạt hiệu quả dữ liệu cao: chỉ với khoảng 0,5 tỷ token đã vượt

XLM-R-base; đến mốc 5 tỷ token, mô hình cao hơn XLM-R-base ở 11/12 tác vụ, dẫn đầu cả bảng ở gán nhãn từ loại, và ổn định hơn rõ rệt so với các bản **large** khi tinh chỉnh – dù chỉ tiếp xúc khoảng 3,5% lượng token mà PhoBERT-large đã huấn luyện.

Thực nghiệm cũng bộc lộ một số giới hạn: ngân sách 5 tỷ token còn nhỏ so với các mô hình so sánh, một vài tác vụ phân loại đã bão hòa, và hai tác vụ biểu diễn câu – vốn kèm một bước tinh chỉnh tương phản có yếu tố ngẫu nhiên – mới chỉ chạy một lần nên chưa ước lượng được phương sai giữa các lần chạy. Các giới hạn này được thảo luận đầy đủ cùng hướng khắc phục ở Chương 5.

Chương 5

Kết luận và Hướng phát triển

5.1 Kết luận

Khóa luận đã nghiên cứu và xây dựng **ViNeoBERT** – một mô hình Encoder hiện đại cho tiếng Việt – bằng cách thích ứng NeoBERT thông qua kỹ thuật SALT được cải tiến, thay vì huấn luyện lại từ đầu. Đối chiếu với các mục tiêu đặt ra ở Chương 1, đề tài đã đạt được các kết quả sau:

- **Về nghiên cứu kiến trúc:** Chúng tôi đã phân tích các cải tiến kiến trúc của NeoBERT (RoPE, SwiGLU, Pre-RMSNorm, tỷ lệ chiều sâu trên chiều rộng tối ưu) và xác định đúng hai điểm mấu chốt ảnh hưởng tới việc chuyển giao: chuẩn hóa Pre-RMSNorm nhạy với độ lớn của véc-tơ – dẫn tới bước hiệu chỉnh độ lớn – và việc lớp đầu ra không dùng chung trọng số nên phải được khởi tạo riêng, độc lập với lớp véc-tơ đầu vào (Chương 2, 3).
- **Về phương pháp chuyển giao:** Chúng tôi đã triển khai thuật toán SALT và cải tiến nó cho cặp ngôn ngữ Anh–Việt với ba đóng góp: (1) mở rộng tập điểm neo bằng cặp dịch Anh–Việt, kèm cơ chế lọc từ đồng tự khác nghĩa đặc thù cho ngôn ngữ hệ chữ Latin; (2) áp dụng phép chiếu theo từng token cho cả lớp đầu ra, kết hợp khởi tạo hệ số điều chỉnh theo tần suất token và hệ số tỉ lệ trọng số $\gamma = 0,1$;

(3) bổ sung giai đoạn căn chỉnh với phần thân đóng băng trước khi tiếp tục tiền huấn luyện (Chương 3).

- **Về kiểm chứng thực nghiệm:** Thí nghiệm so sánh có kiểm soát các phương án khởi tạo trên cùng một lượng token (Mục 4.4) xác nhận từng lựa chọn thiết kế: hệ số tỉ lệ $\gamma = 0,1$ là cần thiết – nếu để nguyên biên độ hoặc chỉ giảm một nửa, huấn luyện bị mắc kẹt ở mức mất mát cao; khởi tạo SALT riêng cho hai ma trận đạt điểm F1 cao hơn hẳn cách dùng chung trọng số mặc định (67,52 so với 54,09 trên UIT-ViQuAD sau 100 triệu token); và căn chỉnh với phần thân đóng băng giảm khoảng 26% perplexity, nâng F1 lên 72,15 với chi phí phụ trợ không đáng kể.
- **Về hiệu quả của mô hình:** Sau khoảng 5 tỷ token CulturaX (~ 20 GB văn bản), ViNeoBERT đạt 76,62 điểm F1 trên đọc hiểu UIT-ViQuAD (hơn XLM-R-base 3,3 điểm), cao hơn XLM-R-base ở 11/12 tác vụ của bảng đánh giá đa tác vụ, đạt điểm cao nhất ở gán nhãn từ loại và cạnh tranh được với các bản **large** có gấp 1,5–2,3 lần tham số (Mục 4.5). Pha mở rộng ngữ cảnh ~ 200 triệu token ở độ dài 4.096 giữ pseudo-perplexity ổn định trên toàn dải ngữ cảnh, biến trần 4.096 token kế thừa từ kiến trúc thành năng lực sử dụng được. Đáng nói nhất, lượng tiếng Việt mô hình tiếp xúc chỉ khoảng 5 tỷ token – chỉ bằng khoảng 3,5% lượng token mà PhoBERT-large đã huấn luyện (~ 140 tỷ, 40 epoch trên 20GB), trong khi XLM-R được tiền huấn luyện trên kho dữ liệu đa ngôn ngữ rất lớn (2,5TB CC-100). Kết quả này xác nhận giả thuyết trung tâm của đề tài: *tái sử dụng không gian biểu diễn sẵn có là con đường tiết kiệm để đưa các kiến trúc Encoder thế hệ mới đến với tiếng Việt.*

Bên cạnh mô hình, đề tài đóng góp một quy trình có thể tái lập – khởi tạo SALT cải tiến, căn chỉnh với phần thân đóng băng, tiếp tục tiền huấn luyện theo lịch WSD – cùng bộ mã nguồn công khai đi kèm.

5.2 Hạn chế của đề tài

Trong khuôn khổ thời gian và tài nguyên của một khóa luận, đề tài còn các hạn chế sau:

- **Ngân sách huấn luyện:** khoảng 5 tỷ token ($\sim 20\text{GB}$ văn bản) vẫn nhỏ so với dữ liệu tiền huấn luyện của các mô hình so sánh; ViNeoBERT chưa vượt PhoBERT-base-v2 ở nhóm tác vụ phân loại câu ngắn, dù xu hướng đi lên của đường cong F1 theo ngân sách cho thấy khoảng cách có thể tiếp tục thu hẹp khi tăng ngân sách.
- **Phạm vi đánh giá:** bộ đánh giá tập trung vào các tác vụ hiểu và biểu diễn câu; một số năng lực như nhận dạng thực thể có tên hay ứng dụng ngữ cảnh dài chưa được khảo sát, và hai tác vụ Retrieval–Reranking mới chạy một lần nên chưa ước lượng được phương sai.
- **Ngữ cảnh dài:** pha mở rộng ngữ cảnh lên 4.096 token mới được kiểm chứng bằng độ đo nội tại (pseudo-perplexity theo độ dài chuỗi, Mục 4.5.3); chưa có đánh giá trên các tác vụ đòi hỏi ngữ cảnh dài như hỏi đáp đa đoạn hay truy xuất tài liệu.
- **Tokenizer:** từ vựng kế thừa từ ViDeBERTa hoạt động ở mức âm tiết; ảnh hưởng của việc không tách từ (so với PhoBERT dùng dữ liệu đã tách từ) chưa được phân tích riêng.

5.3 Hướng phát triển

Từ các kết quả và hạn chế trên, chúng tôi đề xuất năm hướng phát triển:

1. **Mở rộng ngân sách tiếp tục tiền huấn luyện:** thêm dữ liệu và huấn luyện lâu hơn, nhằm thu hẹp và tiến tới vượt các mô hình họ PhoBERT trên các tác vụ ứng dụng.

2. **Trộn lại một phần dữ liệu tiếng Anh khi tiếp tục tiền huấn luyện:** giữ lại một tỷ lệ nhỏ (khoảng 5–10%) dữ liệu tiếng Anh – ngôn ngữ nguồn của NeoBERT – trong hỗn hợp huấn luyện ở Giai đoạn 3 như một cơ chế *phát lại* (replay), nhằm bảo toàn biểu diễn ở các lớp trong và giảm mất thông tin. Ibrahim và cộng sự [30] cho thấy chỉ cần phát lại một phần nhỏ dữ liệu cũ là đủ để chống mất thông tin khi phân phối dữ liệu dịch chuyển; đáng chú ý hơn, Elhady và cộng sự [29] chỉ ra rằng trong chuyển đổi ngôn ngữ, việc giữ lại dữ liệu tiếng Anh tuy *không* cải thiện perplexity nhưng lại quyết định sự xuất hiện của năng lực trên các tác vụ ứng dụng – một tín hiệu mà mất mát MLM đơn thuần không phản ánh. Đề tài dùng khởi tạo SALT và giai đoạn căn chỉnh với phần thân đóng băng làm cơ chế bảo vệ chống mất thông tin, còn phát lại tiếng Anh là một cơ chế hỗ trợ tự nhiên đáng để khảo sát tiếp.
3. **Khai thác ngữ cảnh dài trên các tác vụ ứng dụng:** xây dựng và đánh giá trên các tác vụ đòi hỏi ngữ cảnh dài (hỏi đáp đa đoạn, truy xuất tài liệu) để bổ sung cho kiểm chứng nội tại bằng pseudo-perplexity ở Mục 4.5.3; khi cần vượt trần 4.096 token, các kỹ thuật nội suy RoPE như YaRN [14] là ứng viên tự nhiên.
4. **Mở rộng bộ đánh giá:** bổ sung nhận dạng thực thể có tên, truy xuất trên kho ngữ liệu quy mô lớn hơn và lặp lại nhóm tác vụ biểu diễn câu qua nhiều lần chạy, nhằm đánh giá toàn diện và tin cậy hơn chất lượng biểu diễn của mô hình ở vai trò một Encoder đa dụng.
5. **Tổng quát hóa quy trình:** áp dụng quy trình của đề tài cho các ngôn ngữ ít tài nguyên khác hoặc các kiến trúc nền khác, qua đó kiểm chứng tính tổng quát của các cải tiến (tập điểm neo theo cặp dịch, khởi tạo riêng lớp đầu ra, căn chỉnh với phần thân đóng băng) ngoài phạm vi cặp Anh–Việt và NeoBERT.

Tài liệu tham khảo

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [2] Y. Liu, M. Ott, N. Goyal, *et al.*, “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [3] A. Dubey *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [4] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [5] L. Le Breton, Q. Fournier, M. El Mezouar, J. X. Morris, and S. Chandar, “NeoBERT: A next-generation BERT,” *arXiv preprint arXiv:2502.19587*, 2025.
- [6] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042.
- [7] C. D. Tran, N. H. Pham, A. Nguyen, T. S. Hy, and T. Vu, “ViDeBERTa: A powerful pre-trained language model for Vietnamese,” in *Findings of the Association for Computational Linguistics: EACL 2023*, 2023, pp. 1071–1078.

- [8] S. Lee, S. Hong, H. Moon, and H. Lim, “Semantic aware linear transfer by recycling pre-trained language models for cross-lingual transfer,” *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, arXiv:2505.10945.
- [9] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [10] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” *arXiv preprint arXiv:2402.01613*, 2024.
- [11] B. Warner, A. Chaffin, B. Clavié, *et al.*, “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” *arXiv preprint arXiv:2412.13663*, 2024.
- [12] Y. Levine, N. Wies, O. Sharir, H. Bata, and A. Shashua, “The depth-to-width interplay in self-attention,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2006.12467, 2020.
- [13] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, 2024, arXiv:2104.09864.
- [14] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “YaRN: Efficient context window extension of large language models,” *arXiv preprint arXiv:2309.00071*, 2023.
- [15] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” *arXiv preprint arXiv:1606.08415*, 2016.
- [16] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020.

- [17] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:1910.07467, 2019.
- [18] R. Xiong, Y. Yang, D. He, *et al.*, “On layer normalization in the transformer architecture,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [19] G. Penedo, Q. Malartic, D. Hesslow, *et al.*, “The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only,” *arXiv preprint arXiv:2306.01116*, 2023.
- [20] A. Wettig, T. Gao, Z. Zhong, and D. Chen, “Should you mask 15% in masked language modeling?” In *Proceedings of the 17th Conference of the European Chapter of the ACL (EACL)*, 2023.
- [21] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, arXiv:1711.05101, 2019.
- [22] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *International Conference on Learning Representations (ICLR)*, arXiv:1608.03983, 2017.
- [23] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations (ICLR)*, arXiv:2307.08691, 2024.
- [24] K. Ethayarajh, “How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [25] A. Ormazabal, M. Artetxe, G. Labaka, A. Soroa, and E. Agirre, “Analyzing the limitations of cross-lingual word embedding mappings,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 4990–4995.

- [26] J. Gao, D. He, X. Tan, T. Qin, L. Wang, and T.-Y. Liu, “Representation degeneration problem in training natural language generation models,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [27] S. Gururangan, A. Marasović, S. Swayamdipta, *et al.*, “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [28] W. de Vries and M. Nissim, “As good as new. how to successfully recycle English GPT-2 to make models for other languages,” in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021.
- [29] A. Elhady, E. Agirre, and M. Artetxe, “Emergent abilities of large language models under continued pretraining for language adaptation,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, arXiv:2506.00288, 2025.
- [30] A. Ibrahim, B. Thérien, K. Gupta, *et al.*, “Simple and scalable strategies to continually pre-train large language models,” *Transactions on Machine Learning Research (TMLR)*, 2024, arXiv:2403.08763.
- [31] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, *et al.*, “MiniCPM: Unveiling the potential of small language models with scalable training strategies,” *arXiv preprint arXiv:2404.06395*, 2024.
- [32] A. Hägele, E. Bakouch, A. Kosson, L. Ben Allal, L. Von Werra, and M. Jaggi, “Scaling laws and compute-optimal training beyond fixed training durations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2405.18392, 2024.
- [33] B. Minixhofer, F. Paischer, and N. Rekabsaz, “WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer

- of monolingual language models,” in *Proceedings of NAACL-HLT*, 2022, pp. 3992–4006.
- [34] K. Dobler and G. de Melo, “FOCUS: Effective embedding initialization for monolingual specialization of multilingual models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
 - [35] Y. Liu, P. Lin, M. Wang, and H. Schütze, “OFA: A framework of initializing unseen subword embeddings for efficient large-scale multilingual continued pretraining,” in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.
 - [36] A. F. T. Martins and R. F. Astudillo, “From softmax to sparsemax: A sparse model of attention and multi-label classification,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.
 - [37] T. Vu, D. Q. Nguyen, D. Q. Nguyen, M. Dras, and M. Johnson, “VnCoreNLP: A Vietnamese natural language processing toolkit,” in *Proceedings of NAACL-HLT 2018: Demonstrations*, 2018, pp. 56–60.
 - [38] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” in *International Conference on Learning Representations (ICLR)*, arXiv:2111.09543, 2023.
 - [39] M. Junczys-Dowmunt, R. Grundkiewicz, T. Dwojak, *et al.*, “Marian: Fast neural machine translation in C++,” in *Proceedings of ACL 2018, System Demonstrations*, 2018.
 - [40] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 5, pp. 135–146, 2017.

- [41] E. Grave, P. Bojanowski, P. Gupta, A. Joulin, and T. Mikolov, “Learning word vectors for 157 languages,” in *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC)*, 2018.
- [42] C. Meister, W. Stokowiec, T. Pimentel, L. Yu, L. Rimell, and A. Kuncoro, “A natural bias for language generation models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, arXiv:2212.09686, 2023.
- [43] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [44] T. Nguyen, C. V. Nguyen, V. D. Lai, *et al.*, “CulturaX: A cleaned, enormous, and multilingual dataset for large language models in 167 languages,” in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, arXiv:2309.09400, 2024, pp. 4226–4237.
- [45] K. V. Nguyen, S. Q. Tran, L. T. Nguyen, T. V. Huynh, S. T. Luu, and N. L.-T. Nguyen, “VLSP 2021 – ViMRC challenge: Vietnamese machine reading comprehension,” *VNU Journal of Science: Computer Science and Communication Engineering*, 2022, arXiv:2203.11400; introduces UIT-ViQuAD 2.0.
- [46] A. Conneau, R. Rinott, G. Lample, *et al.*, “XNLI: Evaluating cross-lingual sentence representations,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [47] T. V. Huynh, K. V. Nguyen, and N. L.-T. Nguyen, “A new benchmark dataset and mixture-of-experts language models for adversarial natural language inference in Vietnamese,” *Expert Systems with Applications*, 2024, arXiv:2406.17716; ViANLI dataset.

- [48] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018, pp. 353–355.
- [49] K. V. Nguyen, V. D. Nguyen, P. X. V. Nguyen, T. T. H. Truong, and N. L.-T. Nguyen, “UIT-VSFC: Vietnamese students’ feedback corpus for sentiment analysis,” in *Proceedings of the 10th International Conference on Knowledge and Systems Engineering (KSE)*, 2018.
- [50] V. A. Ho, D. H.-C. Nguyen, D. H. Nguyen, *et al.*, “Emotion recognition for Vietnamese social media text,” in *Proceedings of the 16th International Conference of the Pacific Association for Computational Linguistics (PACLING 2019)*, Springer CCIS; arXiv:1911.09339; UIT-VSMEC dataset, 2020.
- [51] L. T. Nguyen, K. V. Nguyen, and N. L.-T. Nguyen, “Constructive and toxic speech detection for open-domain social media comments in Vietnamese,” in *Proceedings of the 34th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems (IEA/AIE)*, arXiv:2103.10069; UIT-ViCTSD dataset, 2021.
- [52] A. Warstadt, A. Singh, and S. R. Bowman, “Neural network acceptability judgments,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 7, pp. 625–641, 2019, CoLA dataset.
- [53] P.-T. Nguyen, X.-L. Vu, T.-M.-H. Nguyen, V.-H. Nguyen, and H.-P. Le, “Building a large syntactically-annotated corpus of Vietnamese,” in *Proceedings of the Third Linguistic Annotation Workshop (LAW III)*, Vietnamese Treebank; source of UD_Vietnamese-VTB, 2009.
- [54] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, “SemEval-2017 task 1: Semantic textual similarity multilingual and cross-lingual focused evaluation,” in *Proceedings of the 11th*

International Workshop on Semantic Evaluation (SemEval-2017), 2017, pp. 1–14.

- [55] A. Conneau, K. Khandelwal, N. Goyal, *et al.*, “Unsupervised cross-lingual representation learning at scale,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451.
- [56] J. Salazar, D. Liang, T. Q. Nguyen, and K. Kirchhoff, “Masked language model scoring,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2699–2712.

Phụ lục A

Bộ lọc chính tả tiếng Việt cho trích xuất cặp dịch Anh–Việt

Phụ lục này mô tả chi tiết bộ lọc chính tả được dùng ở Mục 3.4.3 để bảo đảm các ứng viên đều là từ tiếng Việt hợp lệ. Một token chỉ được giữ lại khi vượt qua *toàn bộ* các quy tắc sau:

1. **Ký tự tiếng Việt:** loại mọi token chứa f, j, w, z – các ký tự không thuộc bảng chữ cái tiếng Việt.
2. **Độ dài:** tối đa 7 ký tự cho một âm tiết.
3. **Một cụm nguyên âm:** token phải chứa *đúng một* cụm nguyên âm liền mạch (kể cả nguyên âm mang dấu thanh).
4. **Quy tắc phụ âm đầu:**
 - k, gh, ngh bắt buộc đứng trước các nguyên âm hàng trước $i, e, ê, y$;
 - ngược lại, c, g, ng không được đứng trước các nguyên âm này;
 - q luôn đi kèm u ;
 - p không đứng một mình làm phụ âm đầu (loại các dạng vay mượn như $pô, pin$).

5. **Tổ hợp không xuất hiện trong tiếng Việt:** loại các chuỗi không xuất hiện trong âm tiết tiếng Việt như *ea, ee, oo, ii, uu, ion, eon*.
6. **Nguyên âm bắt buộc âm cuối:** các nguyên âm ngắn *ă, â* (và các tổ hợp *iê, uô, ươ*) bắt buộc phải có phụ âm cuối đi kèm, không được đứng cuối âm tiết (loại các mảnh từ như *nhấ, tầ, nghiê*).
7. **Phụ âm đầu/cuối hợp lệ:** phụ âm đầu phải thuộc tập $\{\emptyset, b, c, ch, d, đ, g, gh, gi, h, k, kh, l, m, n, ng, ngh, nh, ph, q, qu, r, s, t, th, tr, v, x\}$; phụ âm cuối phải thuộc tập $\{\emptyset, c, ch, m, n, ng, nh, p, t\}$.
8. **Quy tắc thanh-phụ âm cuối:** âm tiết kết thúc bằng *c, ch, p, t* bắt buộc mang thanh sắc hoặc thanh nặng (loại các dạng sai như *sec, nhac, mít*).
9. **Danh sách loại trừ:** một danh sách nhỏ các chuỗi thỏa cấu trúc âm tiết nhưng là nhiễu trong thực tế (ví dụ mảnh phiên âm, từ cổ ít dùng) được loại thủ công.

Ngoài ra, ở Tầng 3 của quá trình trích xuất cặp dịch (Mục 3.4.3), một danh sách chặn (blacklist) gồm các từ chức năng và từ dễ gây nhập nhằng ngữ nghĩa của tiếng Việt (*các, những, được, để, thế, cái, ...*) được áp dụng trước khi dịch, nhằm tránh tạo ra các cặp neo kém tin cậy từ những từ có phân bố ngữ nghĩa rộng.

Phụ lục B

Định nghĩa hình thức các độ đo đánh giá

Phụ lục này trình bày định nghĩa hình thức của các độ đo được dùng ở Mục 4.3. Các định nghĩa dạng văn xuôi đã được nêu trong phần thân; phần dưới đây bổ sung công thức tường minh để tiện tra cứu.

Với hỏi đáp đọc hiểu, F1 ở cấp token là trung bình điều hòa của độ chính xác P và độ bao phủ R , được tính giữa câu trả lời dự đoán và đáp án chuẩn, như Công thức (B.1):

$$F1 = \frac{2PR}{P + R}. \quad (B.1)$$

Hệ số tương quan Matthews cho phân loại nhị phân, tính từ các đại lượng dương tính thật (TP), âm tính thật (TN), dương tính giả (FP) và âm tính giả (FN), như Công thức (B.2):

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (B.2)$$

Với truy hồi và xếp hạng lại, gọi $rel_i \in \{0, 1\}$ là mức liên quan của ngữ cảnh ở thứ hạng i và $rank_q$ là thứ hạng của ngữ cảnh đúng đầu tiên cho

truy vấn q , hai độ đo được xác định như Công thức (B.3):

$$\text{nDCG}@k = \frac{1}{Z} \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (\text{B.3})$$

trong đó Z là hệ số chuẩn hóa (DCG của thứ hạng lý tưởng) đưa nDCG về đoạn $[0, 1]$, và Q là tập truy vấn đánh giá.